

Gungnir: overview

Architecture, decisions and their reasoning, Rust design, and the algorithms — each mapped to the mission it serves

Wayne Roessling · github.com/WayneRoessling/gungnir · main at 0cb9f0a, 2026-09-10 · every figure sourced to a file, a CI run, or a table row

Contents

Contents	1
Part I, Chapter 1 -- How to use this guide	4
The three-sentence pitch	4
The one honest status sentence	4
How the guide is organized	4
The rule: name the alternative and the condition that ruled it out	4
Canonical numbers	5
Chapter 2 -- The problem and the mission set	5
2.1 The problem, in the repository's own framing	5
2.2 The ten mission threads	6
2.3 The ten vignettes	8
2.4 Measures of effectiveness with targets	8
2.5 The roles and the authority matrix	9
Chapter 3 -- The five engineering scenarios	9
What each scenario is built to break	10
Truth-scored replay	10
The ten plan-07 sample sets in CI	11
The statistical self-check (built and gated)	11
What the swarm shows about limits	12
Part II, Chapter 4 -- The layer model and the 51 crates	12
4.1 The layers	12
4.2 The one-way rule, as written	12
4.3 How the rule is enforced	13
4.4 The 51 crates by layer	13
4.5 Three placement corrections	16
4.6 What a technical lead will ask	16
Chapter 5 -- Deployment profiles, the embedded/remote switch, and resilience	17
5.1 One crate set, three profiles	17
5.2 The switch: two traits and one configuration tag	17
5.3 The remote side: gungnir-remote	18
5.4 The node side: gungnir-node and the v2 routes	18
5.5 Failover, store-and-forward, reconciliation	18

5.6 Durability per profile (D-04) and the fsync defect	19
5.7 What the tests prove, and the race PR #79 fixed	19
5.8 Status, and the documentation defect to explain first	19
Chapter 6 -- Two GPU contexts and the point-cloud path	20
6.1 Two contexts, by decision	20
6.2 The CPU reference	20
6.3 The WGSL pipeline (item 117 in PR #46, item 119 in PR #53)	21
6.4 GAP-098 and D-41	21
6.5 The runner and D-10 as amended	21
6.6 Exactly what is gated where	21
Chapter 7 -- Security architecture	22
7.1 Two kinds of caller (D-02)	22
7.2 Custody: a provider with no getter	23
7.3 Journal sealing with the edge inverted	24
7.4 Audit	24
7.5 Two defects the tests caught	24
7.6 What is not built, and what the repository says twice	24
Chapter 8 -- The architecture process: TOGAF ADM, UAF, principles, contracts, generators	25
8.1 The two frameworks, for a non-technical reader	25
8.2 How the ADM was tailored	25
8.3 The UAF view set	26
8.4 Principles AP-01 to AP-17 and contracts C-01 to C-17	27
8.5 The compliance assessment	27
8.6 The register, decision ledger, and roadmap generator	28
8.7 The DoDAF cross-reference	28
Part III, Chapter 9 -- Rust design decisions by layer and crate	28
9.1 Traits as facades and the embedded/remote switch	28
9.2 Newtypes and type-level invariants	29
9.3 Error philosophy	30
9.4 Ownership at boundaries	30
9.5 Concurrency	31
9.6 Lint and toolchain governance	31
9.7 Dependency governance	31
9.8 The testing pyramid	32
9.9 Performance discipline	32
Chapter 10 -- Twelve Rust design questions this repository answers from its own code	33
How to use this chapter	33
The twelve questions	33
Part IV, Chapter 11 -- The decision ledger, D-01 to D-43, mapped to mission	35

How to read the ledger	35
Topology, scope, and deployment	35
Security, identity, and custody	36
Data, interop, and standards	38
Algorithms, oracles, and gates	39
UX and roles	40
Governance, hosting, and licensing	41
ML and GPU	41
Decisions taken against the recommended default	42
Ledger rows a reviewer will find stale	42
Chapter 12 -- The design notes, DN-01 to DN-30	43
What a design note is	43
The thirty notes	43
Invariants the type system holds	45
Design versus reality: the amendments and the §3a corrections	46
Part V, Chapter 13 -- The algorithms	47
How to read this chapter	47
Estimation	47
Association	50
Random finite sets	51
Track management and fusion	52
The bearing path	53
The pipeline	54
Decision support	55
Point clouds and geometry	56
Time and data	57
Part VI, Chapter 14 -- Process, defects, what is not built, and likely questions	59
14.1 Timeline	59
14.2 Trust tiers and what "signed by the owner" means	59
14.3 Defects the process caught in its own work	60
14.4 What is not built	61
14.5 Fifteen likely questions	62

Part I, Chapter 1 -- How to use this guide

The three-sentence pitch

Gungnir is a Rust command-and-control workspace for air defense and counter-UAS: 50 crates (51 with the excluded fuzz crate) spanning sensor ingest, multi-target tracking, identification, policy, intercept planning, a desktop operator application, and a headless service node, under one one-way dependency graph that a test checks edge by edge against ARCHITECTURE.md (gungnir-app/tests/dependency_graph.rs). The tracking core -- Kalman, EKF and UKF, IMM, particle, square-root, Jonker-Volgenant, JPDA, MHT, GM-PHD, GM-CPHD, LMB, and track-to-track fusion -- is differentially tested from fixtures under testdata/oracles/: against the pinned libraries filterpy 1.4.5, Stone Soup 1.9.1 and scipy 1.18.1 where one carries the algorithm, and against project-authored derivations where none does (MHT hand-derived, GM-CPHD and LMB in-house, track-to-track fusion hand-derived, GM-PHD against a numpy Vo-Ma recursion), each substitution recorded in the module, fixture and verification table; the out-of-sequence async pipeline is model-checked under loom. I am the owner, architect, technical lead, and reviewer-signer: I write the specifications and pass criteria, an agent drafts, an adversarial review and structural CI gates follow, and no change to a human-owned crate is done until I sign it (docs/agentic-workflow.md); the status sentence below names the ones still waiting, and the record shows my reviews catching defects the agent's tests missed.

The one honest status sentence

As of 2026-09-09 (main at ee12f7e), the tracking core apart from the LMB filter, the pipeline that runs (constant-velocity Kalman with GNN, IMM selectable per DN-28), the ASTERIX, ADS-B, SAPIENT, and MISB adapters, mutual TLS and key custody, the v2 transport and mid-session failover are **built and gated** in CI, and 20 of 21 operator panels **draw real content**, each checked by a headless egui probe asserting it painted non-empty text -- 17 against specific sentences, with only PN-16's probe a verification-table gate; the LMB filter, the dense-group wiring, DN-30, the cloud-KMS ManagedService, the Cat 129 adapter, the fires three-state friendly check (GAP-090), and the loom model checks are **built, unsigned** (ARCHITECTURE.md section 10; the gap register); STANAG 4676, Cat 048 encode, the delta-GLMB, gRPC, the assistant panel (PN-19), trained models, the Cursor-on-Target codec (DN-25's codec half, GAP-091), 3D Tiles streaming, and the GPU hardware gate are **not built**; and no real sensor, no real deployment, and no usability session has run. CI on ee12f7e itself is red on one gungnir-node harness race (tests/account_provisioning.rs:50, BrokenPipe); cite same-day run 34361396023 on ce609437 for the green figure.

This document's body is the audit as it stood at that commit; every file, line number, test count, and status word below is stated as of ee12f7e/2026-09-09 and has not been re-verified for this pass. Only the masthead, this page's canonical-numbers table, and the framing text have been refreshed to main at 0cb9f0a, 2026-09-10 -- one day and seven commits later, none of which touched the systems described here (they were UAF/Sparx-EA export tooling and a license-header pass). Re-verifying every embedded claim against the current tree would mean redoing the audit; treat any figure below as accurate to within a day.

How the guide is organized

Part I orients: this chapter, the mission set (MT-01..MT-10), and the five engineering scenarios. Part II is architecture: layers and the one-way rule, deployment profiles, the two GPU contexts, security, and the TOGAF and UAF process. Part III is Rust engineering by layer and crate, plus twelve Rust questions answerable from this code. Part IV is the decisions (D-01..D-43) and design notes (DN-01..DN-30). Part V explains each algorithm in six steps: problem, idea, math, variant, mission use, verification. Part VI is timeline, signature discipline, defects the process caught in its own work, what is not built or unsigned as of 2026-09-09, and honest answers to likely questions.

Note: if time is short, read Chapter 14 first. It lists the weaknesses a technical reviewer is most likely to find in the public repository, with source and answer.

The rule: name the alternative and the condition that ruled it out

Every design claim in this guide is stated as a choice, with the alternative and the condition that ruled it out, as the repository's decision ledger does. Three examples: D-43 made Assignment::total_cost an Option<f64> rather than refusing the input, erroring on a non-finite total, or documenting that no finite total was ever promised, because fuzzing found Ok with -inf on an all-finite matrix and numerical stability is human-owned (gungnir-association/src/assignment.rs). The IMM oracle is filterpy's IMMEstimator, not the Stone Soup IMM the verification table planned, because Stone Soup 1.9.1 has no IMM at all -- no such symbol exists in the pinned package; the substitution is recorded in the module, fixture, and table, though the inspection is not reproducible from the repository. The central harness gungnir-oracle was designed and never populated -- its own doc comment says so and cites GAP-082, though the register's GAP-082 row is a closed todo! () item and the finding itself sits under GAP-061; the distributed tests/*_diff.rs suite replaced it and the stub stays a named NotImplementeded.

When a decision went against the recommended default, the guide says so and gives the owner's reason. When the repository contradicts itself, the guide names the defect rather than picking a side: docs/architecture.md records the GAP-067 walk promoting section-2 rows to Specified, while the verification table's section-2 preamble still says "None of these rows is a pass/fail gate yet" and the register keeps GAP-067 Open.

Canonical numbers

Quantity	Value	Source
Crates	50 workspace members plus gungnir - fuzz, excluded (51)	Cargo.toml
Rust source	399 first-party .rs files, 156,094 lines including tests	census over the tree at ee12f7e
Tests in the CI nextest job	1,863 passed, 3 skipped ([ignore]d cloud-keystore tests), 112.7 s; doctests, the frame-budget gate, loom, miri, GPU and fuzz run as separate steps or workflows	run 34361396023, main ce609437, 2026-09-09
Test attributes	1,832 #[test] attributes and a further 54 #[tokio::test] (4 plain, 50 flavor = "multi_thread"); the two greps are disjoint, so 1,886 attributes against nextest's 1,866 tests is not a contradiction -- ignored tests, the excluded fuzz crate and shared helpers account for it	grep over gungnir - *
Oracle suite	17 *_diff.rs files in 9 crates; 14 generators; 20 fixtures	gungnir-*/tests/, testdata/oracles/
Gated verification rows	39 in section 1	docs/verification-capability-table.md
Dependency edges	169 among the 50 members, acyclic, one-way	gungnir-app/tests/dependency_graph.rs
unsafe, todo!(), unwrap/expect	0 / 0 / 0 outside tests, main, benches, verifier crates; source scans, not a forbid, and unsafe is measured rather than scanned	no_reachable_todo.rs (todo!()), architecture_compliance.rs (unwrap/expect); zero unsafe is a grep, armed only by miri.yml's diff trigger
Gap register	104 entries: 77 closed, 20 in progress, 6 open, 1 planned	docs/mission/gap-analysis/gap-register.md
Decisions and design notes	D-01..D-43; DN-01..DN-30	decisions-needed.md; docs/design/
CI workflows	8	.github/workflows/
UAF and TOGAF	58 views (25 generated), 7 matrices; 23 documents; 17 principles, 17 contracts, 7 machine-checked	docs/architecture/uaf/, docs/architecture/togaf/
History	257 commits, all by Wayne Roessling, 178 with a Claude co-author trailer, 2026-09-07..10; program documented from 2026-09-04	git log; ARCHITECTURE.md section 10
Toolchain and lints	Rust 1.98; clippy::all = deny, pedantic warn	rust-toolchain.toml; Cargo.toml [workspace.lints]
License	AGPL-3.0-or-later with section 7 terms, commercial license and CLA (D-34)	LICENSE, LICENSE-ADDITIONAL-TERMS.md, CLA.md

Chapter 2 -- The problem and the mission set

2.1 The problem, in the repository's own framing

docs/mission/mission-analysis.md §1 sets one rule for everything under docs/mission/: every capability, gap, architecture view, screen, test track, and learned model must "trace back to something a person in a command post has to do." The level is tactical and low operational: a C2 desktop and a service node, a small team of operators, a chain of authority above them. The vocabulary is function-based (detect, track, identify, assess, decide, engage, assess effects) with a joint/NATO mapping in docs/mission/glossary.md §1, and every source is unclassified.

The mission set (mission-analysis.md §2):

Priority	Mission	Repository's stated reason
Lead	Integrated air defense and counter-UAS for a defended-asset list	Tightest timelines; the cost asymmetry between cheap attackers and expensive interceptors that "a good C2 can partly redress"
Supporting	Maritime domain awareness and port defense	Shares sensors, geography, and command posts with coastal air defense

Priority	Mission	Repository's stated reason
Supporting	Land picture and fires cueing	Same track, identity, and policy machinery; the FPV threat makes force protection a counter-UAS problem
Cross-cutting	Intelligence	Feeds identification and prioritization in every domain
Cross-cutting	Planning and battle management	Sets the conditions the live missions run under

The lead-domain statement in docs/mission/air-defense-and-counter-uas.md §1 is the shortest honest definition of the product:

Protect a prioritized list of defended assets in a sector against air threats from small multirotors to ballistic missiles by maintaining a single, honest air picture; identifying and prioritizing threats; recommending the cheapest adequate engagement within policy; presenting it to the human who holds engagement authority; recording the decision; handing off to the effector system; and assessing the result, while never engaging anything without a recorded human decision and never presenting stale or uncertain data as fresh and certain.

Two design rules fall out of that sentence. **Recommendation-only** is contract C-01: gungnir-policy/src/authority.rs states that no verdict is ever "permitted to act" and the most either engine returns is PolicyVerdict::RequiresHumanApproval; gungnir-app/tests/no_execution_without_decision.rs checks that execution is constructed only behind DecisionRecord::is_actionable, statically and at runtime. Built and gated. **Picture honesty** is MOE-06, target 0: gungnir_fusion_async::PIPELINE_IMPLEMENTED gates TrackingService::is_healthy(), and Quality.is_stale is checked before any scoring in gungnir-assessment. Built and gated. The latency budgets come from air-defense-and-counter-uas.md §4 and maritime.md §2: a propeller drone at 40 km gives about 13 minutes, a cruise missile at 30 km about 110 seconds, an FPV 10 to 120 seconds, a USV at 40 knots from 10 km about 8 minutes.

One documentation defect to state first: every file under docs/mission/ is dated "first draft, 2026-09-04" and is stale in places. mission-threads.md still marks "(gap)" on MT-01 step 7, MT-02 step 2, MT-08 step 1, and MT-09 steps 1 and 3, all built and tested since; roles-and-stakeholders.md §1 says "not yet" for three roles in code since 2026-09-05; air-defense-and-counter-uas.md §9 says PIPELINE_IMPLEMENTED is false, true since 2026-09-06 (GAP-011). Where a mission document and a test disagree, the test is the truth.

2.2 The ten mission threads

docs/mission/mission-threads.md writes each thread as steps marked S (system), H (human), or S+H (the system prepares, a person completes). Each thread below is given twice: the document's specified steps in the present tense, then, after "Exercises", the code that would run them at ee12f7e with its status word. The status words describe the code, not the document, and no step has ever run on a real sensor. MT-01 to MT-06 share one spine, named once: the gungnir-ingest gateway (malformed payloads quarantined); gungnir-fusion-async (source-time reorder buffer, per-track Joseph-form Kalman filter or CV/CT IMM by baseline, chi-square gate, GNN over Jonker-Volgenant assignment, gungnir-track lifecycle); gungnir-assessment; the Bellman/DP allocator in gungnir-allocation through gungnir-intercept-service; the gungnir-policy chain; the gungnir-command queue; HTTP handoff and warnings (endpoint_delivery.rs); the gungnir-store journal. The spine is built and gated; its human-owned parts are signed except DN-30's measurement-noise-from-the-baseline change to gungnir-fusion-async, which is built and unsigned (docs/design/README.md, DN-30 row). Built and gated standalone but not on the running path: EKF, UKF, particle, square-root, RTS, JPDA, MHT, track-to-track fusion and registration; LMB is built, unsigned.

MT-01 One-way attack drone raid against a defended-asset list

Tens of propeller drones arrive over one to two hours, announced by a peer, acoustic nodes, or long-range radar; the air operator, supervisor, sensor manager, and commander (priorities) work the queue while fire groups engage. The system tracks across sources at different rates, classifies, scores against the asset list, recommends the cheapest adequate effector within readiness, geometry, and geofences, queues it for the authority, hands off, warns, assesses, and journals. Exercises the whole spine, noisy-OR evidence fusion in gungnir-identification, acoustic bearings through offer_bearing (initiate nothing), and under saturation the dense-group count in gungnir_fusion_async::dense_group over gungnir-rfs's PHD/CPHD (the GM-PHD filter signed 2026-09-06 and the CPHD 2026-09-09; the dense-group wiring built, unsigned, off by default for a measured reason -- 6.3 ms added per epoch for a 20-target raid at the default 400-component cap, against a 4 ms per-frame budget, docs/performance-budgets.md -- and not projected to the service); live acoustic classification evidence is not built. Scenario 4, TT-01.

MT-02 Mixed salvo of cruise missiles, drones, and decoys

A peer launch warning or a low, fast local track opens the thread, and steps 3 to 6 must complete "in under thirty seconds for a missile detected at 30 km"; the area-layer authority accepts pre-delegated cases while the supervisor may hold. The

system separates fast tracks by kinematics, predicts impact through horizon dropouts, scores missiles first, recommends the area layer for missiles and the point layer for drones, and re-plans for leakers. Exercises the peer launch-warning path (an alert, never a track), FilterPredictor and ConstantVelocityPredictor, AuthorityPolicy pre-delegated cases, and the intercept service's last-good-plan rule, all built and gated; signature-class evidence is not built, and MOP-07's 500 ms has no gate. Scenarios 1 and 4, TT-02.

MT-03 Small UAS over a protected site

A multirotor or a fiber-optic FPV over a site, ten seconds to two minutes; the site operator, site defense cell, sensor manager, and supervisor act. The system correlates RF, radar, acoustic, and visual reports into one track, classifies, assesses approaching or loitering, recommends jam, interceptor drone, small arms, or observe with geofence checks on jamming, and on ISR raises the alert state and re-tasks sensors. Exercises the SAPIENT adapter for human, acoustic, and passive-RF nodes (built and gated, signed 2026-09-08; it reads the protobuf-JSON mapping over TCP, not native protobuf), ASTERIX Cat 205 bearings (signed 2026-09-09) and Cat 129 reports (Cat 129 built, unsigned), bearing crossing in gungnir-coord, GeofencePolicy, and the Loitering detector, built and gated; link-controlled versus autonomous discrimination is not built. TT-03; no engineering scenario.

MT-04 Uncrewed surface vessel attack on a port or anchored ship

About eight minutes from a 10 km detection at 40 knots, identification in the first two; the port defense operator and authority, patrol craft, a helicopter, a shore unit, and the port authority act. The system tracks through clutter and dropouts, cues cameras to low-confidence inbound tracks, identifies by no AIS, speed, heading, and group behavior, computes CPA to protected ships, and recommends patrol craft, helicopter, shore effector, barrier closure, or ship movement inside lane geofences. Exercises the coastal-radar clutter model (Pd 0.8, 2.0 false alarms per scan), AIS through gungnir-interop and cooperative_identity.rs, the CooperativeIdentityLost and CooperativeMismatch detectors, CPA in prediction.rs (DN-02), and the static hazard layer (DN-14), built and gated; clutter handling is gate, JV, and lifecycle only, and the cross-domain risk of a missile-armed USV to the helicopter is not built. Scenario 2, TT-04.

MT-05 Surface picture compilation

Continuous, hundreds of tracks, no trigger; the port defense operator and sensor manager keep the picture that authorities and peer command posts receive. The system fuses radars, AIS, cameras, patrol, and peer reports, flags AIS-off, inconsistent speed, loitering, and spoofing, and exchanges the picture. Exercises AIS-to-track association (a report with no nearby track is counted, not invented), the gungnir-analytics anomaly detectors (built and gated; no detector alters a track), and the gungnir-api v2 snapshot and event stream over opt-in mutual TLS with a required client certificate, verified against a real tokio-rustls handshake using test-generated certificates and no real certificate authority; the exchange producer is in progress (GAP-065), STANAG 4676 and the planned anomaly model (GAP-080) are not built, and MOP-04's false-track rate has no gate. Scenario 2, TT-05 (139 entities).

MT-06 Convoy and battery tracking with cueing of fires

Minutes to hours of intermittent land tracks, opened by ISR tasking or acoustic detection of artillery fire; the land operator, intelligence analyst, fires authority, ISR operators, and airspace control act (D-07 put fires in the first release). The system tracks vehicles through stops and cover, registers sensors against each other, carries identity across sorties, recommends a fires task deconflicted against friendly positions, airspace, and no-fire areas, hands off with provenance, and records BDA. Exercises the MISB ST 0601 adapter (signed 2026-09-09), the reorder buffer under the isr_video model (35 percent out of order, injected bias), gungnir-identity similarity and lineage, and FiresDeconflictionPolicy (signed; a check that cannot be evaluated does not pass); registration in gungnir-track-fusion is built, gated, and signed but not wired (GAP-013), and the three-state friendly check (GAP-090) is built, unsigned. Scenario 3, TT-06.

MT-07 Sensor management under electronic attack

GNSS denial, link jamming, radar interference, or a lost sensor; the sensor manager and supervisor act, and the commander accepts an uncovered asset. The system correlates health, ingest gaps, clock skew, and track-quality collapse into one alert, recomputes coverage, recommends re-tasking, and marks tracks degraded so the planner never allocates on them. Exercises ClockSkewEstimator, the FeedSilent and FeedImplausible detectors, gungnir-observability correlation (never more incidents than raw alerts), gungnir-analytics/src/coverage.rs, SensorPlanner, and per-class staleness aging (GAP-012), built and gated; "accept coverage gap" has no coarse action in role_permits yet, and MOP-14's 10 s is not measured. Scenarios 3 and 5, TT-07.

MT-08 Collection management and identification evidence fusion

Continuous intelligence work by the intelligence analyst, commander, sensor manager, and peers. A person states a requirement; the system turns it into tasking with its coverage cost, collects with provenance, fuses evidence into identities with confidence, maintains the order of battle across sessions, and disseminates with releasability. Exercises gungnir-app/tests/requirements.rs (a sensor acknowledging a task is not an answer), EvidenceFusionEngine, order_of_battle.rs, and releasability (GAP-062), built and gated; the CAP-2.12 concurrence criterion waits on operator sessions (GAP-057), cross-session analysis products are not built, and IFF decoding is deferred (D-09). Scenario 1, TT-08

(VG-08's own "Exercised by" line omits the scenario the mapping summary gives it).

MT-09 Defended-asset planning, laydown, and rehearsal

Hours to days, offline; the planner, commander, sensor manager, supervisor, and analyst act. A person sets the asset list and priorities, plans laydown over terrain, configures identification criteria, control status, authorities, and geofences; the system validates the baseline, rehearses by replaying a scenario under the plan, audits the apply, and folds the journal into measures after the shift. Exercises `AssetListAssessor` (DN-01), `laydown_rehearsal.rs` under `ReplayClockAuthority`, `gungnir-config` validation, `audit_trail.rs`, `review.rs`, `measures.rs`, and `battle_rhythm.rs`, built and gated; under DN-26 (signed 2026-09-06, confirmed 2026-09-07) the laydown options table and the viewport push are built (the push since 2026-09-08), while the rehearsal section, the gap-acceptance control and first-engagement range are not, and no adoption control exists by design (DN-26 section 6 rule 4); GAP-087 and GAP-045 are both in progress. TT-09.

MT-10 Disconnected operation and reconnection

A desktop loses its node for minutes to hours; the site operator continues, the supervisor at the node resolves conflicts, the commander's delegations govern. The system detects silence, falls back to embedded services with the state shown, queues detections, decisions, and audit entries, forwards and reconciles on reconnect, and leaves conflicts to a person. Exercises `failover.rs` and `failover_e2e.rs` (a real client against an in-process loopback node and the real `GET /v2/history`), the `gungnir-remote` outbox (`OUTBOX_CAPACITY = 100_000`, drop-oldest and counted; the bound is never filled by a test), `gungnir-resilience` reconcile, and `RoleRankArbiter`, built and gated; the D-15 rule that delegations expire while disconnected is not verified as a distinct object anywhere the survey reached (the queue's expiry semantics are gated; the delegation object is not), and the reconciliation row's criterion still names an arbitration rule pending GAP-067. Scenario 4, TT-10.

2.3 The ten vignettes

`docs/mission/vignettes.md` sets every vignette at the fictional Vell estuary (Kalsund port, Ostmark power station and sector command post, Halden airfield, the Hoge ridge radar); each has a committed sample set under `testdata/tracks/samples/`. Each bullet below gives the vignette's forces and the success criterion `vignettes.md` states for it, not a measured result -- no vignette has been scored against those criteria.

- VG-01: 45 one-way attack drones in three streams by night, six decoys; no drone reaches the power station (MT-01; TT-01).
- VG-02: the same raid plus four cruise missiles and two decoys, engaged under pre-delegated area authority in seconds (MT-02; TT-02).
- VG-03: a quadcopter over the airfield driven off by jamming kept outside the ADS-B and airfield navigation geofence, then a fiber-optic FPV warned acoustically (MT-03; TT-03).
- VG-04: four USVs at 38 knots at 03:00, one lost in clutter and reacquired by camera cue (MT-04; TT-04).
- VG-05: 140 vessels on a weekday afternoon -- `scenarios.yaml` sets TT-05's `expected.entities` to 139 and the vignette says 140, the repository's own inconsistency -- with one AIS-off loiterer and one spoofed coaster flagged within two minutes (MT-05; TT-05). The committed sample set is a 0.10-scaled 420 s slice carrying 17 entities.
- VG-06: a shoot-and-move battery cued to fires inside its window; a convoy keeps its identity through 20 minutes of tree cover (MT-06; TT-06).
- VG-07: the raid under GNSS jamming with the ridge radar lost; operators know within a minute what they cannot see (MT-07; TT-07).
- VG-08: civil and friendly aircraft on a busy afternoon; an adversary ISR UAS declared on evidence, an unknown never engaged (MT-08; TT-08).
- VG-09: Monday planning; the rehearsal shows the new laydown engages the sea stream earlier than the old, which TT-09 states as laydown B earlier than A (MT-09; TT-09).
- VG-10: the port cell loses the node for eleven minutes mid-raid; six local decisions, one conflict resolved by a person (MT-10; TT-10).

All ten sample sets replay through the gateway with zero quarantines (`gungnir-ingest/tests/test_track_samples.rs`) and through `LiveTrackingService`, the receipt-order path, within the asserted 1e-6, observed 0.0, against the same pipeline run offline in source order (`gungnir-tracking-service/tests/sample_set_replay.rs`) -- a self-consistency check of the ordering and reorder-buffer path, not agreement with an independent reference.

2.4 Measures of effectiveness with targets

From `docs/mission/measures.md` §1, with MOE-13 from `docs/mission/capabilities/measures-catalogue.md`. "Confirmed" means proposed by the drafting agent and confirmed or adjusted by the owner on 2026-09-04 (D-16 raised MOE-08 to 0.9 and tightened MOE-10 to 30 s); "doctrine" is a public absolute.

Id	Measure	Threads	Target	Source
MOE-01	Defended-asset protection	MT-01, 02, 04	0.95 for priority-1 and -2 assets	confirmed
MOE-02	No fratricide, no civil engagement	MT-01 to 04, 08	0	doctrine
MOE-03	Cost discipline (drones engaged by point layers)	MT-01, 02	at least 0.9	confirmed
MOE-04	Decision timeliness (fraction of time to impact)	MT-01 to 04	p95 under 0.3	confirmed
MOE-05	Decision completeness	all	1.0	doctrine
MOE-06	Picture honesty	MT-07, 10	0	doctrine; confirmed
MOE-07	Surface picture completeness	MT-05	0.98	confirmed
MOE-08	Fires timeliness	MT-06	0.9	confirmed
MOE-09	Identity continuity across a gap	MT-06, 08	0.9	confirmed
MOE-10	Degradation recovery	MT-07	30 s to show; battle rhythm to accept	confirmed
MOE-11	Continuity under disconnection	MT-10	1.0 reached; 1.0 resolved	doctrine; confirmed
MOE-12	Rehearsal effect	MT-09	rehearse every plan change; ratio observed, no target	confirmed
MOE-13	Intelligence product timeliness	none	0.95	confirmed

What exists against them is the journal fold in `gungnir-reporting`, gated by `gungnir-app/tests/measure.rs` (GAP-047; two folds of one journal agree exactly, and MOE-02 reads a later reclassification). No MOE has an operational value: no real sensor, deployment, or exercise has run, and every value in the repository comes from synthetic replay.

2.5 The roles and the authority matrix

D-05 (2026-09-04) adopted eight roles: the five original (operator, supervisor, analyst, sensor manager, administrator) plus commander, planner, and intelligence analyst, in code since 2026-09-05 (GAP-068, closed). `gungnir-security/src/lib.rs` carries a ninth variant, `SecurityOfficer` (D-30), which "operates nothing" and holds one action, `KEY_ESCROW_RECOVER`. The planner is view-only by the owner's decision; the administrator is "not a decision-maker in the engagement chain."

The authority matrix (`roles-and-stakeholders.md` §4, headed "initial, to be confirmed with policy") answers the first question a C2 reviewer asks: who may decide an engagement. Point-layer acceptance: the operator by delegation, the supervisor, the commander. Area-layer acceptance: the supervisor for pre-delegated cases only, the commander outright. Weapons control status, hold or cease, plan apply, and reconciliation conflicts: supervisor and commander. Sensor tasking: the sensor manager decides, the operator may cue a camera, the supervisor concurs, the intelligence analyst only requests. Model promotion: the analyst with supervisor concurrence. Accepting a coverage gap: the commander alone. Everyone else recommends or views. D-15 adds the delegation rule: the supervisor may pre-delegate point-layer engagements of confirmed-hostile small UAS classes, "area layer and missiles never."

In code, `gungnir-security/src/authz.rs::role_permits` is the coarse matrix and `gungnir-policy/src/authority.rs` refines it by role, action, layer, and class (GAP-058 closed; DN-09 signed 2026-09-05). Built and gated. The supervisor's missing `RELEASE_PRODUCT` was found 2026-09-08 as a discrepancy against §4 and closed to match the row, signed by the owner the same day (`ARCHITECTURE.md` §10).

Note: the honest answer to "who can fire" is that nothing in Gungnir can. The most any policy engine returns is `RequiresHumanApproval`; the engagement authority is a person whose decision is recorded, and a test fails the build if an execution path is constructed without one.

Chapter 3 -- The five engineering scenarios

`gungnir-scenario` is a deterministic ground-truth and sensor simulator: the same `Scenario` and seeded `Rng` produce an identical `GeneratedTimeline`, and observations are emitted in receipt order, so source time runs backwards wherever a

sensor is late, the condition `gungnir-fusion-async` exists to handle (`gungnir-scenario/src/lib.rs`). It is never a normal dependency of a shipping crate: `gungnir-app/tests/dependency_graph.rs` refuses any normal edge into it from outside the Verifier layer, and the one normal edge that exists is `gungnir-oracle`'s. Each scenario was chosen as the cheapest single case that forces several pipeline stages to run against each other rather than against an isolated stub (`docs/scenario-crate-narrative.md`). Everything is synthetic; no real sensor has fed this pipeline.

What each scenario is built to break

#	Variant (lib.rs)	Shape	Built to break	Rows the coverage map assigns it (<code>docs/scenario-crate-narrative.md</code>)
1	ManeuveringAircraft	one fast jet, CV to 100 s, coordinated turn at $\omega = 0.035$ rad/s to 200 s, constant acceleration to 300 s; one radar_medium	a six-state filter that cannot represent a turn; the CV-only baseline carries one track it never confirms	EKF, UKF, IMM; whole-pipeline replay; backend switching
2	MaritimeClutter { pd, clutter_rate }	six merchant vessels, staggered land-mask occlusions; one radar_coastal, Pd and false alarms per scan from the variant; 600 s	gating, assignment and lifecycle under clutter and occlusion	JPDA, MHT, lifecycle; the generator's own self-check
3	UrbanConvoy { injected_bias_m }	four trucks 120 m apart; radar_medium, isr_video carrying the bias (35 percent out of order, 2.5 s mean latency), acoustic; 480 s	out-of-sequence multi-rate ingest; a concurrency bug that only appears when the OOS buffer and the multi-rate schedule interact; sensor registration	fusion-async OOS, track fusion, registration
4	DenseSwarm { target_count }	interleaved columns on opposing headings at 180 m lane spacing, staggered births, every fifth entity dies at $0.6 \times$ duration; 240 s	association past its limit; cardinality that changes; frame budgets at load	PHD/CPHD, GLMB/LMB, metrics, egui and snapshot budgets
5	AdversarialGeometrySoak { cycles }	origin at 89.9 N on the antimeridian, a transport crossing the pole about 55 s in; three nearly collinear radars 400 m apart	pole and antimeridian singularities; ill-conditioned covariance; long soaks	coord, square-root soak, RTS, geofence containment

That last column is the coverage map's assignment, not the verification table's own *Data source* column, and the two differ. For four of the rows listed, the table's data-source cell names a synthetic fixture rather than the generator: coord coordinate frame transforms (line 91), track-fusion track-to-track fusion (line 89), sensor registration (line 90) and the fusion-async out-of-sequence row (line 87). The code agrees with the table there -- none of `gungnir-coord`, `gungnir-track-fusion` or `gungnir-fusion-async` has a dependency on `gungnir-scenario`, `dev` or `normal`. The section 2 rows in the column are a third case: whole-pipeline replay, backend switching, the egui and snapshot budgets and geofence containment are not in the coverage map at all, but their data-source cells do name the scenario crate or a scenario by number.

Those rows sit in two sections of the verification table. EKF, UKF, IMM, JPDA, MHT, lifecycle, the RFS filters, metrics, coord, the square-root soak, RTS, registration, track fusion, the fusion-async out-of-sequence row and the generator's own self-check are section 1 rows with a running gate; backend switching (line 130), geofence containment (line 150), the egui and snapshot budgets (lines 129 and 120) and the whole-pipeline replay (line 119) are section 2 rows, which that section's own preamble still says are not pass/fail gates yet (`docs/verification-capability-table.md` line 115).

Per `gungnir-scenario/src/sensor.rs`, radar_medium scans at 1 s with one-sigma noise of 25/60/150 m (range/cross/height), Pd 0.92; radar_coastal at 2.5 s, 15/60/0 m, with model defaults of Pd 0.8, 15 percent dropout and 2.0 false alarms per scan -- Scenario 2 overrides Pd and the false-alarm rate from its variant and sets the dropout to zero (`plan_maritime_clutter: radar.dropout = 0.0;`), so its missed detections come from Pd and the staggered land mask rather than the model's dropout term.

One distinction: a coverage-map entry names the scenario shape, but the oracle actually run is often a checked-in Python fixture of that shape. The EKF, UKF, IMM, RFS, square-root, JPDA, lifecycle, metrics, RTS, coord and track-fusion diff tests all read `testdata/oracles/` -- the track-fusion file carries the registration case too -- and the fusion-async out-of-sequence test builds its own timeline; the 100,500-cycle square-root soak runs on its own construction in `gungnir-filters/tests/sqrt_diff.rs`, not Scenario 5. Only seven test files consume the live generator: the three in `gungnir-tracking-service`, two in `gungnir-scenario` and two in `gungnir-ingest` described below. Two benches and `gungnir-app/tests/frame_budgets.rs` also drive it; the only mention of the crate anywhere in `gungnir-fusion-async` is a doc comment.

Truth-scored replay

`gungnir-tracking-service/tests/scenario_truth_replay.rs` is built and gated: it replays a scenario through `LiveTrackingService` on seed 7 and scores every target against the nearest track, using the noiseless truth the estimator never saw, at the last observation in the timeline (`docs/verification-capability-table.md` §1, line 39). The check runs

in that direction only: `fn` score folds each live target's error to the minimum over the track set, so a track matching no target is counted and printed rather than scored. Each bound is argued in the module documentation.

Scenario	Bound	Argument	Measured
1	500 m	above 3σ of the worst axis (450 m), below the 697 m the jet moves between scans in its acceleration phase	169 m (238 m before DN-30)
2	300 m	5σ of the worst axis; under a tenth of the 3.3 km between the nearest vessels	83 m (85 m before DN-30)
1 to 4	1,000 m	coverage only: no target alive at the comparison time is untracked	passes; Scenario 4 at 40 targets

DN-30 is the correction behind the parenthetical figures: `PipelineSettings::default()` assumed measurement noise of $[400, 400, 900] \text{ m}^2$ where Scenario 1's radar is $[625, 3600, 22500]$, twenty-five times the assumed height term. Reading the noise from the baseline is DN-30, and DN-30 is one increment with one status: built and gated on 2026-09-07 and not signed. It changes `PipelineSettings::from_baseline`'s signature in `gungnir-fusion-async`, which is human-owned, so it awaits the owner's review under that tier (`docs/design/DN-30-measurement-noise-from-the-baseline.md`). Scenarios 3 and 4 are deliberately not position-scored: their targets pass within 120 m and 233 m of each other, inside the noise, so a nearest-track match would not say which target a track is of.

The IMM-through-a-turn row (table line 55; DN-28, signed 2026-09-07) replays Scenario 1 truncated to source time under 199 s, the CV and CT phases only, because the terminal constant-acceleration phase is `MotionModel<9>` and no six-dimensional filter represents it. Under identical settings, `kf-cv` yields one track that never confirms; `imm-cv-ct` yields one confirmed track within the 500 m bound, measured at 138 m in the verification pass. One point the test itself records: the scenario radar's noise and the truth's own turn rate are supplied as per-test overrides, because `PipelineSettings::default()` still carries the mismatched noise. The module's own documentation records a second point, about the full scenarios rather than this truncated one: the count criterion holds only in its first half -- every target alive at the comparison time is accounted for, but the full Scenario 1 replay ends with two tracks for one aircraft and Scenario 2 with ten for six vessels, reported rather than asserted.

`whole_pipeline_replay.rs` runs in CI on every pull request and every push to main -- `.github/workflows/ci.yml` triggers on nothing else, so a push to a topic branch with no open pull request runs it nowhere -- and runs all five scenarios, Scenario 4 at 40 targets and Scenario 5 at 200 cycles, through the live service against the same pipeline offline within $1e-6$; Scenario 3's receipt order is genuinely out of sequence, so the comparison is not trivially true. It checks the ordering and reorder buffer, not the mathematics. The verification-table row it answers is the section 2 "Whole-pipeline scenario replay" row (line 119). The test is built and running, and the two documents disagree about what that makes the row: `docs/architecture.md` records it as **Specified 2026-09-07** after the GAP-067 walk, under a heading saying the walk was done and confirmed by the owner, while the verification table's section 2 preamble still says none of these rows is a pass/fail gate and its owner-confirmation cell is blank (line 172); the gap register keeps GAP-067 Open. That is a documentation defect in the repository, not a settled status, and this guide names it rather than picking a side.

The ten plan-07 sample sets in CI

The test-track suite (`docs/test-tracks/`) has 58 platforms in 30 kinematic classes and ten scenarios TT-01 to TT-10, one per mission vignette. Everything in it is synthetic too: the sets are generated on the fictional Vell estuary of `docs/mission/vignettes.md`, and no real site, route, unit or platform identity appears (`testdata/tracks/README.md`). Its ten committed sample sets under `testdata/tracks/samples/` total 103 entities and 11,564 detections (summed from each set's `metadata.json`; TT-09 and TT-10 are TT-01 under a second laydown and under a link loss, `docs/test-tracks/scenarios.yaml`). Three gates run on them: `gungnir-scenario/tests/reference_parity.rs` regenerates every set from the Rust port of `gen_tracks.py` and compares `truth.jsonl`, `detections.jsonl`, `detections-truth.jsonl` and `events.jsonl` byte for byte, with `sensors.json` and `metadata.json` compared as values because the reference writes them in Python dict order (GAP-016 closed; the `lib.rs` header still calling the composition engine "what remains" is a stale comment); `gungnir-ingest/tests/test_track_samples.rs` replays every set through the gateway with zero quarantines; and `gungnir-tracking-service/tests/sample_set_replay.rs` submits each set in receipt order with the latency its sensor model produced, up to the generator's 4.9 s cap, and checks the live service against the same `run_batch` pipeline run offline in source order within $1e-6$, observed as 0.0 on all 51 tracks -- a self-consistency check of the ordering, channels and reorder buffer, not a second opinion about the mathematics. The source-time back-jump is asserted greater than zero; the observed range, 1.50 s (TT-08) to 4.67 s (TT-07), is printed, not asserted.

The statistical self-check (built and gated)

`gungnir-scenario/tests/statistical_self_check.rs` backs the table's "Ground-truth and sensor simulation" row: empirical detection and clutter rates must fall within 2σ of the configured Pd (0.6, 0.75, 0.9) and clutter rate (0.5, 1.0, 2.0 per scan). Twelve fixed seeds are pooled first, because a single 2σ trial fails one run in twenty by construction. The denominator, `detection_opportunities()`, is recomputed from the plan rather than counted during generation, so a generator that skipped opportunities could not hide by shrinking both sides. Worst deviation recorded: 0.90σ (table line 20).

What the swarm shows about limits

Scenario 4 is the scene the built, unsigned dense-group wiring is sized against -- docs/performance-budgets.md puts the dense-swarm scenario at 200 tracks, and the component cap is derived from that rather than from the literature default -- though it does not feed the wiring's gate. gungnir-fusion-async has no dependency on gungnir-scenario, dev or normal, so it could not consume the generator: the gate in gungnir-fusion-async/tests/dense_group.rs runs on a hand-built line-abreast raid, raid(count, spacing, scans), and the measured figures quoted here come from the doc-comment measurement table in gungnir-fusion-async/src/dense_group.rs on the same raid. The whole-pipeline replay does run Scenario 4 at 40 targets, but leaves PipelineSettings::dense_group at its None default, so the mode is not engaged there either. What the wiring does: past MAX_DETECTIONS detections -- that is, at MAX_DETECTIONS + 1 and not at MAX_DETECTIONS -- a PHD or CPHD runs beside the per-track filters and reports a count carrying no TrackId; a 20-target raid is counted as 21.1 expected targets at 6.3 ms per epoch, and 200 targets cost 196.3 ms -- release profile on one development machine, recorded as an order of magnitude rather than a regression baseline -- so the mode is off by default against the 4 ms frame budget of docs/performance-budgets.md. The MOT16/17/20 and KITTI benchmark rows are not built.

Part II, Chapter 4 -- The layer model and the 51 crates

4.1 The layers

The workspace is 50 member crates plus gungnir-fuzz, which the root Cargo.toml excludes because the fuzz crate has its own toolchain quirks: 51 crate directories, 399 first-party .rs files, 156,094 lines including tests (census at ee12f7e). The authoritative placement of every crate is not a document but a table in a test: the LAYERS constant in gungnir-app/tests/dependency_graph.rs, "ARCHITECTURE.md §7.1, transcribed. Every workspace crate must appear here." Its Layer enum has eight values in dependency order.

Layer	Crates	Role
Core	11	The tracking core: motion models, frames, estimators, association, lifecycle, RFS filters, fusion, metrics, the async pipeline, the allocator, the scenario generator
Verifier	3	Depend on what they verify; nothing depends on them
Model	1	gungnir-model, the canonical data model; may depend on Core only
Facade	2	The two service traits the binaries hold; may not depend on Productization (AP-10)
Data	2	3D file I/O and point-cloud compute; nothing above the model
Productization	26	Eventing, storage, configuration, ingest, security, policy, command, API and the rest
Deployment	4	Remote backends, the wgpu device, the 3D viewport, the egui panels
Binary	2	gungnir-node and gungnir-app, wiring only (C-13)

CLAUDE.md compresses this to the one-line rule an agent has to carry: "core, then service facades, then productization, then UI." ARCHITECTURE.md §1 says the same in prose: fourteen core crates (eleven capability, three verification), and every other layer reaches them "only through the two service facades in §2, or through the primitives gungnir-core and gungnir-coord own and re-export upward." One documentation defect to know about: the TOGAF phase C application document counts "Fifty crates, of which two are binaries, in seven layers" with productization at 25; it is dated 2026-09-04, predates gungnir-ml (2026-09-06) and folds Verifier into the core count. The code's table is the truth, and the root README.md is also stale by one ("49 members plus gungnir-fuzz").

4.2 The one-way rule, as written

The rule lives in three places that say the same thing at three levels of detail.

docs/agent-coding-standards.md §1.1 fixes the chain inside the core: gungnir-core / gungnir-coord -> gungnir-filters -> gungnir-association -> gungnir-track -> gungnir-rfs / gungnir-track-fusion / gungnir-metrics -> gungnir-fusion-async, with gungnir-allocation depending on gungnir-core only and gungnir-scenario feeding test and bench code only. Above the core: the model depends on core and coord; the facades on the core and the model; productization on the model, the facades, or each other as drawn in §7.1; deployment on the facades and productization; "and nothing in the core, the model, or the facades depends on a productization, deployment, or UI crate." The closing instruction is the one that matters for an agent-driven codebase: if a task seems to need a new edge, "that's a signal the abstraction is misplaced -- stop and flag it rather than adding the edge."

docs/rust-ui-architecture-coding-standards.md §1 states the UI-side version: data -> app::state -> ui/viewport3d -> render, with "invert the dependency with a trait" as the only permitted way for a lower layer to learn about an upper one.

ARCHITECTURE.md §7 carries the per-crate dependency table and §7.1 the box-drawn graph, and every edge added after the scaffold carries a letter and a justification, (a) through (u). The governing rule, decided by the owner on 2026-09-05 and

recorded in docs/design/dependency-edges.md: new edges are allowed where the coupling is natural, "each drawn in ARCHITECTURE.md §7.1 in the same change that introduces it," argued as "What it buys" against "What it would otherwise duplicate," checked for cycles, and open to rejection by the engineering reviewer "in favour of passing the data in." Acceptance and existence are kept distinct: the gungnir-remote -> gungnir-interop edge (DN-25) is accepted and in no manifest, because "an edge drawn there that no manifest carries would be the graph claiming something untrue." Refused edges are recorded with the same care: assessment -> config (DN-01), intercept-service -> command (DN-06, "Impossible under AP-10"), policy -> security (DN-09), and an edge for the anomaly detectors (DN-15).

4.3 How the rule is enforced

The contract is C-11, "No dependency edge exists that ARCHITECTURE.md does not draw," under principle AP-10; its check is gungnir-app/tests/dependency_graph.rs, automatable "Yes, on every cargo test," violation "Reject and stop for a human" (docs/architecture/togaf/phase-g-implementation-governance/architecture-contracts.md). The file is 597 lines and five tests, and it runs inside cargo nextest run --workspace in ci.yml on every pull request and push to main. Status: built and gated.

1. every_crate_is_placed_in_a_layer: every gungnir-*/Cargo.toml is in LAYERS, every placed crate has a manifest, every named dependency is a workspace crate. "A new crate fails this test until somebody says which layer it is in."
2. the_dependency_graph_is_acyclic: a three-color depth-first search over normal edges; a gray hit panics with the cycle path.
3. every_edge_points_downward: a Core crate may depend only on a Core crate earlier in CORE_ORDER; Model on Core only; Facade on Core or Model; Data on Data, Core or Model; Productization on Core, Model, Facade, Productization or Data; Deployment and Binary on anything except Binary or Verifier; and only a Verifier may depend on gungnir-scenario.
4. the_recorded_edges_are_in_the_manifests: fifteen lettered edges asserted present by name.
5. the_manifests_and_architecture_md_agree_on_every_edge: parses §7's table and §7.1's graph back out of the Markdown, asserts the two agree, asserts more than 40 crates parsed, then compares against the manifests in both directions, naming "an undeclared dependency" or "a stale document."

Only production [dependencies] and [target.<cfg>.dependencies] count; dev-dependencies "are the verifiers' business and may point anywhere." The test's own header admits its origin: the design document "said the acyclicity check ran in a continuous-integration job. It did not: the checks were written for one assessment on 2026-09-04 and discarded, and every edge review since rested on prose. This test is the check."

The fifth test exists because of a miss. Edge (t), gungnir-remote -> gungnir-security, entered a manifest on 2026-09-06 with the move of identity.rs out of gungnir-node; the direction check passed it because a security edge from a deployment crate is legal, and §7's table did not list it. It surfaced on 2026-09-07 when the UAF generator ran in CI for the first time and the view it regenerated from the manifests showed an edge the table did not. The bidirectional test was added the same day and, on its first run, found a second disagreement: §7 listed testkit as a production dependency of gungnir-oracle when it is a dev-dependency.

The numbers, computed from the manifests at ee12f7e: 169 normal crate-to-crate edges among the 50 members, 172 with gungnir-fuzz; acyclic; longest path 9 (gungnir-app, gungnir-node); most depended-on gungnir-model with 31 dependents among the members and 32 counting gungnir-fuzz, which takes it too, then gungnir-eventing and gungnir-coord with 10 each and gungnir-core with 8. docs/design/dependency-edges.md §4a still says "154 crate-to-crate edges, no cycle" as of 2026-09-05 and does not restate the count after edges (g) to (u); the same file uses the letter (s) for two different edges and has two sections numbered 13. These are documentation defects, not graph defects; the test is what keeps the table exact.

4.4 The 51 crates by layer

The human-owned column follows the low-trust tier in docs/agent-ic-workflow.md: **Y** means the crate is on that list; **clause** means the numerical-stability clause (covariance stays PSD, no silent NaN) reaches the crate while the crate as a whole is not on the human-owned list; that document places gungnir-filters' and gungnir-association's math in the medium-risk tier and does not name gungnir-core, gungnir-rfs or gungnir-track-fusion in any tier; **path** means a named path or rule inside the crate, not the crate; **N** means neither. Purpose lines are condensed from each Cargo.toml description and lib.rs doc.

Crate	Purpose	Key traits and types	Human-owned
gungnir-core	CV/CA/CT motion models, shared identifiers, debug-only PSD assertion	MotionModel<const N>, TrackId, TrackStatus, ResourceId, assert_psd	clause
gungnir-coord	ECEF/ENU/NED/geodetic transforms, bearing crossing	CoordTransform, Geodetic, Ecef, Enu, Ned, Wgs84	N

Crate	Purpose	Key traits and types	Human-owned
gungnir-filters	KF, EKF, UKF, particle, IMM, square-root, RTS	Filter, KalmanFilter, Imm, SqrtKalmanFilter, rts_smooth	clause
gungnir-association	gating, GNN, Jonker-Volgenant, JPDA, MHT	Associator, solve_assignment, ChiSquareGate, Jpda, Mht	clause (D-43 signed under it)
gungnir-track	lifecycle: init, confirm, coast, delete	Track, lifecycle	N
gungnir-rfs	GM-PHD, GM-CPHD, LMB; delta-GLMB refuses by name	PhdFilter, CphdFilter, LmbFilter, GlmbFilter, RfsError	clause
gungnir-track-fusion	track-to-track fusion, sensor registration	TrackFuser, CovarianceIntersectionFuser, InformationMatrixFuser, SensorRegistration	clause
gungnir-metrics	CLEAR MOT, purity, fragmentation	compute_metrics, MetricsConfig, TrackingMetrics	N
gungnir-fusion-async	out-of-sequence multi-rate pipeline; the only core crate that uses the tokio runtime, which the host binary owns	ingest, FusionPipeline, PipelineSnapshot, PIPELINE_IMPLEMENTED	Y (concurrency)
gungnir-allocation	Bellman/DP resource-to-track allocator	ResourceAllocator, BellmanDpAllocator	clause (allocator fixes signed 2026-09-06)
gungnir-scenario	five-scenario truth and sensor simulator; test and bench only	Scenario, ScenarioGenerator, TrackLibrary	N
gungnir-testkit	shared proptest strategies; no workspace dependency	psd_covariance, cost_matrix	N
gungnir-oracle	central differential harness; not built (GAP-082), suite is distributed	diff_test_filter (refuses)	N
gungnir-fuzz	three cargo-fuzz targets; workspace-excluded	sensor_ingestion_parser, asterix_feed, cost_matrix_construction	N
gungnir-model	canonical versioned data model; 31 dependents among the members, 32 with gungnir-fuzz	TrackView, DetectionView, PlanView, Measurement, SessionId, SCHEMA_VERSION	N
gungnir-tracking-service	app-facing facade over the tracking core	TrackingService, LiveTrackingService, project_track	path (changes trigger loom.yml and sign-off, coding standards §5)
gungnir-intercept-service	app-facing facade over the allocator	InterceptService, PlanOutcome, DpInterceptService	N
gungnir-data	file I/O for point clouds, DEM, VTK, glTF; no GPU dependency	DataStore, LoadRequest, PointBuffer	N
gungnir-data-fusion	CPU and wgpu point-cloud registration	PointCloudFusion, GpuFusionEngine, CpuIcp	N (medium-risk)
gungnir-venting	broadcast event bus with monotonic sequence	EventBus, InProcessBus, Envelope	N
gungnir-store	append-only JSON-lines journal, retention, sealing	EventJournal, FileEventJournal, DurabilityPolicy	N (medium-risk)
gungnir-config	validated versioned baselines; embedded/remote switch	ConfigBaseline, BackendConfig, validate	N
gungnir-mission	mission and session lifecycle	MissionState, MissionManager	N
gungnir-time	source versus receipt time, replay clock	TimeAuthority, SourceTime, LateDataPolicy	N
gungnir-ingest	protocol adapters and the validation/quarantine gateway	IngestGateway, ProtocolAdapter, SourceAuthenticator	Y (the gateway)
gungnir-sensor-management	sensor registry, modes, tasking, coverage	SensorRegistry, SensorControl, CoverageRegion	N

Crate	Purpose	Key traits and types	Human-owned
gungnir-interop	schema catalog, Arrow, ASTERIX/ADS-B/AIS/MISB codecs	SchemaCatalog, AsterixCat048Codec, AdsbCodec, Stanag4676Codec (refuses)	N (medium-risk)
gungnir-identity	cross-session entity lineage	IdentityResolver, EntityLineage	N
gungnir-identification	friend/foe/unknown evidence fusion	IdentificationEngine, EvidenceFusionEngine	N
gungnir-model	model and feature traits; ONNX behind a default-off feature; no dependents	Model, FeatureExtractor, OnnxModel	N
gungnir-geo	geofences, map layers, hazards	GeoService, Geofence	N
gungnir-analytics	line of sight, coverage volumes, anomaly detectors	LineOfSight, CoverageVolume, anomaly	N
gungnir-policy	verdicts, geofence and authority enforcement	PolicyEngine, PolicyChain, PolicyVerdict	Y
gungnir-command	human-approval queue over verdicts	ApprovalWorkflow, DecisionRecord	Y
gungnir-assessment	threat scoring, time to impact, reward matrix	ThreatAssessor, reward_matrix, prediction	N
gungnir-decision	courses of action, what-if against the current pipeline snapshot	DecisionSupport, CourseOfAction	N
gungnir-modelops	baseline registry: validate, promote, roll back	ModelRegistry, PromotionState	N
gungnir-security	authentication, authorization, keys, sealing, audit	Authenticator, Role, KeyProvider, AuditLog	Y
gungnir-api	v2 contract and axum transport	ApiHandler, transport::serve, v2	path (write paths)
gungnir-observability	health monitor, watchdog, alerts	HealthMonitor, Alert	N
gungnir-resilience	store-and-forward, checkpoints, reconcile	StoreAndForwardQueue, reconcile	N (medium-risk)
gungnir-collab	shared picture, authority arbitration; reached by no binary	SharedPictureSync, RoleRankArbiter	N (medium-risk)
gungnir-workflow	layouts, alert lifecycle, cases, reviews	PanelId, WorkspaceLayout, AlertLifecycle	N
gungnir-replay	deterministic journal playback	ReplaySession	N
gungnir-reporting	reports recomputed from the journal	ReportGenerator, MissionReport	N
gungnir-remote	remote facade implementations, node link, TLS identity	connect, RemoteTrackingService, LinkTls, identity::issue	path(identity.rs, LinkTls::client_config)
gungnir-renderer	the one wgpu device, lent for compute	GpuContext	N
gungnir-viewport3d	three-d scene, 2D projection, layers	ViewportState, SceneRenderer, TrackGlyph	N (medium-risk)
gungnir-ui	egui panels and theme; reads model views only	panels, theme, RenderProbe	N
gungnir-node	headless service node for the connected profiles	main, TICK, account	N (wiring only)
gungnir-app	eframe desktop; "wiring, not logic"	AppState, update::tick	N (wiring only)

Two rows deserve a footnote in conversation. `gungnir-allocation` is named in the medium-risk tier of `docs/agent-workflow.md` ("the Bellman/DP resource-allocation solver"), yet `ARCHITECTURE.md` §10 item 92 records that the numerical-stability clause reaches it and that both allocator fixes were signed by the owner as human-owned tracking work on 2026-09-06; the tier document was not amended, so the two documents state the crate's tier differently. And the `gungnir-remote` path is the precedent for how the list grows: the owner added `identity.rs` on 2026-09-06 "Scoped to the

path, not the crate," because "a list that swallowed the whole crate would make routine work need a signature and the signatures would stop meaning anything."

4.5 Three placement corrections

ARCHITECTURE.md §10 records, in the 2026-09-05 implementation entry, that "Three type placements moved during implementation, each recorded in the note that assumed otherwise... None of the three added an unapproved edge, which is what they were avoiding." Each is a case where the design pass assumed a type could live somewhere the graph forbids.

Frame anchoring (DN-01 §3a). The defended-assets design had gungnir-assessment hold an AssetListView. It cannot: "assets are geodetic, tracks are local ENU metres, and the conversion lives in gungnir-coord, which gungnir-assessment does not depend on." The caller anchors each asset to the local frame (AssetAnchor, anchor_list), and the assessor scores against anchored positions. The note keeps the correction on the page "because it is the kind of frame mismatch a design pass is prone to missing and an implementation always finds."

Primitive snapshots (DN-15 §3a). The anomaly-detector design typed the detectors as `fn(&[TrackView], ...)`. "That cannot be built: gungnir-analytics does not depend on gungnir-model, and adding that edge is exactly what D-13 forbids... The note contradicted its own section 4." The detectors take TrackSnapshot and SensorHealthSnapshot, which carry primitives, and "the cooperative-mismatch comparison moves to the caller, because the classification type belongs to the model." Edge (c), the analytics-to-sensor-management edge accepted the same day (DN-12, coverage_from_registry), is also what gave gungnir-analytics its dependency on the model, because the coverage types name sensors by SensorId; the two coverage-report edges are (f) gungnir-api to gungnir-analytics and (g) gungnir-node to gungnir-analytics. The detectors kept their primitive inputs, and DN-15 §4 records the edges as "None, per D-13."

SessionId. DN-20 assumed SessionId was a model type; it was in gungnir-store, and the review case in gungnir-workflow would have needed a sixth edge to reach it. Six crates share the type, so on 2026-09-05 it moved down to gungnir-model with the store re-exporting it: "That is agentic-coding-standards.md §1.2 working as intended: the type moved down rather than the dependency reaching across" (ARCHITECTURE.md §7.1). It is now SessionId(pub u64) at gungnir-model/src/lib.rs:162 and one of the 18 names every_shared_type_has_exactly_one_definition scans.

4.6 What a technical lead will ask

Why does gungnir-model exist as its own layer? So that "what a track is" is defined exactly once (gungnir-model/src/lib.rs; ARCHITECTURE.md §7.2). TrackView, DetectionView, ResourceView, PlanView, SystemHealth and the event schema are what let ingest, identity, store and the API agree on shapes without each inventing one. The layer may depend on Core only, so the identifier primitives the core and the model share (TrackId, TrackStatus, ResourceId, Geodetic) moved down into gungnir-core and gungnir-coord and are re-exported upward; the facades re-export the views as their public contract; gungnir-ui, gungnir-viewport3d, gungnir-policy, gungnir-command, gungnir-decision, gungnir-assessment and gungnir-ingest all compile against the model's types rather than the core's (ARCHITECTURE.md §7.2) -- they reach the tracking core's data only through the model, never through a core crate directly, whatever else they also depend on in the productization layer, or in the facade layer above the core (gungnir-policy also takes gungnir-geo, gungnir-decision takes assessment, policy and analytics, gungnir-ingest takes time, interop and the gungnir-tracking-service facade). The ownership rule is a gate, not a convention: every_shared_type_has_exactly_one_definition in gungnir-app/tests/architecture_compliance.rs scans 18 shared names (C-07). The same reasoning explains smaller placements, such as SensorMode living in the model rather than in sensor management: "An event has to carry it and events live in the model, which cannot depend on a crate that depends on it" (ARCHITECTURE.md §10 item 56). Built and gated; SCHEMA_VERSION is 3.

Why is gungnir-data-fusion separate from gungnir-data? Because one of them must never need a GPU. gungnir-data converts every third-party type (las, vtkio, tiff, gltf) into an internal, dependency-free one and carries "No egui/three-d/wgpu dependency -- unit-testable with plain cargo test." gungnir-data-fusion is "the only data-layer crate that needs a wgpu: :Device (borrowed from gungnir-render)," and upper layers depend on its PointCloudFusion trait, on "the interface, not the wgpu internals directly" (docs/rust-3d-data-ecosystem-build-vs-adopt.md §3.5, the dependency-inversion rule). Splitting them keeps the loaders and the CPU reference (CpuIcp) in the ordinary test run, and lets the GPU path be checked against the CPU one on the same fixtures, within a stated tolerance -- transform within 1e-3 (m, rad) of CpuIcp and inlier ratio within 0.01 -- over the same unmodified solve_rigid_transform both paths call. Status by half: the CPU ICP is built and its unit tests run on every CI cargo nextest run (gungnir-data-fusion/src/{cpu_reference,transform_solve}.rs), with a criterion of translation within 0.05 m and rotation within 0.5 mrad agreed 2026-09-04; docs/architecture.md calls the row Specified as of the 2026-09-07 GAP-067 walk, while the verification table's section-2 preamble still says none of its rows is a gate yet and the register keeps GAP-067 Open -- a documentation contradiction, not a second measurement. The GPU path is built and its four kernels are naga-validated in CI on every cargo nextest run, but the GPU-versus-CPU tests are #[ignore]d behind the gpu-tests feature, the one hardware pass on an RTX 5060 Ti is the implementing agent's self-report, and the registered runner has not completed a dispatch, so the agreement between the two paths is built, ungated. The point-to-plane CPU ICP is built and

unit-tested but has no verification-table row.

Why does `gungnir-metrics` depend on `gungnir-association`? Because CLEAR MOT's second stage is a minimum-cost assignment, and the workspace already has a gated solver. `gungnir-metrics/src/clear.rs` reproduces the three ordered stages of `py-motmetrics`' accumulator: carry forward matches still inside the gate, assign the rest, then count unmatched truth as misses and unmatched hypotheses as false positives; a stage-2 pair for a truth previously matched elsewhere is an identity switch. Stage 2 calls `gungnir_association::solve_assignment`, "the same Jonker-Volgenant optimum `scipy.optimize.linear_sum_assignment` computes and is itself gated against `scipy`," with beyond-gate pairs made unassignable by `motmetrics`' own large-constant substitution, reproduced with its derivation in `add_expensive_edges`. A second solver would have been a duplicate beside the gated one, which is the duplication the edge rule exists to refuse; and `CORE_ORDER` places association before metrics, so the edge points downward. Built and gated: `tests/motmetrics_diff.rs` measures 2.2e-16 against `py-motmetrics` 1.4.0 with a 1e-3 criterion and every underlying count exact. The recorded finding is that purity "is not a `motmetrics` metric," so that row checks the aggregation given an agreed matching rather than an independent definition. The crate's own header records why its signature changed: the scaffold's two flat slices "could not have been implemented as written," since an identity switch is undefinable on a single snapshot.

Note: if asked to name a weakness of the graph, name the two dependency-edges.md defects and the stale 154 count before a reviewer does, then point at the fifth test as the reason they are documentation defects rather than graph defects.

Chapter 5 -- Deployment profiles, the embedded/remote switch, and resilience

5.1 One crate set, three profiles

`ARCHITECTURE.md` §8 states the decision in five words: "embedded or remote, same crates." The services layer is compiled into the desktop for standalone use and into the headless `gungnir-node` binary for connected use. No domain crate holds profile logic (principle AP-05, "One product, three profiles").

The three profiles as `ARCHITECTURE.md` §8 designs them -- none of the connected two has been deployed:

Profile	Services run in	System of record	Users	Network
Disconnected desktop	<code>gungnir-app</code> (embedded <code>LiveTrackingService</code> , <code>DpInterceptService</code> , <code>FileEventJournal</code>)	The desktop's own <code>gungnir-store</code> journal	One operator	None required; <code>gungnir-api</code> may listen on loopback
On-prem connected	One or more <code>gungnir-node</code> instances	The node	Operators, supervisors, analysts sharing one mission	LAN
Cloud connected	The same <code>gungnir-node</code> binary, hosted	The node	As on-prem, plus remote clients and peer C2	WAN; higher latency budget, stricter security posture

The desktop targets Windows 11; the node targets Linux x86_64 containers, and `ci.yml` builds the image on every pull request. `release.yml` has never run (facts R9 §5.1). Status: the disconnected desktop is **built and gated**; the connected profiles are **built and gated** only in-process over loopback -- the transport and the desktop's failover path have no gate off that machine. The node half is thinner than that: `gungnir-node`'s own headless-loop criterion -- start, run ticks, interrupt, exit cleanly -- has no automated test, confirmed in writing by the GAP-067 walk on 2026-09-07 ("none automated," `docs/verification-capability-table.md`; `docs/architecture.md` line 79). The smoke run on every batch is a person running the binary. No node has ever served a desktop across a network; no cloud deployment exists.

5.2 The switch: two traits and one configuration tag

`gungnir-app::AppState` holds `Box<dyn TrackingService>` and `Box<dyn InterceptService>` (`gungnir-app/src/state.rs`:61-62); the traits are the deployment seam. `TrackingService` (`gungnir-tracking-service/src/lib.rs`:80-134) has `submit_detection`, `poll`, `tracks`, `is_healthy`, and two defaulted methods, `bearing_rays` and `pipeline_stats`, which return empty so a backend with no pipeline behind it can say so without a health flag lying. `InterceptService` (`gungnir-intercept-service/src/lib.rs`:105-131) has `plan`, `is_healthy`, and a defaulted `withheld`.

The choice is a `serde`-tagged enum, `gungnir_config::BackendConfig` (`gungnir-config/src/lib.rs`:915-923): Embedded by default, or Remote { endpoint } from a baseline such as { "backend": { "kind": "remote", "endpoint": "http://node.local:7410" } } named by `GUNGNIR_CONFIG` (`deploy/README.md`). Health is reported, never inferred: `LiveTrackingService::is_healthy` returns `pipeline_alive` && `PIPELINE_IMPLEMENTED` (`lib.rs`:745-747), true since 2026-09-06 (GAP-011). `ARCHITECTURE.md` §2 still prints the pre-GAP-066 trait shapes; the code is the truth.

5.3 The remote side: `gungnir-remote`

`connect(endpoint, credential, runtime)` returns as soon as the link task is spawned, and its doc says plainly that ok does not mean a node answered: both remote services report `is_healthy()` false until a snapshot has arrived (`gungnir-remote/src/lib.rs:220-241`). One task owns the link and writes into a `Projection` both services read; the frame loop never blocks on the network (`link.rs`). `request` carries snapshot and health, `tokio-tungstenite` the event stream; the latter takes no TLS feature, since the client wraps its own `tokio-rustls` stream from pinned roots, and an https endpoint with no pinned roots is refused (facts R1 §3.3). `RECONNECT_DELAY` is 2 s (`link.rs:59`).

The heartbeat constants have one owner, `gungnir-api/src/transport.rs`: `HEARTBEAT_INTERVAL` 2 s (line 123), `HEARTBEAT_MISSES_TOLERATED` 3 (line 130), `HEARTBEAT_MARGIN` 1 s (line 134), and `HEARTBEAT_TIMEOUT` derived as `interval × misses + margin = 7 s` (lines 142-143; asserted at line 1886). The derivation is the point of D-23 (2026-09-06). The connectivity budget had said "fallback under 2 s from the last successful heartbeat" beside a 10 s beat and an independent 35 s timeout, so a node that went silent with its socket open was unnoticed for 35 s, the case the budget existed for. Raising the budget to 35 s was rejected as the widen-the-criterion move the workspace forbids. The budget was split into visibility -- the operator sees the picture going stale within 2 s of the last beat (`docs/performance-budgets.md` line 45), with PN-01's strip turning overdue past two beats rather than one so a beat arriving fractionally late does not flicker it red (`gungnir-app/src/status.rs:117-133`) -- and declaration, the link called gone within four beats, about 7 s: three tolerated misses plus the 1 s margin (line 46). The timeout is derived in code "because two independently edited constants is precisely how the conflict was created" (`docs/mission/gap-analysis/decisions-needed.md`). Status: **built and gated**.

5.4 The node side: `gungnir-node` and the v2 routes

The node builds a multi-thread Tokio runtime, binds with `bind` (plaintext refused off loopback, `ApiError::UnprotectedBind`), and serves the axum v2 router from `gungnir-api/src/transport.rs:832-868`: 14 route paths, 18 method endpoints (claim C33). Every route except `POST /v2/session` resolves a caller: a session token makes an operator, and a mutual-TLS client certificate that maps to a machine identity or exchange party is admitted on the certificate alone (`transport.rs:872-927`, claim C30). "A token on every route" overstates it.

Two behaviors carry the design. `POST /v2/detections` queues the detection for the ingest gateway, which authenticates the sensor and validates the detection on its next tick as it would a sensor feed; it answers 202 rather than 204 because the detection "has been taken, not that it has been believed" (`transport.rs:1187-1192`). It is the only route with an inbound schema-version check (409 on ahead, behind, or absent). `POST /v2/plans/{id}/decision` answers 501 after authentication (`transport.rs:1613-1614`: "an unauthenticated caller learns nothing about what this node does or does not run"; DN-23 §6 is what requires the token on every route but `POST /v2/session`, before GAP-062 admitted machines on their certificates) because a node runs no approval queue: the desktop routes plans through the policy chain and the queue (GAP-038), and a node publishes `PlanProposed` and stops (`transport.rs:26-32`, :1603-1624). `GET /v2/history` serves an in-memory window of `BACKLOG_CAPACITY = 8,192` envelopes (`transport.rs:104`). Status: **built and gated**, in-process over loopback with `rcgen` certificates generated per test.

5.5 Failover, store-and-forward, reconciliation

The sequence in `gungnir-app/src/failover.rs` (GAP-050; D-03, D-23, D-15):

1. The desktop judges the link's silence against `HEARTBEAT_TIMEOUT`, imported through `gungnir_remote::link` rather than a new manifest edge to `gungnir-api` (`failover.rs:25`).
2. Past the timeout it falls back to embedded services, records `Fallback { endpoint, since, silent_s, last_seq, ... }`, and says so twice, on the strip and on the record, as the module doc puts it (`failover.rs:8-9`): it publishes `Event::Link(LinkEvent::Fallback { endpoint, silent_s, at })` to the record (`failover.rs:166-174`) and pushes "running embedded; decisions taken now are this desktop's and will need reconciling" onto the operator alerts PN-01 shows (`failover.rs:175-179`). The link keeps retrying.
3. Detections submitted while detached queue in the `gungnir-remote` outbox, bounded by `OUTBOX_CAPACITY = 100,000` (`gungnir-remote/src/lib.rs:218`), oldest dropped and counted, flushed to `POST /v2/detections` on reconnection.
4. When the node answers, the desktop fetches the node's journal from `last_seq + 1` over `GET /v2/history` (`failover.rs:119-133`) and runs `gungnir_resilience::reconcile`: concatenate, sort by mission time, drop an envelope whose mission time and event both match one already merged, and report every `DecisionConflict` (one `PlanId` decided differently) rather than resolve it (`gungnir-resilience/src/lib.rs:97-133`).

5. PN-18 is where the merge and its conflicts are put in front of a person (`failover.rs:12-13`), and it is **built, ungated**: `gungnir-app/src/workspace.rs:357` draws it through `render_reconciliation` (`workspace.rs:80-138`), which shows the outage window, the local decisions taken meanwhile, the outbox, each reported conflict and the switch-back control, and the only automated check on it is a headless egui probe asserting it painted non-empty text. PN-19 (assistant, GAP-044) is the one panel that still renders a placeholder naming its gap; `docs/architecture.md` line 87's "PN-16, PN-18 and PN-19 remain" is a stale cell, and the dead `Reconciliation => "GAP-050"` arm at `workspace.rs:250` is unreachable because the match at `:357` returns first. Switch-back is a person's act: `failover.rs:300-330` refuses while no outage exists, the node has not answered, the reconciliation is still being computed, a conflict is still unresolved (the check at `:311-321`), or the link has gone silent again (D-15). Nothing switches back on its own.

D-03 (2026-09-04) locked the defaults: mission-time order, duplicates dropped, conflicts reported; higher role wins a conflict, earlier decision wins a tie. The ranking rule is `gungnir-collab::RoleRankArbiter`, expiry-first since 2026-09-05 -- **built and gated** but wired to nothing: `gungnir-collab` has no dependents and neither binary reaches it, which is consistent with reconciliation reporting conflicts rather than resolving them (facts R1). Documentation defect: `gungnir-resilience/src/lib.rs:10-12` still says the rule "is not yet locked."

Two limits to state first. The 100,000 bound is a constant plus code review: no test fills the outbox; the only bounded-queue test drives `StoreAndForwardQueue` at capacity 2 (claim C34). D-15's second half, delegations kept at disconnection and expiring after a configured interval, was not found in code (facts R2 D-15): pre-delegation is **built and gated**, disconnected expiry **not built**.

5.6 Durability per profile (D-04) and the fsync defect

D-04 (2026-09-04) settles fsync per profile. `DurabilityPolicy` (`gungnir-store/src/durability.rs:37-49`) has two variants with the trade written on them: `SyncEveryEnvelope` for the node, whose budget is "an accepted envelope is on disk within 100 ms" and which is the system of record for every connected desktop; `Buffered { fsync_interval }` for the desktop, fsync on session save and every 5 s (`DESKTOP_FSYNC_INTERVAL`, line 52), because a hard kill "costs a local session, not the mission picture." The fsync is `sync_data`, not `sync_all`; the module doc records that the previous code called `Write::flush` on a `File`, which "is not a fsync and never was," so the node had not been meeting D-04 either. Node ingest measured about 0.26 ms per fsynced envelope against the 100 ms budget on 2026-09-05, with the tracking stage still a stub at the time (facts R6 §10); a measurement, not a gate, and not an end-to-end figure for the built pipeline. Status: **built and gated** (`gungnir-store/tests/durability_policy.rs`; a desktop fsync failure raises an alert, `gungnir-app/src/update.rs:218-226`).

The defect: `AppState::save_session` called `journal.sync()` without draining the event bus, so anything published after the last frame reached the bus, never reached the journal, and was gone. `ARCHITECTURE.md` §10 item 66's words (GAP-005): "an fsync of a file the envelope was not written to is a durable record of nothing, and it looks like it worked." Reachable by every event type; fixed 2026-09-05 by draining before syncing.

5.7 What the tests prove, and the race PR #79 fixed

`gungnir-app/tests/failover.rs` drives a scripted `NodeLink` under a replay clock: the merge, the "history gone" case, and one detection accepted during an outage queued to the link. `gungnir-app/tests/failover_e2e.rs` signs one desktop `AppState` in to a `NodeApi` served in-process on loopback, over the real request and `tokio-tungstenite` client; drops the server's runtime to produce a real `HEARTBEAT_TIMEOUT` of silence; sees the fallback; brings the same `NodeApi` back on the same port; reconciles over the real `GET /v2/history`; and asserts the conflict is reported and switch-back refused until a person asks. Both run on every CI cargo `nextest run --workspace`. The `gungnir-node` binary, a second process, and a real network are not involved.

The e2e test failed four times: three on 2026-09-08 (once on main, Actions run 34232995914) and once on 2026-09-09 (run 34348951731), each with the link already showing connected. PR #36 read this as slowness and raised the envelope deadline from 5 s to 10 s. That was the wrong diagnosis, and the test now says so: "no deadline recovers an envelope that was never delivered, and the 10 s duly failed the same way" (`failover_e2e.rs:176-184`). PR #79 found the cause: `NodeLink` sets `connected = true` when the snapshot HTTP response lands and only then opens its `WebSocket` and subscribes. An envelope published into that window reaches no receiver, and because the subscription carries `from_seq 0` ("everything from now"), it is never replayed. A probe measured 1 of 25 envelopes lost, and 0 of 25 under 40-way CPU oversubscription: an ordering race, not load (facts R9 §4.5). The fix waits on `NodeApi::subscriber_count()` (`failover_e2e.rs:196-198`); the deadline returned to 5 s.

Note. "Connected" and "subscribed" were two states the link reported as one. The wrong fix widened a deadline that was never a criterion; the right fix waited on the condition itself.

5.8 Status, and the documentation defect to explain first

Of the four verification-table rows behind this chapter, only the `gungnir-app` "Backend switching" row is recorded as Specified, and it is recorded in `docs/architecture.md` (line 88, 2026-09-07), not in the verification table itself. The GAP-067 walk's own preamble in that file (lines 21-25) goes further than one row: it promotes every row still reading "Tested" to Specified without rewriting the cell, and "Tested" is exactly how the `gungnir-remote` (line 78), `gungnir-store` (line 95) and `gungnir-resilience` (line 115) rows still read. The Cross-layer "Disconnected reconciliation," `gungnir-remote` "Store-and-forward," and `gungnir-resilience` rows nevertheless sit in `docs/verification-capability-table.md` §2, whose preamble still says "None of these rows is a pass/fail gate yet," and `docs/mission/gap-analysis/gap-register.md` keeps GAP-067 Open. The reconciliation row's criterion still reads "resolved by the arbitration rule," contradicting what was built (a person resolves, D-15). These are repository contradictions to name, not sides to pick.

The honest answer. Failover, store-and-forward, and reconciliation are **built and gated** in the working sense that their tests run on every push over a real transport in one process on loopback, though three of the four table rows behind them are still §2 draft rather than pass/fail gates. The 100,000-detection bound is untested, the disconnected-expiry half of D-15 is not built, nothing checks what PN-18 shows beyond that it painted text under a headless egui probe (GAP-074, still in progress), and no desktop has ever failed over from a node on another machine.

Chapter 6 -- Two GPU contexts and the point-cloud path

6.1 Two contexts, by decision

The desktop uses the GPU twice, through two contexts that never share a buffer.

Context one: OpenGL, for everything the operator sees. `gungnir-app/src/main.rs` runs `eframe` with `Renderer::Glow`, so the egui panels and the three-d 0.18 viewport draw through one OpenGL context. This works because `eframe 0.29`, `egui_glow 0.29` and `three-d 0.18` share one `glow 0.14`: `gungnir-app/tests/gl_attachment.rs` asserts the `eframe-to-three-d` half at compile time, and `cargo tree -d` is the other half, reporting a single `glow 0.14.2` across all three -- which rules out `three-d 0.19` (ARCHITECTURE.md section 9). Status: **built, ungated** for the draw itself; `gungnir-viewport3d/src/gl.rs` records that "the drawn result has not been looked at," there being no headless probe for a GL draw.

Context two: a headless wgpu 22 compute device, for point-cloud fusion only. `gungnir-render::GpuContext::new()` requests a high-performance adapter with no surface and `Features::empty()`, and returns `RenderError::NoAdapter` on a host without a usable GPU. The egui-over-wgpu presentation path is **not built** (`EguiRenderer::is_active()` is always false); ARCHITECTURE.md section 9 retires the earlier one-shared-device claim.

Why Arc, not a lifetime

`GpuContext` holds `Arc<wgpu::Device>` and `Arc<wgpu::Queue>`, and `GpuFusionEngine` takes those Arcs rather than borrowing. The reason is written in `gungnir-render/src/lib.rs`: `Device` and `Queue` implement neither `Copy` nor `Clone` in `wgpu 22`, and a struct holding both the context and a `GpuFusionEngine<'a>` borrowing from it (`AppState`) would be self-referential, which safe Rust cannot express without pinning or a crate the workspace does not carry. The borrowed `GpuFusionEngine<'a>` is the rejected design (item 117). `gungnir-data-fusion` takes no dependency edge on `gungnir-render` (its manifest names only `gungnir-data` among workspace crates); its constructor takes plain `wgpu` types.

No zero-copy path, accepted

Fused output is designed to cross to the viewport as a CPU `PointBuffer`, and no fused output reaches it today: the voxel-fusion stage has no caller outside its test, and `PointCloudLayer` draws the two loaded clouds rather than a fusion result. ARCHITECTURE.md section 3 accepts the readback cost "because fusion output changes far less often than the frame rate," and the UI standards forbid attempting buffer sharing. `gpu/pipeline.rs` reads back the reduced sums through a staging buffer and solves the transform on the CPU. The viewport borrows the positions (a test checks `std::ptr::eq`) and decimates to a 20,000-point draw budget.

6.2 The CPU reference

Kabsch point-to-point (`transform_solve.rs`): center both sets on their centroids, form $H = \sum s_i t_i^T$, take $H = U \Sigma V^T$, and set $R = V U^T$, flipping the sign of V 's last column when $\det(V U^T)$ is negative so a reflection is never reported as a rotation; $t = c_t - R c_s$. `CpuIcp` (`cpu_reference.rs`) uses brute-force nearest neighbors, $O(n \cdot m)$ per iteration, so the reference stays legible. **Built**, and its criterion (translation within 0.05 m, rotation within 0.5 mrad, degenerate cases return `Divergence`) runs in CI -- though the row sits in the verification table's section 2, whose preamble still says none of its rows is a pass/fail gate yet.

Normals by PCA (`normals.rs`): the unit eigenvector of the smallest eigenvalue of each point's k-nearest-neighbor covariance. The sign is deliberately unresolved; collinear neighborhoods are `Degenerate`, not an arbitrary axis. **Built**,

unit-tested in CI.

Point-to-plane (`point_to_plane.rs`): for a small rotation the residual $n \cdot (R_p + t - q)$ is linear in $x = [\omega; t]$ through $n \cdot (\omega \times p) = \omega \cdot (p \times n)$, giving 6×6 normal equations $Ax = b$. The solve refuses systems whose eigenvalue ratio falls below 1e-6. The motivation is recorded: the single-sinusoid surface two sibling test files share gave $\text{cond}(A)$ above 1e17, about 280 with a second term -- from an uncommitted numpy check, so "recorded," not "reproducible." The module's seven tests cover the solve rather than repeating one result: a repeated-correspondence degeneracy refusal, a fewer-than-six-correspondences refusal, a target with no normals, an empty-overlap case, an already-aligned case, a single linearized solve that drives the mean residual below 3e-4 m, and one ICP-loop test that recovers a small known transform (about 0.02 rad, 3 cm) on a noise-free 36-point grid, corresponding by nearest neighbor, to within 2e-3 per coordinate. **Built**, unit-tested in CI, with no verification-table row, no GPU counterpart, and no caller outside its module.

6.3 The WGSL pipeline (item 117 in PR #46, item 119 in PR #53)

Stage	Kernel	What it does
Spatial hash	<code>spatial_hash.wgsl</code>	Uniform grid, atomic<u32> slot claims into fixed-capacity buckets; a dropped point can only cause a missed correspondence, never a wrong one
Correspondence	<code>correspondence.wgsl</code>	3×3×3-cell nearest neighbor, distance gate, optional normal-angle gate; complete because <code>cell_size >= max_correspondence_dist</code>
Reduction	<code>reduction.wgsl</code>	Workgroup tree reduction of Kabsch raw moments, no floating-point atomics, because neither WGSL nor this crate's <code>Features::empty()</code> descriptor guarantees one is available; <code>WORKGROUP_SIZE = 64</code> keeps 18 fields × 64 × 4 bytes under the 16 KiB baseline
Voxel fusion	<code>fuse Voxels.wgsl</code>	Confidence-weighted merge; no CPU oracle and no caller

The GPU path is **point-to-point only**; the reduced moments feed the same Kabsch solve `CpuIcp` uses. Two defects were found while the path was being built. On hardware, the cell size defaulted to the bounding-box diagonal, crowding a 36-point cloud into one or two cells (self-registration failed to converge); in plain `cargo test`, `GpuContext::new` was called eagerly from `AppState`, so three 0.01 s tests in `fires_deconfliction.rs` took over 300 CPU-seconds. The fix is `FusionBackend` in `gungnir-app/src/fusion.rs`: `new()` touches no `wgpu` API, `engine_for` resolves once and memoizes, and `NoAdapter` or `GpuInit` falls back to `CpuIcp`. `pointcloud::register` calls it from the tick once a loaded pair exists and steps one iteration per tick. The only input is a pair of files named in `PointCloudConfig`; no live sensor feeds this path, and it has not run in a deployment. PN-09 draws `PointCloudRegistrationLine { NotConfigured, Pending, Gpu, CpuFallback { reason } }`, so a failed GPU path reads as CPU-with-a-reason. Fallback, laziness and the PN-09 line are **built and gated** in plain `cargo test`.

6.4 GAP-098 and D-41

GAP-098 (item 112, **built and gated**, closed): `PointCloudConfig` names a source and a target, a loader thread reads both, and `PointCloudLayer` draws them; GAP-098 landed eleven tests. What loads end to end in the default build today is a pair that declares no CRS -- the five-point LAS fixture as both source and target (`a_pair_of_undeclared_files_still_loads_in_order`, 5 and 5 points). The Autzen COPC target is read through its hierarchy (4,767 points) and then **refused by name** in `pointcloud::place`, because GAP-102 made it declare NAD83 / Oregon GIC Lambert (ft) against a `local-enu` baseline; with the `crs` feature on, that file loads as both halves (4,767 and 4,767, `gungnir-app/tests/pointcloud_crs.rs`).

D-41 (2026-09-08) chose full projection support through `proj 0.31` over a narrower WGS84-only first step. Status: **built and gated in CI only**, behind the off-by-default `crs` feature, because `proj-sys` cannot build `libproj` on Windows MSVC. Horizontal only, no geoid shift; the `pyproj` check was one point, script not committed; the default Windows build -- the release target, on a `release.yml` that has never been run -- cannot convert: a file whose frame needs converting is refused, and the conversion call itself returns `NotImplemented`. GAP-023 and GAP-102 are both still in progress in the register.

6.5 The runner and D-10 as amended

Runner `gungnir-rtx-5060ti` is the owner's workstation. `gpu-fusion.yml` is `workflow_dispatch` only, permanently, by decision (D-10, 2026-09-08): the repository is public, so a `pull_request` trigger would let a fork run its own code on that machine, while `dispatch` can only be fired by a caller with write access. Running the command by hand was not declined -- `cargo test -p gungnir-data-fusion --features gpu-tests -- --ignored` runs on any GPU host, and did -- but it is not the record: `docs/release-governance.md` defines the evidence package as the CI logs of every gate that ran, and a hand run leaves none. The workflow fails a run that executed zero tests.

6.6 Exactly what is gated where

Check	Where it runs	Status
Four kernels parse and validate under naga (syntax, types, bindings; not numerics)	plain <code>cargo test</code> , CI	built and gated

Check	Where it runs	Status
Raw-moment Kabsch algebra vs direct centered computation	plain cargo test, CI	built and gated
CPU ICP 0.05 m / 0.5 mrad row; point-to-plane and normals unit tests	plain cargo test, CI	runs on every CI test job; the CPU ICP row is a section 2 draft row and point-to-plane has no table row at all
FusionBackend fallback and lazy rule; PN-09 line; the GAP-098 loader, including the undeclared pair that loads and the Autzen COPC target that is refused by name	plain cargo test, CI	built and gated
Four #[ignore]d GPU tests: two check the section 2 criterion against CpuIcp (transform within 1e-3 m/rad, inlier ratio within 0.01), one checks self-registration convergence, one checks voxel fusion's self-consistency (no CPU oracle for that stage)	--features gpu-tests -- --ignored on any GPU host, never inside plain cargo test; the recorded route is a gpu-fusion.yml dispatch on the owner's runner, which has not completed one	built, ungated in CI
g_low shared by eframe, egui_g_low and three-d	the eframe-to-three-d half in a compile-time test, plain cargo test, CI; the remaining half by cargo tree -d, by hand	the compile-time half is gated, the cargo tree -d half is not; the draw itself is unlooked-at

"Gated" in that table means the check runs and must pass on every CI push. That is a different thing from a verification-table pass criterion, which is why the CPU ICP row is qualified above and point-to-plane appears with no row of its own.

The honest answer. The four gpu-tests, two of them the differential checks against CpuIcp, passed once on an RTX 5060 Ti, per the implementing agent's self-report in ARCHITECTURE.md section 10 item 117; no log is in the repository. The one dispatch (run 34282656425, 2026-09-08) sat queued for over 19 hours; the earlier pull_request runs -- four in gh run list --workflow=gpu-fusion.yml at survey time, each cancelled at 24h0m, all on 2026-09-07 and all from before the D-10 amendment made the workflow dispatch-only -- each hit the 24-hour limit. (docs/mission/gap-analysis/gap-register.md, GAP-061, says nine such runs; the live listing shows four. Count the listing.) GAP-024 is In progress. Known documentation defects: the section 10 Open bullets still say the GPU step returns NotImplemented and GpuContext::new has no caller, and gpu-fusion.yml's header still says the gpu-tests feature is empty; both are stale against items 117 and 119.

Chapter 7 -- Security architecture

gungnir-security is a human-owned crate (docs/agent-workflow.md): every change carries the owner's signature or is marked unsigned. The design rests on one sentence from DN-22: "An accreditor asks about custody before they ask about ciphers."

7.1 Two kinds of caller (D-02)

D-02 (2026-09-04) split callers in two: mutual TLS for machines (desktops, nodes, peers, sensors) and short-lived signed tokens for operator sessions, with local accounts as the disconnected desktop's fallback. Tokens for everything and mTLS for operators were both rejected.

Operators

Passphrases are hashed with argon2id at the crate's defaults (Argon2::default, argon2 0.5, random salt) and stored as PHC strings (gungnir-security/src/session.rs::hash_passphrase). Verification refuses an unknown operator and a wrong passphrase with the same AuthFailure::Rejected, and computes a hash even when the account is unknown, so timing does not tell a prober which half was wrong. Failure backs off (doubling from a small base, capped) and never locks out: DN-23 rule 3 says an operator locked out of a C2 console mid-engagement is the worse outcome (session.rs::failures_back_off_and_never_lock_out). **Built and gated.**

A session token is HMAC-SHA256 over JSON claims, encoded <hex payload>.<hex mac> (gungnir-security/src/token.rs). The claims are operator, role, established, and expires, nothing else; the module doc says "It is not a capability." A token says who and until when; what the operator may do is decided per request by authz::role_permits against the nine-role matrix, so widening a role never reissues tokens and a stolen token carries no more authority than its operator has now. The MAC is checked in constant time (subtle::ConstantTimeEq::ct_eq, token.rs:146) before the payload is parsed, and a tampered token and an expired one fail identically. A MAC rather than a signature (D-20): the node both issues and verifies, so a public key would buy nothing and would put a private key in the process outside the custody boundary. **Built and gated.**

Attribution follows DN-23 rule 1: authentication never invents it. SessionState is SignedIn | NobodySignedIn | Expired | StoreUnavailable, not Option<OperatorSession>, and a decision recorded with nobody signed in carries

operator_id: None rather than a role name (gungnir-app/tests/authentication.rs::a_concurrence_names_the_operator_only_after_a_real_sign_in). One caveat: OperatorSession has public fields and is built by struct literal in three places -- gungnir-security/src/session.rs:543 (inside sign_in, after verify_account returns Ok), gungnir-security/src/token.rs:154, and gungnir-api/src/transport.rs:235 -- each after a verification, so the property holds by convention and review, not by field privacy.

Machines

Mutual TLS is rustls 0.23 with WebPkiClientVerifier::builder, so a client certificate is required, not optional (gungnir-api/src/tls.rs); it is opt-in, configured by all three of GUNGNIR_TLS_CERT, GUNGNIR_TLS_KEY, and GUNGNIR_TLS_CLIENT_CA, and a partial set is refused rather than half-applied (tls.rs::from_env_parts). The default node sets none of them and serves plaintext on loopback, which 7.6 returns to. Plaintext off loopback is refused by ApiError::UnprotectedBind, an error rather than a warning because "a node that logged and carried on would still be listening" (gungnir-api/src/transport.rs:1017-1019). The test PKI is generated per test by rcgen and never checked in (D-22). The certificate's subject CN is the party (gungnir-api/tests/party.rs::the_subject_common_name_is_read_from_the_certificate). Two functions resolve it: transport.rs::machine_identity (line 911) admits a connection whose CN the baseline maps to a registered sensor or effector identity, and the sensor and effector routes try it first; transport.rs::caller (line 891) otherwise treats a verified certificate with no Authorization header as a machine and refuses it unless its party holds an exchange agreement -- "authentication answers who you are, the agreement answers what you may do" (DN-18). One correction to carry: transport.rs:16 still says every route but POST /v2/session requires a token, which is true of operators only. **Built and gated.**

Releasability (DN-17) is enforced per party at the API: collection responses are filtered by removal with a withheld count, so a peer never mistakes a shortened list for the whole picture. DN-17 rule 4 further requires that a single restricted item answer not-found rather than forbidden, so the error code discloses nothing -- a criterion the model expresses as gungnir_model::permits_single returning false, with unit tests but no caller: v2 exposes no single-item route and there is no 404 path anywhere in gungnir-api. That rule is designed, not built. Releasability::combine takes the most restrictive input, which stops a report laundering a restricted track by aggregation; #[default] Internal keeps anything unmarked inside. Eleven tests in gungnir-model/src/releasability.rs. Collection filtering is **built and gated**; the single-item rule is **not built**. DN-17, the design behind both, was signed by the owner 2026-09-05 (releasability.rs:7).

7.2 Custody: a provider with no getter

KeyProvider (gungnir-security/src/keys.rs) exposes exactly active, state, seal, unseal, rotate, sign. DN-22 §3: "A provider that hands out key bytes has no custody boundary at all." sign exists because a TLS handshake needs a transcript signature and "the resolution is not to add a getter": rustls::sign::SigningKey is a trait, so the provider signs and gungnir-remote/src/identity.rs builds a host's certificate over a key that never leaves it. Two tests in gungnir-app/tests/architecture_compliance.rs guard the surface; honestly described, they are a nine-name signature tripwire plus a six-method pin, not a type-level proof.

The sealed form is <purpose 1 byte> || <version 1 byte> || <96-bit nonce> || <ciphertext || tag> under AES-256-GCM -- provider.rs::HEADER is 2 + 12, and asymmetric.rs uses the same layout -- so every ciphertext names the key that opens it and rotation never rewrites existing data. DN-22 amendment 1 (b) specifies KeyId as 8 bytes on the sealed form (docs/design/DN-22-key-management.md:216) and the implementation writes 2 (provider.rs:41), one more doc/code mismatch of the kind 7.6 collects. The Debug impls on key-holding types print no material (asymmetric.rs, keystore.rs, provider.rs, managed_service.rs, account_store.rs); FileEventJournal's reports whether it is sealing, never anything about the key.

Profile	Mechanism	Status
Disconnected desktop, passphrase	One file, keystore.sealed, wrapping key derived by argon2 from the account passphrase with its own salt	Built and gated; signed 2026-09-06
Disconnected desktop, OS keystore (D-39)	Same file; the argon2 input is a high-entropy secret held by Keyring 4.2.0 (Windows Credential Manager, macOS Keychain, Linux Secret Service)	Built and gated against the keyring_core::mock::Store and the no-keystore fallback, ungated on every real backend; signed 2026-09-08 (PR #63). Real Windows Credential Manager round trips were manual on the owner's machine; macOS Keychain and the Linux Secret Service backend are unexercised

Profile	Mechanism	Status
Escrow (D-27, D-30)	Journal data keys wrapped to a SecurityOfficer public key by ECDH P-256, HKDF-SHA-256, AES-256-GCM; the private half never on a node; KEY_ESCROW_RECOVER (key.escrow_recover) is the officer's only permitted action	Wrapping, recovery and the authority rule are built and gated; signed 2026-09-06. The offline recovery tool that would hold the officer's private half and write the DN-22 §11 audit row is not built : no such binary target exists in any member Cargo.toml, and EscrowOfficerKey::recover has no caller outside tests
Cloud node, ManagedService (D-42, DN-22 amendment 5)	Envelope encryption: AWS KMS or Azure Key Vault wraps a 32-byte secret once at start; seal, unseal, and rotation are local; only sign is a per-call round trip	Built, unsigned. Gated against an in-crate fake (one call at open, zero across a thousand seals); the SDK backends compile with three #[ignore]d tests; no cloud round trip has ever run

The OS-keystore review found a first-run race: a second process's set_password landing between this one's write and its return. ensure_secret now re-reads the store rather than trusting what it generated (os_keystore.rs::read_back). ARCHITECTURE.md says closed; the code's own comment says it narrows the window and names the residual, since no backend offers compare-and-swap. Say "narrowed." The same 2026-09-08 work found a second cost and closed it before the entry landed: unconditionally issuing a persistent identity in link_tls_for left about 190 stray Windows Credential Manager entries behind on one full test run, so the persistent path is now attempted only when security.key_provider is already OperatingSystemKeystore (ARCHITECTURE.md item 111, "A cost found and fixed within this same entry, not carried into it").

7.3 Journal sealing with the edge inverted

DN-22 §4 forbids a new dependency edge, and gungnir-store naming KeyProvider would be one. So gungnir-store/src/sealing.rs declares a two-method JournalSealer and the binary wires a provider to it; the store learns nothing about keys, purposes, or ciphers. Each line is sealed on its own, so the file stays JSON-lines and a torn final line still drops cleanly. If sealing fails, the append fails: falling back to plaintext "is the silent downgrade AP-02 exists to prevent." A desktop with no provider configured starts and reports EncryptionStatus::NotConfigured, which the strip does not draw as a fault; one that has a provider it cannot open -- nobody signed in yet, or an unreachable keystore -- starts anyway and reports EncryptionStatus::UnavailableWritingPlaintext, which the strip does draw as a fault. The code makes that distinction deliberately -- "Never configured is not a fault; a keystore that could not be reached is" (gungnir-app/tests/encryption_status.rs:251) -- and the two arms that produce the states are gungnir-app/src/state.rs:1076-1087. Gated by gungnir-node/tests/encryption_at_rest.rs::a_sealed_journal_does_not_contain_its_plaintext against the in-process provider, with rotation and switch-on tests that leave earlier lines readable. **Built and gated.**

7.4 Audit

AuditLog records operator, action, mission time, and detail, append-only and independent of the mission journal; every decision, sensor command, requirement, and review leaves a row (gungnir-app/tests/audit_trail.rs). Key use is not audited per operation; DN-22 requires a row for every rotation, retirement, destruction, override, and escrow recovery. Escrow recovery is the one that has no writer: as 7.2 records, DN-22 §11 assigns that row to an offline recovery tool that is not built, and nothing in the workspace writes an AuditEntry under key.escrow_recover. Splitting PUBLISH_EXCHANGE from RELEASE_PRODUCT, so the two acts are distinguishable by action name alone, exposed that Supervisor had lacked RELEASE_PRODUCT since the initial commit; a role_permits test now pins the two to agree for every role (signed 2026-09-08, PR #22).

7.5 Two defects the tests caught

The honest answer: one was a test defect, one was real, and both are in ARCHITECTURE.md item 64.

The first mutual-TLS test asserted that the handshake completed. Under TLS 1.3 the client finishes before the server has validated its certificate, so connect returning Ok says nothing; the rejection arrives on the first read. Rewritten to make a request, the test confirmed anonymous and wrong-authority clients had been refused all along: "the implementation had been right and the test had been measuring the wrong thing."

The second was real. read_session tolerates a torn final line, and that tolerance swallowed a sealing failure: a journal whose key was missing returned an empty session rather than an error, so an operator would have seen a mission that recorded nothing. A sealing error is never treated as a torn line now (encryption_at_rest.rs::a_sealed_journal_without_its_key_says_the_key_is_missing).

7.6 What is not built, and what the repository says twice

Not built: a real CA, HSM, or fielded deployment -- no certificate anywhere in this workspace comes from a certificate authority. The test PKI is rcgen-generated per test, and a node's own serving identity is a self-signed rcgen certificate issued at runtime (gungnir-remote/src/identity.rs::issue, params.self_signed) over a key the provider never releases; that runtime path has never been exercised outside a test. Also not built: any cloud KMS round trip; the offline escrow-recovery tool DN-22 §11 assigns the key.escrow_recover audit row to (7.2); and release.yml has never run, so cargo audit, the SBOM, and cosign signing exist as workflow text only. Unsigned: ManagedService. Documentation defects to name before a reviewer does: os_keystore.rs, account_store.rs, keystore.rs, and gungnir-remote/src/identity.rs still read "not signed" although ARCHITECTURE.md items 103, 104, and 111 record the 2026-09-08 signature; DN-22 §14 says "once per rotation" while managed_service.rs::rotate makes no service call; the workspace Cargo.toml says rcgen is never a normal dependency while gungnir-remote lists it under [dependencies] (line 32); and, as 7.2 notes, DN-22 amendment 1 (b) specifies an 8-byte KeyId on the sealed form while provider.rs::HEADER is 2 + 12. The default baseline serves plaintext on loopback and refuses every caller: the honest default, not a gap.

Chapter 8 -- The architecture process: TOGAF ADM, UAF, principles, contracts, generators

8.1 The two frameworks, for a non-technical reader

UAF (Unified Architecture Framework, an OMG standard, version 1.2 here) is a grid of standard views for describing a defense system: what it must do, who operates it, what services and software provide each function, what standards it meets, and how it is deployed. TOGAF (The Open Group Architecture Framework, version 10 here) is a method rather than a set of views: the Architecture Development Method, or ADM, is a cycle of phases (Preliminary, A through H, plus requirements management) that governs how an architecture is agreed, principled, contracted, checked for compliance, and changed. Gungnir uses both because they answer different questions: the UAF views hold the content, and the 23 TOGAF documents under docs/architecture/togaf/ reference those views rather than redrawing them (docs/architecture/togaf/README.md).

8.2 How the ADM was tailored

docs/architecture/togaf/preliminary/tailored-adm.md states the test applied to every candidate deliverable: "does it change what someone does, or does it only describe what another document already holds? Only the first kind is produced." The document's own count is that nine of TOGAF's named deliverables are omitted or merged, and it states that each is "a decision with a reason, not an oversight." That count cannot be reproduced from its own table, which marks four deliverables **Omitted** and two Merged; adding the four phase B, C, and D architectures produced as short documents over the UAF views rather than as full deliverables gives ten, not nine. Treat the nine as a documentation defect in tailored-adm.md. The nine rows below are those four omissions, the two mergers, the four architectures collapsed into a single row, and the two deliverables produced in an unusual form. One row carries no reason: the Architecture Board terms of reference has an empty reason cell. That too is a defect in tailored-adm.md, not a reason to invent one.

TOGAF deliverable	Disposition	Reason given
Request for Architecture Work	Omitted	The owner is the sponsor and the architect; the plan set is the request
Communications Plan	Omitted	One team, one repository
Business Transformation Readiness Assessment	Omitted	No organization is being transformed; the product is greenfield
Capability Maturity models beyond the assessment	Omitted	Would not change any decision this year
Architecture Board terms of reference as a separate document	Merged into the governance framework	No reason stated
Implementation Governance Model	Merged into architecture-contracts.md	The review pipeline already is the model
Business, Data, Application, Technology Architecture	Produced as short documents over the UAF views	Content stays in the views
Architecture Contract	Produced as C-01 to C-17	Expressed as review checks, not a signed supplier document
Compliance Assessment	Produced	Run against the code, not asserted

The remaining named deliverables are produced as ordinary documents.
Iterations are not calendar cycles. One ADM iteration is one engineering increment from docs/gungnir-capabilities.md section 7:

Increment	ADM emphasis	Closes with
I1 Productize the core (done)	Phases B, C, D	The architecture as described in ARCHITECTURE.md section 7
I2 Integrate real data	Phases C and D	Transition architecture T2 and a compliance assessment
I3 Close the decision loop	Phases B and C	T3 and a compliance assessment
I4 Operationalize and scale	Phases D and F	T4, a compliance assessment, the release evidence package

Phases E, F, G, and H run continuously: "the work packages are the gap register, the migration plan is the closure roadmap, governance is the review pipeline on every change, and change management is the decision log." The governance framework records the board of one with two reviewers on call as "a governance weakness rather than a simplification," and the stakeholder map states that no stakeholder has been interviewed.

8.3 The UAF view set

Everything is built from one element registry, docs/architecture/uaf/model/elements.yaml (11 element kinds: capabilities, operational performers, activities, services, resources, personnel types, standards, projects, information elements, actual resources, and requirements) and relationships.yaml (9 relationship kinds). The registry feeds 58 view files, 32 PlantUML and 2 Mermaid diagram sources, and 7 traceability matrices (counted under docs/architecture/uaf/ at ee12f7e).

docs/architecture/uaf/tools/build_uaf.py does three jobs: it regenerates the resources, uses, requirements, satisfies, and carried_by registry sections from the crate manifests and the requirements tables; it generates 25 of the 58 views from code and mission text; and it checks the registry. The generated views are Rs-Sr (crates by layer), Rs-Cn (the dependency graph), Rs-If (every public trait and its method signatures, parsed from source), Sv-If (the two facade traits, the two API traits, and the API endpoint table), If-Sr (every public type in gungnir-model, with a class diagram), and twenty operational views, one process view and one interaction scenario per mission thread:

Thread	Op-Pr-MT (activity diagram, system and human lanes)	Op-Is-VG (sequence diagram)
MT-01	One-way attack drone raid against a defended-asset list	Night raid on the power station
MT-02	Mixed salvo of cruise missiles, drones, and decoys	Salvo with the raid
MT-03	Small UAS over a protected site	Quadcopter over the airfield
MT-04	Uncrewed surface vessel attack on a port or anchored ship	USV attack on the anchorage
MT-05	Surface picture compilation	The afternoon surface picture
MT-06	Convoy and battery tracking with cueing of fires	Battery on the far bank
MT-07	Sensor management under electronic attack	Raid under GNSS denial with a radar loss
MT-08	Collection management and identification evidence fusion	Who is that aircraft
MT-09	Defended-asset planning, laydown, and rehearsal	Planning the week's laydown
MT-10	Disconnected operation and reconnection	The port cell loses the node

The Op-Pr views are parsed from docs/mission/mission-threads.md, each step mapped to operational-activity ids by a hand-kept table in the generator; the Op-Is views are parsed from docs/mission/vignettes.md. The 33 remaining views are authored.

The seven matrices under docs/architecture/uaf/traceability/ (capability-to-activity, activity-to-service, service-to-resource, resource-to-standard, role-to-activity, requirement-to-capability, requirement-to-resource) are all generated from relationships.yaml; the last two were added under GAP-083 when requirements became the eleventh element kind. Design intent is tagged (planned) rather than drawn as implemented. The registry check (check, build_uaf.py lines 817-885) is the no-orphan rule: every relationship endpoint exists; every leaf capability is exhibited by a performer; every activity is realized by a service or performed by a human role; every service is implemented by a resource and its code path resolves to a real pub item in that crate's source; every requirement naming a capability has a satisfies row; every element token in any diagram source exists. Run with --check at ee12f7e it reports 0 problems and 64 notes. The CI job uaf-registry (.github/workflows/ci.yml lines 107-123) runs the full regenerate-and-check and then git diff --exit-code -- docs/architecture/uaf, so a manifest or mission-text change without a regeneration fails the check on the pull request. Status: built and gated. Its first run after hosting found an edge (gungnir-remote to gungnir-security) in a manifest but missing from the top-level dependency-graph table, and commit b8002d8 removed a nondeterministic ordering that had failed the diff check at random.

Known defects in this set: uaf/README.md still says "Ten kinds" while the registry has eleven; every generated file carries a hard-coded DATE = "2026-09-04" whatever day it was regenerated; the generator's member regex matches the exclude line, so Rs-Sr.md counts 51 workspace members where the manifest has 50; and rendering of the 34 diagram sources was not exercised on 2026-09-04 for want of a renderer on the drafting host, per the UAF README, with no rendered output in the tree at ee12f7e.

8.4 Principles AP-01 to AP-17 and contracts C-01 to C-17

Each principle has one enforcing contract. All seventeen of each were signed by the owner on 2026-09-05, "after each batch was checked against what the code actually does rather than against what the document asserts" (architecture-principles.md). As of ee12f7e, seven contracts have a test or CI job on every push; of the other ten, eight are reviewer-checklist items and two (C-16, C-17) rest on repository settings and on review of the verification table.

Contract	Principle	Machine check	Status
C-01	AP-01 Recommend, never act	gungnir-app/tests/no_execution_without_decision.rs (a static scan plus a driven desktop)	Built and gated; not dispensable
C-03	AP-12 Not-implemented is an explicit error	no_reachable_todo.rs; the unwrap scan in architecture_compliance.rs	Built and gated
C-04	AP-03 Named actor and time	gungnir-app/tests/audit_trail.rs	Built and gated
C-05	AP-04 Unclassified, openly sourced	Citation half: every document-and-section citation in a source comment resolves to a file and a numbered heading	Citation half built and gated; sourcing half is review
C-07	AP-06 One owning crate per type	Definition scan over 18 shared primitives	Built and gated
C-11	AP-10 One-way dependencies	gungnir-app/tests/dependency_graph.rs, five tests, both directions against ARCHITECTURE.md	Built and gated
C-14	AP-14 Pinned, recorded stack	every_workspace_dependency_is_recorded; the cargo-deny CI job	Built and gated
C-16	AP-16 Gates cannot be waived	The workflows exist and run; whether branch protection makes them merge conditions is not visible in the repository	Partly verifiable
C-17	AP-17 A measure has a target before a test	A diff check of the verification table	Not built; rests on review and on the table's own statement that "No criterion below was changed to make a test pass"
C-02, C-06, C-08, C-09, C-10, C-12, C-13, C-15	Honest status, profiles, provenance, journal, releasability, traits, binaries wire, two GPU contexts	Reviewer checklist	Review items (C-09 cites a journal round-trip test that was not verified for this guide)

Five findings were recorded at signature rather than resolved first. First, C-01 and C-04 had no running check (GAP-039 and GAP-059, both Open then; both tests now exist and both gaps are Closed). Second, AP-08 was signed knowing D-04's desktop buffering can lose up to 5 s of journal on a hard kill. Third, `todo!()` reachability was asserted, not proven, with 22 calls remaining; ARCHITECTURE.md section 10 item 78 later records that every one was the body of a public function, "so the claim that none was reachable was never true," and all are now named errors held at zero by `no_reachable_todo.rs`. Fourth, two capabilities (`solve_assignment`, `compute_metrics`) are free functions rather than traits, accepted as exceptions to AP-11. Fifth, AP-16 had no enforcement mechanism because the workspace was not yet under version control; hosting came 2026-09-07.

8.5 The compliance assessment

phase-g-implementation-governance/compliance-assessment.md records one run, 2026-09-04: "Six passes, one observation, two findings, one not verifiable." C-11 passed on 147 edges across 50 manifests; C-07 on nine shared types; C-14 on twenty recorded dependencies; the unwrap scan on 148 files; 143 section citations and 291 Markdown files with zero broken links. The observation counted 32 `todo!()` bodies. The two findings became gaps: CA-F1 (every check was an ad hoc script, "A contract enforced by a person who remembers to run a script is a preference") became GAP-081, and CA-F2 (nothing proved the `todo!()` bodies unreachable) became GAP-082. Both are Closed; the five checks became the test files in the table above on 2026-09-06. A third note, CA-F3, came out of a check that passed rather than from a finding: it recorded that the standards prose omitted the 3D-data crates from the layer order, and it was folded into GAP-081, which is why `dependency_graph.rs` has a `Layer::Data`. Since that run the numbers moved: 169 normal edges among the 50 members (172 with `gungnir-fuzz`), 18 shared types, zero `todo!()`. The `togaf/README.md` results table still shows the 2026-09-04 figures and "Are any of these checks automated? No." That is the dated record, not the current state.

8.6 The register, decision ledger, and roadmap generator

docs/mission/gap-analysis/tools/gen_gaps.py generates five documents from one data set: gap-register.md, coverage-matrix.md, technical-gap-map.md, closure-roadmap.md, and decisions-needed.md. It is deterministic, so a clean re-run is also the check that no hand edit crept in. Five guards stop it with SystemExit: a duplicate gap id (line 918), a duplicate decision id (921), a decision with no outcome (931), a dependency cycle (941), and a capability with less than full coverage and no gap (963). The CI job gap-register (ci.yml lines 125-149) runs the generator and git diff --exit-code -- docs/mission/gap-analysis. Status: built and gated. The job's own comment says why it exists: "the omission it checks for has already happened twice." D-34's rows were dropped by a commit whose copy of the generator predated the decision, and GAP-092 and GAP-093 were restored from git history after the same thing; "Both were found by hand, after the fact, by someone who happened to be looking."

The duplicate-id guard landed 2026-09-09 (PR #76, commit 8a62287) after four number collisions among branches running in parallel between 2026-09-06 and 2026-09-08, and a fifth followed it the same morning (ARCHITECTURE.md section 10, Open block, and the GAP-095, GAP-098, and GAP-103 entries). The four the guard was written for are the ones its own commit message names: the night-theme gap was filed as GAP-090 on 2026-09-06 and renumbered to GAP-094 on 2026-09-07, which was itself already claimed, so it moved again to GAP-095, two collisions for one gap; two open pull requests claimed GAP-097 within hours on 2026-09-08; and two changes claimed GAP-102 on 2026-09-08, the younger moving to GAP-103 on 2026-09-09. The fifth is the one the guard did not catch in time: GAP-104, filed as GAP-101 on 2026-09-08 and moved to 103, moved again to 104 in commit 5b7f20f at 08:46 on 2026-09-09, eighteen minutes after the guard merged at 08:27. The rule adopted: the merged and already-cited claim keeps the number, the younger moves.

8.7 The DoDAF cross-reference

docs/architecture/togaf/framework-cross-reference.md maps 36 DoDAF 2.02 views (AV-1, AV-2, CV-1 to CV-7, OV-1 to OV-6c, SV-1/2/4/6/7/10b, SvcV-1/2/4/6, DIV-1 to DIV-3, StdV-1/2, PV-1 to PV-3) to the UAF view or document that answers each and the TOGAF phase that discusses it. Three mismatches are stated: DoDAF has no security viewpoint, UAF actual resources have no clean DoDAF home, and TOGAF phases and DoDAF views answer different questions. Section 4 states what is not claimed: "no DoDAF meta-model conformance has been asserted, and no accreditor has seen it." The decision recorded 2026-09-04 was a table and nothing further.

The honest answer, if asked what this process has not done: the compliance assessment has run once; no second architect has reviewed the views; phases B, C, and D still await their reviewers; the registry's authored service statuses are the 2026-09-04 snapshot (SV-01 Tracking still reads scaffold while PIPELINE_IMPLEMENTED is true); and the dates before 2026-09-07 are the documents' own, since git history begins with hosting.

Part III, Chapter 9 -- Rust design decisions by layer and crate

Every point below names a file, and every capability carries its status word. Where the repository's documents disagree with its code, the disagreement is named. Figures are as of main ee12f7e, 2026-09-09.

9.1 Traits as facades and the embedded/remote switch

The two service facades are the deployment seam. gungnir-tracking-service/src/lib.rs:80-134 defines TrackingService: Send + Sync with submit_detection, poll, tracks, is_healthy, and two defaulted methods (bearing_rays, pipeline_stats) that return empty values so a backend with no pipeline behind it "has nothing to report" rather than a health flag claiming a capability it does not carry. gungnir-intercept-service/src/lib.rs:105-131 defines InterceptService the same way, returning PlanOutcome { Fresh, Stale, NoPlan } after GAP-066 found that a stale plan "came back looking exactly like a fresh one".

gungnir-app::AppState holds Box<dyn TrackingService> and Box<dyn InterceptService> (gungnir-app/src/state.rs:61-62); the desktop and the node depend on the traits, never on the eight crates behind the tracking one or on the allocator behind the intercept one. The switch is gungnir_config::BackendConfig (gungnir-config/src/lib.rs:915-923): Embedded is the serde default, Remote { endpoint } selects gungnir-remote's client implementations of the same two traits. ARCHITECTURE.md §2 still prints the pre-GAP-066 method shapes; the code is the truth, and that is a known documentation defect.

Health is reported, never inferred. LiveTrackingService::is_healthy is self.pipeline_alive && gungnir_fusion_async::PIPELINE_IMPLEMENTED (lib.rs:745-747); gungnir-remote's connect returns Ok as soon as the link task is spawned and both services report unhealthy "until a snapshot has actually been received" (gungnir-remote/src/lib.rs:220-241). The embedded/remote switch and mid-session fallback are **built and gated** in CI (gungnir-app/tests/failover.rs, failover_e2e.rs) against an in-process loopback NodeApi, not a second process or a fielded node.

Trait objects also guard the custody boundaries: `gungnir-store/src/sealing.rs:39` declares `JournalSealer`: `Send + Sync` and the binary wires `gungnir-security` into it, so `gungnir-store` takes no dependency edge; `gungnir-security/src/keys.rs:119-151` declares `KeyProvider` with exactly six methods and, deliberately, none returning key material.

9.2 Newtypes and type-level invariants

Identifier newtypes are tuple structs with one owning crate each, enforced by `architecture_compliance.rs:131` (`every_shared_type_has_exactly_one_definition`).

Type	File	What it carries or prevents
<code>TrackId(pub u64), ResourceId(pub u32)</code>	<code>gungnir-core/src/ident.rs:19, 36</code>	one definition workspace-wide; re-exported upward
<code>MissionTime(pub f64)</code>	<code>gungnir-model/src/time.rs:16</code>	mission seconds, distinct from <code>SourceTime</code> in <code>gungnir-time/src/lib.rs:19</code>
<code>SessionId(pub u64)</code>	<code>gungnir-model/src/lib.rs:162</code>	shared by journal, replay, reporting, resilience, review; moved out of <code>gungnir-store</code> so no edge reached across
<code>DecisionId(pub u64)</code>	<code>gungnir-model/src/plans.rs:40</code>	lets a facade key on a decision without depending on <code>gungnir-command</code> (DN-06 §3, AP-10)
<code>SensorId, SensorTaskId, PlanId, AssetId, OperatorId, PendingApprovalId</code>	<code>gungnir-model, gungnir-security/src/lib.rs:87, gungnir-command/src/lib.rs:148</code>	every <code>ApiHandler</code> call names the authenticated caller
<code>PlainListener(pub TcpListener)</code>	<code>gungnir-api/src/tls.rs:241</code>	a listener that is not TLS is a different type from <code>TlsListener</code>
<code>EscrowOfficerKey(SecretKey)</code> (private field)	<code>gungnir-security/src/asymmetric.rs:356</code>	Debug prints <code>"EscrowOfficerKey(P-256, private)"</code> ; no binary constructs one

The structural invariants matter more than the newtypes:

- **A bearing cannot initiate a track, by the absence of a function.** `gungnir-fusion-async/src/lib.rs:38-43`: `Detection` "is a position and only a position"; `BearingDetection` (`:71-88`) is a separate struct with `elevation_rad: Option<f64>` ("a missing elevation is not a zero one"), and there is "no function that accepts one and creates a track". The `SubmissionEnum` (`:103-111`) keeps the two on one channel without a flag to misread. **Built and gated**: 100 consistent bearings initiate 0 tracks (table row `fusion-async` "Bearing update"). In `gungnir-model` a bearing is a `Measurement::Bearing` variant, not a separate type; the distinct type lives at the pipeline.
- **ReportedPosition is never a track.** `gungnir-model/src/exchange.rs:62-80`: a self-report "is the same case" as DN-16's launch warning, "and takes the same answer". Its policy consumer, the fires three-state friendly check (`gungnir-policy/src/fires.rs:41-60`, GAP-090), is **built, unsigned**; nothing outside a test constructs a `ReportedPosition` yet.
- `Concurrence::UnattributedRole` (`gungnir-model/src/requirements.rs:47-55`) records that "this deployment could not say who acted" as a variant, "not an anonymous operator", so an auditor never reads a role name as a person.
- `Assignment::total_cost: Option<f64>` (`gungnir-association/src/assignment.rs:95-101`, D-43, signed 2026-09-09): None means the optimal value is not representable, which "an all-finite matrix can still do"; the pairing is the optimum either way. Chosen over an error variant because the callers use only the pairing.
- **RFS output carries no TrackId field** (`gungnir-fusion-async/src/dense_group.rs:33-40`): "a check can be forgotten and an absent field cannot be read"; asserted by exhaustive destructuring so adding one stops the test compiling.
- **Indices, not identifiers, across the allocator boundary** (`gungnir-allocation/src/lib.rs:59-69`): an earlier (`ResourceId, TrackId`) field filled with row numbers was "type-correct and wrong".
- **Named NotImplemented variants**, six of them with `what/waiting_on` fields -- `FilterError` (`gungnir-filters/src/lib.rs:112-117`), `RfsError` (`gungnir-rfs/src/lib.rs:1546`, the delta-GLMB, **not built**), `TrackFusionError` (`gungnir-track-fusion/src/lib.rs:81-85`), `DataError` (`gungnir-data/src/lib.rs:51-56`), `FusionError` (`gungnir-data-fusion/src/lib.rs:58-62`, "distinct from `Divergence`: a solver that ran and did not converge and a solver that does not exist are opposite claims") and `ViewportError` (`gungnir-viewport3d/src/lib.rs:33-37`, streaming, **not built**) -- and three that name the capability without the field pair: `InteropError::NotImplemented(&'static str)` (`gungnir-interop/src/lib.rs:40`) carries the codec's own name (Cat 048/205 encode, STANAG 4676, **not built**), and `ApiError::TransportNotImplemented` (`gungnir-api/src/lib.rs:37`, a unit variant) and `RemoteError::TransportNotImplemented(String)` (`gungnir-remote/src/lib.rs:207`) are the transport's.

- **Safe defaults in the derive:** `WeaponsControlStatus` defaults to `Hold` (`gungnir-model/src/policy_settings.rs:43-51`); `Releasability` to `Internal`; `SessionState` is a four-variant enum, `"not Option<OperatorSession>"` (`DN-23 §3`).
- **Const generics:** `KalmanFilter<Model, const N, const M>` (`gungnir-filters/src/kalman.rs:46-64`) makes a dimension mismatch a compile error, and the motion model is a type parameter so `F` and `Q` are evaluated at the `dt` actually taken.

Not present: no `PhantomData` type-state machines and no `#[non_exhaustive]` public enums -- the twenty textual hits for the latter are all `finish_non_exhaustive()` calls in `Debug` impls. `#[must_use]` appears on 688 items with `must_use_candidate` allowed, so each is deliberate.

9.3 Error philosophy

The rule (`docs/agent-coding-standards.md §3.1`): every fallible public function returns `Result<T, E>` with a crate-local `thiserror` enum (51 files declare one); an unimplemented capability returns an explicit variant rather than panicking on a runtime path; `unwrap/expect` only in tests, `main()`, and debug-gated checks; adding `anyhow` is a stack change needing sign-off.

Enforcement is a line-oriented source scan, not a clippy lint: `gungnir-app/tests/architecture_compliance.rs:200-256` (`no_unwrap_or_expect_outside_tests_and_main`) strips `#[cfg(test)]` modules and `fn main`, and exempts `/tests/`, `/benches/` (19 unwraps live there today), `/examples/`, `/fuzz_targets/`, and the `gungnir-testkit`, `gungnir-oracle`, and `gungnir-fuzz` crates. `no_reachable_todo.rs` finds zero `todo!()` anywhere. Both run under `cargo nextest run --workspace` in `ci.yml`; neither has a verification-table row, and `panic!/unreachable!` are not scanned.

"A `Result` even here": `gungnir-security/src/token.rs:167-176` returns `Result<Vec<u8>, InvalidLength>` from `mac` although `HMAC` accepts any key length, "because a panic here would take the transport down with it" (signed 2026-09-06, GAP-081). `Mutex` poisoning is recovered rather than propagated in the `loom` shim (`unwrap_or_else(PoisonError::into_inner)`, `gungnir-fusion-async/src/sync.rs:141-150`) and in the `journal`, which warns before recovering (`gungnir-store/src/lib.rs:245-257`), each with its reason written beside it.

The `Associator` trait is fallible by design: `associate` returns `Result<Vec<Option<usize>>, AssociationError>` (`gungnir-association/src/lib.rs:43-53`) "when the cost matrix does not define an optimum". Refusal beats guessing throughout: `NonFiniteCost { row, col }`, `TooManyHypotheses`, `MalformedScene` ("a partial enumeration returns association probabilities that look ordinary and are wrong", `assignment.rs:69-90`); `NotPositiveDefinite`, `SingularInnovation`, and an `IMM` with fewer than two modes refused (`gungnir-filters/src/lib.rs:118-127`); a failed seal fails the `append` rather than falling back to `plaintext` (`gungnir-store/src/sealing.rs:24-30`); `ApiError::UnprotectedBind` is an error, not a warning, because "a node that logged this and carried on would still be listening".

Refusals are typed at the wire. `POST /v2/detections` answers 409 with both schema versions named when the caller's version is ahead, behind, or absent (`gungnir-api/src/transport.rs:1235`, `v2/mod.rs:226-228`); that is the only route with an inbound schema check. `POST /v2/plans/{id}/decision` answers 501 after authentication because "this node runs no approval queue" (`transport.rs:1620`). Where distinguishability is the vulnerability, the error is deliberately one variant: `unknown operator` and `wrong passphrase` are both `AuthFailure::Rejected`, and an `argon2` hash is computed either way (`gungnir-security/src/session.rs:404-437`).

9.4 Ownership at boundaries

The Tokio runtime is host-owned and lent as a `Handle`. `LiveTrackingService::new(runtime:&tokio::runtime::Handle)` (`gungnir-tracking-service/src/lib.rs:442`); the desktop builds it in `gungnir-app/src/state.rs:1270-1274`, the node in `gungnir-node/src/main.rs:80-84`. `gungnir-fusion-async/src/lib.rs:5-9` calls itself "the *only* crate that uses the `tokio` runtime"; that was true of pipeline work and is now stale of the workspace: eight members declare `tokio`, and each binary builds one runtime whose `Handle` it lends -- `gungnir-remote`'s `link` task runs on the host's, not on one of its own. It is a documentation defect, not a dependency-graph one.

The `wgpu` device is shared through `Arc`, and the reason is written down. `gungnir-render/src/lib.rs:24-44`: `wgpu::Device/Queue` are neither `Copy` nor `Clone` in the pinned `wgpu 22`, and a caller holding both `GpuContext` and a borrowing `GpuFusionEngine<'a>` in one struct (`AppState`) "would be self-referential, which safe Rust cannot express without pinning or a crate this workspace does not carry". `Arc` changes "nothing about there being exactly one `wgpu::Device` in the process". The shipped `GpuFusionEngine` has no lifetime parameter; the borrowing design is the one rejected. `gungnir-data-fusion` takes the `Arcs` (`src/lib.rs:100-102`, `212-217`) and carries no dependency edge to `gungnir-render`, which its doc comments name only to say where the one device comes from.

`PolicyChain<'a>` is the one signed lifetime-parameter change. `gungnir-policy/src/lib.rs:145`; an owning chain infers `PolicyChain<'static>`, "which is what every prior use inferred". Because `gungnir-policy` is human-owned, a type-level change still went to the owner (signed 2026-09-05, GAP-038, `docs/agent-coding-workflow.md`).

The journal holds a `BufWriter<File>` behind a `Mutex<Option<OpenSession>>` with a dirty flag and `sync_data` (`gungnir-store/src/lib.rs:89-133`); the lock exists because `read_session` takes `&self` and must see buffered envelopes. `FileEventJournal` is deliberately not `Clone`: "two clones would hold two buffered writers onto one file and interleave partial lines". Its Debug reports whether it is sealing, never the key.

The borrow checker as an oracle. `gungnir-decision`'s `what_if` takes `&self`, so "live state provably unchanged" is compiler-enforced and also tested, since that holds only while the fields stay shared borrows (verification table, decision row).

9.5 Concurrency

Channels over shared mutexes is the standard (`docs/agent-coding-standards.md §2.2`): reaching for `Arc<Mutex<..>>` between async tasks "is the trigger to stop and get human review". The `async` crate holds to it: "cross-task state is two channels and nothing else. There is no `Arc<Mutex<..>>` and there are no atomics" (`loom_model.rs:16-20`). The pipeline lives on one task's stack and is mutated only between awaits, so a dropped future "loses at most one in-flight detection and never leaves the buffer half-updated" (`lib.rs:226-230`); every public `async fn` in the crate carries a cancellation-safety note.

The `async/sync` boundary is `crossbeam-channel`, polled with `try_recv` plus a `yield` rather than a blocking `recv` (`lib.rs:232-234`; consumer `gungnir-tracking-service/src/lib.rs:457-461, 683-712`). `tokio::sync` appears in `src/` in exactly two places: `gungnir-api/src/transport.rs` (broadcast for the event stream, oneshot for a sensor-task reply) and `gungnir-remote/src/link.rs:47` (`mpsc` and `watch` inside the link task) -- both channels. The `Arc<Mutex<..>>` in that same file (`link.rs:200`) is the link's projection, read by the synchronous services and written by the `async` link task -- a case §2.2's channels-by-default rule does not name for a service-layer crate either way. The outbound snapshot channel is unbounded, with the cost stated rather than hidden (`lib.rs:145-153`); the remote outbox is bounded at 100,000, drop-oldest and counted (`gungnir-remote/src/lib.rs:218, 361-368`), a bound no test fills.

Loom model checking is built and gated, and the models are unsigned. `gungnir-fusion-async/src/sync.rs` re-exports `crossbeam` normally and, under `all(test, loom)`, substitutes a hand-written MPSC over `loom::sync` because `loom::sync::mpsc` "does not model disconnection" and `crossbeam` is "opaque to loom". `src/loom_model.rs` drives the real `ingest_with` loop with three positive models and one negative model (`unbundled_publication_is_caught, #[should_panic]`) that publishes over two channels the way the crate did before GAP-096 and requires `loom` to find the skew. `loom.yml` runs at `LOOM_MAX_PREEMPTIONS=3` under `RUSTFLAGS=- -cfg loom` and refuses zero explored interleavings (4 model checks, 297 interleavings, GAP-061); the 1/9/36/99 counts at bounds 0-3 are hand-measured figures in the module docs. Stated limits: the inbound producer-versus-drain race is not modeled, and `crossbeam`'s own implementation is out of reach. The `PipelineSnapshot` bundling was signed 2026-09-09 citing these models; the model checks themselves are recorded "written and gated, not signed" in the register. The workflow header records that this gate "fired 22 times, went green 22 times, and model-checked zero interleavings" before GAP-061.

Miri gate for unsafe, currently with nothing to check. Zero unsafe in 399 first-party files is a measured fact (the only word-match is a doc comment), not a compiler guarantee: no `forbid(unsafe_code)` exists, only `unsafe_op_in_unsafe_fn = "deny"`. `.github/workflows/miri.yml` scans every PR diff for an added unsafe token and, on a hit, runs `cargo miri test` over 12 named tracking-core and service crates (`miri.yml:53-58`), not the whole workspace. The `miri` job proper has never had a trigger. Two real concurrency defects were found in owner review: the SAPIENT task sink treating `WouldBlock` as a dead connection (`gungnir-ingest/src/adapters/sapient.rs:282-300`), and the OS-keystore first-run race, narrowed by read-back with a stated residual window (`gungnir-security/src/os_keystore.rs:221-233`).

9.6 Lint and toolchain governance

`Cargo.toml:89-107`: `[workspace.lints.clippy]` sets `all = deny` and `pedantic = warn` at priority -1, with five named allows (`many_single_char_names` for filter notation, `module_name_repetitions`, `must_use_candidate`, `missing_errors_doc`, `missing_panics_doc`); `[workspace.lints.rust]` sets `unsafe_op_in_unsafe_fn = "deny"` and `unexpected_cfgs = { level = "warn", check-cfg = ['cfg(loom)'] }` so the `loom` cfg is declared rather than warned on. Every workspace member opts in with `[lints] workspace = true`; the 50 members build under it, the excluded `gungnir-fuzz` (3 files, 156 lines) inherits no lint table. Seven module-level `#![allow]` lines exist in the tree -- none at a crate root, all in module or test files -- each with a one-line justification per §3.5. `rust-toolchain.toml` pins `channel = "1.98"`; every workflow that pins a toolchain names it, and so does the node container (`rust:1.98-slim-bookworm`); `miri.yml` and `fuzz-nightly.yml` run nightly, which `miri` and `cargo-fuzz` require. `ci.yml:72-90` runs `cargo fmt --check`, `cargo check --workspace --all-targets --locked` under `-D warnings`, `cargo clippy --workspace --all-targets`, `nextest`, and `doctests`; `--locked` stops a pull request going green "with a lockfile nobody reviewed".

9.7 Dependency governance

One version set: 53 `[workspace.dependencies]` entries, every crate "should pull versions from here" (`Cargo.toml:109-113`). Every addition is recorded in `docs/agent-coding-standards.md §2.9` (dated subsections per decision: D-18 `transport`, D-20 `crypto`, D-22, D-39 `keyring`, D-40 `ort`, D-41 `proj`, D-42 `KMS`), and

architecture_compliance.rs:396-400 checks that every entry is named there. Direction is tested: gungnir-app/tests/dependency_graph.rs reads every member manifest and refuses a cycle, an upward edge, and an unplaced crate against ARCHITECTURE.md §7.1 (169 edges, acyclic).

deny.toml is the gate (ci.yml:28-36 runs licenses bans sources on every PR; release.yml adds advisories): [graph] targets restricted to the two release targets; [advisories] yanked = "deny" with dated, path-naming ignores; [licenses] an allow-list of 12 licenses with AGPL deliberately absent, so each of the 50 members is admitted by name through [[licenses.exceptions]] and "a new member crate fails this check until it is added"; a font exception for epaint_default_fonts; [bans] wildcards = "deny", which is why every intra-workspace path dependency carries version = "0.1.0"; [sources] unknown registries and git refused. about.toml and deploy/third-party-notices.hbs generate THIRD-PARTY-NOTICES.md in release.yml, which triggers only on a v* tag and has never run, so the file has never been produced (GAP-061, In progress). The manifest comments carry the engineering: rustls-only TLS, vtkio without xml after RUSTSEC-2026-0194/0195, proj without network, ort with load-dynamic, = pins on the two ADS-B oracle crates.

Two manifest comments are stale documentation defects: Cargo.toml:355-358 says rcgen is "never a normal dependency of any member" while gungnir-remote/Cargo.toml:27-32 depends on it; and the comment that p256 is "deliberately unused" predates gungnir-security/src/asymmetric.rs.

9.8 The testing pyramid

Layer	Count	Where	Status
Unit tests	1,326 #[test] under src/, of 1,832 workspace-wide, and a further 54 #[tokio::test]	mod tests in src/	run on every push
Property tests	2 files	gungnir-coord/tests/invariants.rs; gungnir-testkit/src/lib.rs	run on every push; narrow coverage, stated below
Differential oracles	17 tests/*_diff.rs in 9 crates; 14 generators, 20 fixtures	testdata/oracles/	built and gated , oracle-diff.yml
Integration	124 files, 52 in gungnir-app/tests/	real loopback sockets, real TLS handshakes	run on every push
Architecture-as-tests	3 files, 13 tests	architecture_compliance.rs, dependency_graph.rs, no_reachable_todo.rs	run on every push
Model checking	4 loom models	loom_model.rs	built and gated ; models unsigned
Fuzzing	3 targets	gungnir-fuzz/fuzz_targets/	nightly; found GAP-103 on its first successful run
Benches	5 criterion targets	*/benches/	advisory by owner decision

The honest sentence: proptest is used in two files, although the standard calls property tests "the default for anything with a mathematical invariant" (docs/agent-coding-standards.md §2.5, line 200). The main verification instrument is the differential-oracle fixture: each stamps its oracle's name and version -- filterpy 1.4.5, Stone Soup 1.9.1, scipy 1.18.1, pymap3d 3.2.0, py-motmetrics 1.4.0, and, where no library oracle exists, a project-authored closed form (MHT hand-derived; CPHD and LMB in-house derivations self-checked several ways) -- and oracle-diff.yml discovers _diff targets from cargo metadata and fails if targets discovered differ from targets reporting or if zero tests ran. The CI figure is 1,863 passed, 3 skipped, 112.7 s under nextest (run 34361396023 on ce609437); the run on the tip is red on one gungnir-node provisioning-harness race.

9.9 Performance discipline

No benchmark asserts on absolute numbers (benches/README.md); assertions live in gungnir-app/tests/frame_budgets.rs, whose doc comment table states per row whether the budget can honestly be gated. Journal append of 50 envelopes is **built and gated** as a median-of-9 under 1 ms, asserted in the release profile only (ci.yml:96-99: "a release-only budget that no job runs in release is a gate that cannot fail"); 157 us and 98.9 us median are developer-machine measurements, not the gate. Per-frame update() -- ingest, tracker hand-off and poll, planning and journal on the frame thread; the filter math runs on the pipeline task -- measured 22.3 us p99 and 375.3 us worst over 3,000 headless frames of a synthetic Scenario 3 replay against a 4 ms budget, printed and not asserted pending the owner's row walk (GAP-067, D-16).

The measurement discipline is in docs/performance-budgets.md: the GAP-085 journal fix was a real 36x (5.7 ms to 157 us) by holding the file open; the debug slowness was traced to serde_json at opt-level = 0, not I/O; pinning across an 8 P-core / 12 E-core machine split the figures "on the P/E boundary exactly", so any number is "an order of magnitude, not a regression baseline, unless it was measured controlling for which core class ran it". bench-regression.yml is advisory, on main and by dispatch only, after three false regressions on a different runner class. No [profile] tuning exists in the root

manifest or in any member manifest, so no LTO or codegen-units work is claimed.

The honest answer: "zero-copy", "lock-free", and "miri-clean" are not claims this repository supports. The `async` crate is two channels, the journal and the remote projection use a `Mutex`, and `miri` has had nothing to run.

Chapter 10 -- Twelve Rust design questions this repository answers from its own code

How to use this chapter

Each question below is one a technical reviewer of this codebase is likely to ask. Each answer is grounded in one file at `main` `ee12f7e` (2026-09-09), carries its status where status matters, and ends with the file to open. Every claim was checked against the repository at that commit, and the corrections that check produced are folded in.

Note: name the limit before a reviewer finds it. "Measured, not forbidden" and "gated, unsigned" are stronger answers than a clean claim that does not survive a `grep`.

The twelve questions

1. Where does the Tokio runtime live, and how does pure computation stay out of `async`?

The host binary owns the runtime and lends a `Handle`, and `gungnir-fusion-async` is the only crate the tracking pipeline runs in (`gungnir-fusion-async/src/lib.rs:5-9`, `docs/agent-coding-standards.md` section 2.2). `gungnir-api` and `gungnir-remote` take `tokio` as a normal dependency and run their transport work on that same host runtime; `gungnir-data-fusion` takes it only as a dev-dependency, to block on `wgpu`'s `async` device request in the `gpu-tests` differential test, and does no transport or crypto work at all.

One known documentation defect to name rather than repeat: section 2.2 says "No other crate creates a runtime", and `gungnir-security` does. Its cloud-KMS custody profile builds a `current-thread tokio::runtime::Runtime` on each client's own worker thread, because the AWS and Azure SDK clients are `async` and `KeyProvider` is not (`gungnir-security/src/managed_service.rs:385`, called at `:434` and `:574`). That work therefore does not run on the host runtime. The exception is recorded in the crate's own manifest under D-42 but never written back into the standard; built, unsigned.

`LiveTrackingService::new(runtime: &Handle)` spawns `ingest_with` from the host runtime (`gungnir-tracking-service/src/lib.rs:442-461`). The pipeline lives on one task's stack and is mutated only between awaits, so a dropped future loses at most one in-flight detection (`lib.rs:226-230`). The sync side polls crossbeam with `try_recv`, never a blocking `recv`. Built and gated on a synthetic three-sensor, two-target timeline (`gungnir-fusion-async/tests/oos_convergence.rs`); no real sensor and no deployment have run.

Open: `gungnir-fusion-async/src/lib.rs`.

2. When do you use a trait object and when a type parameter?

Generics where the dimension or model is part of correctness: `KalmanFilter<Model, const N, const M>` evaluates `F(dt)` and `Q(dt)` from the model type at the step actually taken, so a cached matrix cannot go stale (`gungnir-filters/src/kalman.rs:39-64`). Trait objects where the binary wires a capability the crate must not know: `Option<Box<dyn JournalSealer>>` (`gungnir-store/src/lib.rs:123`), a trait declared by the consumer so no edge to `gungnir-security` exists. Built and gated.

Open: `gungnir-store/src/sealing.rs`.

3. How do you design errors a CI log reader can act on?

Crate-local `thiserror` enums throughout. An unimplemented capability is a named variant carrying what and `waiting_on` (`gungnir-filters/src/lib.rs:112-117`, `gungnir-rfs/src/lib.rs:740`), never `todo!()`; a source scan fails on any `todo!()` (`gungnir-app/tests/no_reachable_todo.rs`). The rule is refuse rather than degrade: `TooManyHypotheses` instead of a truncated JPDA enumeration (`gungnir-association/src/assignment.rs:69-90`), and `FusionError::NotImplemented` kept distinct from `Divergence` because they are opposite claims about the same call. Built and gated.

Open: `gungnir-association/src/assignment.rs`.

4. Show me a `Send` bound that is load-bearing rather than decorative.

`Imm` stores `Vec<Box<dyn ModeFilter<N, M> + Send>>` (`gungnir-filters/src/imm.rs:163-168`). The comment says why: `gungnir-fusion-async` holds a `TrackFilter` across an `await` in a spawned task, and `tokio::spawn` requires the

whole future, so everything inside it, to be Send. The traits that cross threads carry Send + Sync explicitly: KeyProvider, JournalSealer, ApiHandler, TimeAuthority. Built and gated; DN-28 signed 2026-09-07.

Open: `gungnir-filters/src/imm.rs`.

5. Zero unsafe -- how, and what does that not prove?

A word-grep over 399 first-party `.rs` files finds one hit, in a doc comment; no `forbid(unsafe_code)` is set. The one place a lifetime would have forced pinning, a `GpuFusionEngine<'a>` borrowing the device inside `AppState`, was replaced by `Arc<wgpu::Device>` with the reasoning recorded (`gungnir-render/src/lib.rs:24-44`). `miri.yml` fires on any PR diff adding unsafe but runs cargo `miri` test on 12 named crates only, and says nothing about dependencies such as `wgpu`. Measured and scanned, not lint-forbidden.

Open: `gungnir-render/src/lib.rs`.

6. How do you model-check real async code, and what does loom not see?

`src/sync.rs` re-exports crossbeam normally and, under `cfg(loom)`, a hand-written MPSC over `loom::sync`, because crossbeam is opaque to loom and `loom::sync::mpsc` does not model disconnection. Three positive models drive the real `ingest_with` and `drain-to-latest` poll over that model channel, so what is checked is the outbound snapshot channel's orderings against crossbeam's documented contract -- not crossbeam itself, and not the inbound producer-versus-drain race, which every model closes before it spawns `ingest`. One negative model, `#[should_panic]`, republishes over the unbundled two-channel shape GAP-096 refused and requires loom to find the skew. CI runs preemption bound 3 only; the 1/9/36/99 interleaving counts are hand-measured in the module docs (`gungnir-fusion-async/src/loom_model.rs:63-64`). The gate went green 22 times checking nothing before this (`loom.yml` header). Built and gated; the model checks themselves are unsigned.

Open: `gungnir-fusion-async/src/loom_model.rs`.

Note: the strongest concurrency answer is the negative model. A test that must fail if the rejected design is reintroduced is evidence the chosen one matters.

7. How do the 50 workspace members stay one codebase?

One `[workspace.lints]` table that each of the 50 workspace members opts into -- the fifty-first crate, the workspace-excluded `gungnir-fuzz`, inherits no lint set: `clippy::all = deny`, `pedantic = warn`, five pedantic allows for lints that do not fit numerical code, one of them -- the filter-notation one -- with a stated reason (`Cargo.toml:89-107`). One version set in `[workspace.dependencies]`, and a test fails if any entry is not named in the standards (`architecture_compliance.rs:396-400`). `dependency_graph.rs` reads every manifest and refuses a cycle, an upward edge, an unplaced crate, and any edge `ARCHITECTURE.md` section 7.1 does not draw, checked in both directions. Rust 1.98 pinned. Built and gated.

Open: `gungnir-app/tests/dependency_graph.rs`.

8. How do you measure before you optimize?

The journal append fell from 5.7 ms to a recorded 157 us per 50 envelopes by holding a `BufWriter<File>` open behind `Mutex<Option<OpenSession>>` with `sync_data` (`gungnir-store/src/lib.rs:89-118`, `sync_data` at `:203`, GAP-085). GAP-092 then measured why debug was slow: `serde_json::to_string` at about 500 us, not I/O, split exactly on the P-core/E-core boundary when pinned per CPU. Bench regression is advisory by owner decision because hosted runners differ, and no `[profile]` tuning exists. Journal append: a median of nine under 1 ms, built and gated in the release profile only. Per-frame `update()`: built, ungated -- 22.3 us p99 (375.3 us worst) over 3,000 headless frames of a synthetic replay on the development machine, measured 2026-09-07 against a 4 ms budget, and it bounds the frame thread's hand-off and poll by design, since the filter math runs on the pipeline task; promotion of the row is left to the owner's walk (`docs/performance-budgets.md`, GAP-067).

Open: `docs/performance-budgets.md`.

9. How do you version a wire format with serde?

`gungnir_model::SCHEMA_VERSION` is 3; each bump is recorded in its doc comment with what broke, and the version-2 entry also records why no deprecated mirror was kept (`gungnir-model/src/lib.rs:136-152`). `POST /v2/detections` carries `#[serde(default)] schema_version`, defaulting to zero rather than current so an old client cannot silently claim to be current, and is refused with 409 naming both versions (`gungnir-api/src/v2/mod.rs:194-198`, `transport.rs:1234`). That is the only version-gated route. 232 `#[serde(default)]` and zero `deny_unknown_fields` tree-wide. Built and gated.

Open: `gungnir-api/src/v2/mod.rs`.

10. Where did a newtype earn its keep, and where did one mislead?

`TrackId`, `ResourceId`, `SensorId`, `PlanId`, `SessionId` and thirteen more shared types have exactly one owning crate, held by `every_shared_type_has_exactly_one_definition` (`architecture_compliance.rs:131-151`); the identifiers among

them are tuple structs, apart from `AlgorithmBaselineId`, which is a two-field struct of profile and name because a baseline is named by both (`gungnir-model/src/profiles.rs:63-66`). `OperatorId` (`gungnir-security/src/lib.rs:87`) is a newtype of the same shape but is not on that test's list. The instructive case is the allocator: assignment once held (`ResourceId`, `TrackId`) filled with row and column numbers, "type-correct and wrong", and nothing could catch it because every value was valid. It is now `Vec<(usize, usize)>` with the story in the doc comment (`gungnir-allocation/src/lib.rs:59-69`) and a test in the caller that pairs tracks 70/71 against resources 40/41 and asserts the pairing names the resource in that row, not the one whose identifier equals the row number (`gungnir-intercept-service/src/lib.rs:462`). Built and gated.

Open: `gungnir-allocation/src/lib.rs`.

11. Describe a refactor you made under test.

GAP-096 widened the single outbound channel's message from `Vec<TimedTrack>` to one `PipelineSnapshot` bundling tracks, retained bearings, stats and the dense-group estimate, read with no `.await` between them (`gungnir-fusion-async/src/lib.rs:139-171`), rather than adding a second and third channel; that rejected multi-channel shape survives as the negative loom model. Earlier: twenty-two `todo!()` bodies became named errors under one scan -- the grep found eighteen, and four took a message and were invisible to every count (`ARCHITECTURE.md` item 78, GAP-082); `PlanView.solutions` became `PlanView::kind` across ten call sites in six crates with `SCHEMA_VERSION` 1 to 2 (DN-05). Built and gated; `PipelineSnapshot` signed 2026-09-09.

Open: `gungnir-fusion-async/src/lib.rs`.

12. How do you govern dependencies for a codebase whose licensing and provenance have to survive review?

`deny.toml` runs `licenses bans sources` on every PR: wildcards denied, so every path dependency carries `version = "0.1.0"`; unknown registries and git sources denied; AGPL deliberately off the allow-list, each of the 50 members admitted by name; six dated advisory ignores. Manifest comments carry the engineering: `rustls-pemfile` dropped on RUSTSEC-2025-0134, `vtkio` without XML, `proj` without network, = pins on the two ADS-B oracle crates. One known documentation defect to name: `Cargo.toml:357` says `rcgen` is "never a normal dependency of any member" while `gungnir-remote/Cargo.toml:32` takes it at runtime -- it moved there under GAP-060 on 2026-09-06 (`docs/agent-coding-standards.md` section 2.9, D-29) and the workspace comment was never updated. No register entry records the contradiction. Built and gated.

Open: `deny.toml`.

Note: for each answer, the second sentence is what it does not cover. Questions 6, 8 and 9 already carry one, written in the file the answer points to; question 5's limit is in `.github/workflows/miri.yml` rather than in the file it opens.

Part IV, Chapter 11 -- The decision ledger, D-01 to D-43, mapped to mission

How to read the ledger

The ledger is `docs/mission/gap-analysis/decisions-needed.md`: a "Decisions needed" row per decision (what was being decided, who decides, which gaps it unblocks), an "Outcomes" row (what was decided and why), and a "Consequences for the register" section. Ids run D-01 to D-43 with no gaps. Two numbering facts a reviewer will notice: the outcome table is not in numeric order, and D-35 to D-38 were filed as D-33 to D-36 on 2026-09-06 and renumbered on 2026-09-07 after a collision with the `Cursor-on-Target` and license decisions (`ARCHITECTURE.md` section 10, item 89).

Seventeen decisions were taken on 2026-09-04, the day the mission analysis was written; five on 2026-09-05; fifteen on 2026-09-06, eight of them (D-24 to D-27 and D-29 to D-32) in two walks run "each with a recommended default" and the rest -- D-23, D-28, D-33, and the four theme decisions D-35 to D-38 from the review filed as `ARCHITECTURE.md` section 10 item 89 -- alongside them; the remaining six between 2026-09-07 and 2026-09-09. Several outcome rows were written at decision time and never updated when the code landed. Where a row and the code disagree, this chapter reports the code and names the stale row.

Status words: **built and gated** (code plus a CI test against a named criterion), **built, unsigned** (gated, in a human-owned crate, owner's signature outstanding), **built, ungated, not built**, and **decision only** for a policy or target with nothing to build. Where a human-owned crate carries the owner's recorded signature the row says **built and gated, signed**. The mission column cites steps of `docs/mission/mission-threads.md` (MT-01 to MT-10).

Topology, scope, and deployment

Id	Decided	Rejected	Why	Mission
D-01 Scope lock (09-04)	Everything in one release: all scaffolded crates, all 56 capabilities through increment I4. Decision only.	A narrower first release	The I4 column became the release content, so no gap can be quietly dropped; D-33 later added Cursor-on-Target "as an addition to the scope lock, taken deliberately and recorded".	All ten threads
D-03 Reconciliation and arbitration (09-04)	Journals merge in mission-time order, duplicates dropped, conflicts reported; higher role wins, earlier decision wins a tie. Built and gated (gungnir-app/tests/failover_e2e.rs).	Replacing gungnir-resilience merge semantics or RoleRankArbiter first	Lock the rule, then build failover on it. An expiry-first rule was signed 2026-09-05 (DN-10 amendment 1): if one site's window closed and another decided, "there is nothing to arbitrate".	MT-10 steps 4 and 5
D-04 Budgets and fsync (09-04)	Node journal fsyncs every envelope; desktops buffer, fsync on session save and every 5 s. Built and gated (gungnir-store/tests/durability_policy.rs).	One policy for both binaries	The node "is the system of record for every connected desktop"; a desktop losing its last seconds "costs a local session, not the mission picture".	MT-01 step 9; MT-10 step 1
D-07 Fires in scope (09-04)	Fires in the first release; deconfliction joins the policy model. Checks built and gated (gungnir-policy/src/fires.rs); the fires planner not built .	A later domain extension	The land picture reuses the track, identity and policy machinery; the FPV threat makes force protection a counter-UAS problem.	MT-06 steps 5 to 7
D-18 API transport stack (09-05)	axum with ws, tokio-tungstenite, reqwest, rustls, tokio-rustls, tower-http. Built and gated (gungnir-api/tests/mutual_tls.rs, gungnir-remote/tests/tls_link.rs).	native-tls; gRPC in the same decision	One TLS implementation in the SBOM, no system OpenSSL; one tungstenite, one tower-http. rustls-pemfile left 2026-09-07 on RUSTSEC-2025-0134.	Every connected step; GAP-041 moved from I4 to I2 because without it "MT-10 cannot run end to end"
D-21 gRPC second transport (09-05)	tonic 0.14 and prost 0.14 pinned, used by no member. Not built.	A second contract	One hyper, tower and http shared with axum; both transports must be generated from gungnir-model "so that two transports do not become two contracts".	Peers that cannot speak v2 (MT-02 step 1, MT-05 step 4); nothing serves them today
D-23 Heartbeat budget (09-06)	HEARTBEAT_INTERVAL 2 s; timeout derived in code as three misses plus a 1 s margin, so 7 s. Built and gated (gungnir-api/src/transport.rs).	Widening the budget to 35 s; declaring the link dead after one miss	See below.	MT-10 step 1 "fallback within seconds"; MT-07 "no decision on hidden staleness"

D-23 is the smallest decision in the ledger and the one that best shows the workspace's rules applied to itself. The connectivity budget said fallback within 2 s of the last heartbeat; the node beat every 10 s with an independent 35 s timeout, so a node that went silent with its socket open was unnoticed for 35 s. GAP-056's connectivity test found the conflict. Widening the budget to match the code was refused as "the widen-the-criterion move the workspace forbids for verification rows, and the spirit applies". Declaring the link dead after a single miss was refused because it "would make every 2 s stall on an intermittent link a full reconnect". The resolution named two things the budget had conflated, visibility (the strip goes stale within 2 s) and declaration (the link is gone within four beats), and derived the timeout from the interval in code "because two independently edited constants is precisely how the conflict was created and how it would recur".

Note: D-23 is a one-minute answer to "tell me about a time a test disagreed with a requirement". Neither the requirement nor the code was bent to the other, and the fix removed the second constant.

Security, identity, and custody

Id	Decided	Rejected	Why	Mission
D-02 Credential mechanism (09-04)	Mutual TLS for machines; short-lived signed tokens for operator sessions; local accounts as the disconnected fallback. mTLS and tokens built and gated .	Tokens for everything; mTLS for operators	Until built "a node serves loopback only and refuses every write path, because it can protect neither the channel nor the caller's identity" (ARCHITECTURE.md section 8.5).	CAP-6.1 on all ten threads; MT-10 step 3
D-15 Delegation (09-04)	Supervisor may pre-delegate point-layer engagements of confirmed-hostile small UAS; area layer and missiles never. Pre-delegation built and gated (gungnir-policy/src/authority.rs::is_pre_delegated); the disconnected-expiry half not built : no code implements it.	A general delegation power	"A pre-delegated plan still needs a recorded decision; what it skips is escalation, not the person." A pre-delegated rule must name a layer and a class (gungnir-config/src/lib.rs).	MT-02 steps 3 and 6 (missile timeline under 30 s); MT-03 step 5; MT-10 step 3
D-20 Authen tication crates (09-05)	argon2, hmac with sha2, subtle. Built and gated (gungnir-security/src/token.rs).	Public-key token signing	The node issues and verifies its own tokens, so a public-key scheme "would put a private key in the process" for no verifier's benefit. All RustCrypto, one audit surface.	As D-02
D-22 Cipher and certificates (09-05)	AES-256-GCM for seal and unseal; p256 ECDSA for baseline signing; rcgen dev-only at the time. Built and gated (gungnir-node/tests/encryption_at_rest.rs).	ChaCha20-Poly1305; Ed25519; hmac for signing; a checked-in test key	AES-GCM is hardware-accelerated on both targets and FIPS-approved. P-256 "because it is FIPS-approved, which is what the accreditor these documents keep in view will ask; Ed25519 is the better curve on most other grounds and loses on that one".	Journal at rest; signed baselines (MT-09 steps 4 and 6)
D-27 Escrow key holder (09-06)	A named security-officer role per deployment; data keys wrapped to the officer's public key (P-256 ECDH, HKDF, AES-GCM); recovery offline and audited. Built and gated, signed 2026-09-06 (gungnir-security/src/asymmetric.rs: wrap and recover, an impostor refused, a non-journal key refused); GAP-084 stays in progress for D-42's ManagedService half and a real cloud round trip.	Escrow to an operating role	"An accreditor asks about custody before ciphers." The private half is never on a node.	Record survivability, MT-01 step 9 and MT-10
D-29 X.509 at runtime (09-06)	rcgen at runtime so the provider signs and the private half never leaves custody. Built and gated, signed (section 10 items 88, 103, 104, 111).	Shipping a certificate generator, D-22's worry: "a way to mint an identity"	Same-day amendment: the code lives in gungnir-remote/src/identity.rs, so rcgen ships in both binaries. The ledger row and the manifest comment ("never a normal dependency of any member") are stale against gungnir-remote/Cargo.toml: a documentation defect.	As D-02
D-30 Security Officer role (09-06)	A Role variant that "operates nothing and may only recover"; one permitted action. Built and gated, signed .	A design note naming a role the enum lacked	Adding a variant touches every exhaustive match on Role; do it once. Nine roles now.	Escrow recovery

Id	Decided	Rejected	Why	Mission
D-39 OS-keystore crate (09-08)	keyring 4.2.0, v1 feature. Built and gated, signed 2026-09-08 ; CI gates the keyring-core mock store and the no-keystore fallback only.	The passphrase-sealed file as the only custody	Duplicate-linkage check run, not assumed: two duplicates on Linux, none on Windows or macOS. Real Credential Manager round trips were manual on the owner's machine; Keychain and Secret Service unexercised. A first-run secret-generation race was found in review and narrowed by re-reading the store; the module says no backend API can close the residual window.	MT-10 step 3 custody without a typed passphrase
D-42 Cloud KMS crates (09-08)	aws-sdk-kms plus aws-config; azure_security_keyvault_keys 1.0.1 with azure_identity. Built, unsigned (gungnir-security/src/managed_service.rs); no cloud round trip made.	azure_security_keyvault (older, unofficial); GCP KMS; Vault	"Choosing it from memory is exactly the error this row exists to prevent." Two self-corrections recorded (the Azure version; aws-config added because the client carries no credential resolution). Open mismatch: DN-22 section 14 says "once per rotation"; rotate() makes no service call.	Cloud-profile record survivability (ARCHITECTURE.md section 8.2)

D-20 and D-22 together answer "why a MAC for sessions but a signature for baselines". A session token is issued and verified by the same node, so a symmetric MAC suffices and keeps private keys out of the process; a baseline must be verifiable by a peer with no shared secret, so it needs a signature. The FIPS argument for P-256 is what the posting's DoD customers will expect, and the ledger states the trade honestly: Ed25519 is the better curve and loses on accreditation alone.

Data, interop, and standards

Id	Decided	Rejected	Why	Mission
D-06 Releasability (09-04)	A marking on views, reports and the contract; per-caller enforcement at the API. Built and gated (AgreementSet::may_send, gungnir-api/src/transport.rs).	Deferring coalition constraints	MT-05 and MT-08 "produce pictures and products that a coalition partner should see some of".	MT-08 step 6; MT-05 step 4
D-08 External agreements (09-04)	Each external party is a configurable endpoint with a message type; synthetic peers for now. EndpointConfig built and gated ; GAP-004 and GAP-065 in progress.	Per-party adapters before agreements exist	"An adapter is configuration rather than code."	Step 7 of MT-01, MT-02, MT-04, MT-06
D-09 Cooperative identity and corpora (09-04)	AIS and ADS-B from open protocols; ASTERIX from public Eurocontrol specs; IFF deferred (controlled key material). AIS, ADS-B, ASTERIX 048/034/205/129 decode built and gated (the Category 205 adapter arm is signed; 048 and 034 are gated with no owner signature recorded; the Category 129 ingest adapter is built, unsigned , GAP-101); STANAG 4676 not built (NotImplemented); IFF not built .	Buying the paywalled STANAG spec	On 2026-09-08 "the owner declined to purchase the paywalled specification".	MT-03 step 2; MT-04 step 3; MT-05 steps 1 and 2; MT-01 step 1

Id	Decided	Rejected	Why	Mission
D-11 uuid v7 (09-04)	UUID v7 for GlobalEntityId. Built and gated.	The u128 newtype; v4	The first 48 bits are a timestamp, so identities sort in minting order, "which is what an after-action review reads them in". Only <code>gungnir-identity</code> takes v7 "because minting is its job".	MT-06 step 3; MT-08 step 5
D-13 Anomaly rules' home (09-04)	Pure functions in <code>gungnir-analytics</code> over snapshots; no new edge. Built and gated on the desktop; the node runs no detector, so the row's "the app and node tick call them" is half true.	A new crate	The detectors take a primitive <code>TrackSnapshot</code> because <code>TrackView</code> would need the forbidden edge to <code>gungnir-model</code> .	MT-05 step 2; MT-07 step 1
D-14 Assistant egress (09-04)	Cloud profile may send the live picture to a cloud model; on-prem derived text only; disconnected local only. Not built (GAP-044 Planned).	One policy for all profiles	The profile boundary fixed before any assistant code exists.	CAP-4.7 supporting on MT-01, MT-07, MT-08, MT-09
D-24 and D-32 AIS and ADS-B (09-06)	AIS pinned to ITU-R M.1371-6 after a table-by-table comparison with -5; ADS-B specification of record ICAO Doc 9871 (not held); <code>gpsd</code> captures as fixtures. Built and gated.	A codec from memory; the owner recording his own 1090ES capture	The owner took the open-source-consensus route instead; the row still says the fixture is "the owner's self-recorded one" (stale against <code>testdata/adsb/SOURCE.md</code>).	As D-09
D-25 GeoTIFF (09-06)	<code>tiff</code> (pure Rust); own the geo tags per OGC GeoTIFF 1.1. Built and gated (<code>gungnir-data/tests/dem.rs</code>).	<code>oxigdal-3d</code> , never pinned	Fewer native dependencies.	MT-09 step 2; MT-07 step 2
D-31 YAML (09-06)	<code>yaml_serde</code> for <code>gungnir-scenario</code> only. Built and gated (byte parity with the Python generator).	<code>serde_yaml</code> (deprecated)	The Python generator stays the reference.	TT-01 to TT-10, every thread
D-33 Cursor-on-Target (09-06, extended 09-07 and 09-08)	In scope. Schema pinned to the 2003 MITRE schema; type tree to the 2005 Developer's Guide, acting only on <code>^a-f-</code> ; framing pinned at <code>atak-civ 5.5.1.8</code> after the earlier refusal was withdrawn as "false". Not built ; corpus unrecorded.	MIL-STD-2525 as the pin ("the ancestor rather than the artifact"); codegen from the <code>GPLv3.proto</code>	<code>ce</code> and <code>le</code> carry no stated confidence level, so the mapping must declare its sigma multiple.	MT-01 step 7; MT-08 step 6; the MT-06 friendly set (GAP-090)
D-41 Real-world CRS (09-08)	<code>proj 0.31</code> binding <code>libproj 9.6.x</code> ; full projection support. Built and gated behind the <code>crs</code> feature, Linux CI only; the Windows release target is built without the feature, so that build refuses CRS files by name.	A WGS84-only first step	"Most real DEM and LIDAR data ships in a real-world CRS." <code>proj-sys</code> cannot build on the author's machine; the row still says the crate is "not yet recorded in <code>agentic-coding-standards.md</code> section 2.9 or built".	MT-09 step 2

Algorithms, oracles, and gates

Id	Decided	Rejected	Why	Mission
D-16 Open targets (09-04)	Walked measure by measure: MOE-08 raised to 0.9, MOE-10 tightened to 30 s, MOP-07 to 500 ms, MOP-04 under 1 per hour; six verification rows set (snapshot p99 1 ms, ICP 0.05 m and 0.5 mrad, GPU within 1e-3 with inlier ratio within 0.01, egui pass p99 8 ms, coverage 1 percent and 0.1 degree, interop round-trip exact with best-effort decoding). Decision only.	Leaving measures proposed	Targets cannot become gates until the owner sets them. Defect to name: docs/architecture.md records the GAP-067 walk promoting section-2 rows to Specified on 2026-09-07, while the verification table's section-2 preamble still says "not gates yet" and the register keeps GAP-067 Open.	MOE-08 to MT-06; MOE-10 to MT-07; MOP-07 to MT-02; MOP-04 to MT-05
D-26 Interceptor speed (09-06)	intercept_speed_mps: Option<f64>; None means no geometry, never a default. Built and gated.	An envelope	Without it "a declared no-go fence still denies nothing": no intercept point to check.	MT-01 steps 5 and 6; MT-02 step 5
D-43 solve_assignment total (09-09)	Assignment::total_cost becomes Option<f64>; None means the optimum is not a representable f64. Built and gated, signed 2026-09-09.	(a) a magnitude precondition; (b) a new error; (c) a documented non-promise	See below.	MT-01 step 2: association runs every scan

D-43 is the most instructive single example in the ledger: a numerical-stability contract in a human-owned crate, where the record shows the owner choosing an answer the drafting agent had not offered. The fuzz gate's first working run (2026-09-08) found solve_assignment returning Ok with total_cost = -inf on an all-finite 5-by-2 matrix with two entries near the representable limit, whose selected entries summed past it. A precondition was rejected because "no precondition can be both safe and tight, because which entries the optimum selects is not known until it has been solved". An error was rejected because the pairing is provably correct in that case and all three non-test callers (GlobalNearestNeighbor::associate, the gungnir-fusion-async associate step, the CLEAR stage in gungnir-metrics) read only the pairing, so "an error would have dropped a whole scan of tracking over a number none of them reads". A doc comment was rejected as "the silent propagation docs/agent-ic-workflow.md's low-trust tier names". The fourth answer "refuses no solvable input, discards no correct pairing, and makes the unrepresentable case impossible to read by accident, because the type enforces it rather than the documentation".

The honesty in the record matters as much as the fix. The signature states that floating-point addition is not associative, so Some versus None is exact about the solver's own accumulation order, and that an order-free sum was "judged not worth the added subtlety in a human-owned file whose one production cost matrix is bounded near 1.1e4". The change record cites 200,000 top-exponent matrices in which 31,679 optima overflowed, every one confirmed unrepresentable by brute force; that measurement is recorded in ARCHITECTURE.md section 10 item 122 and the module doc, with no harness or data in the repository, so say "recorded", not "reproducible". After the change a manually dispatched fuzz run completed its fixed 20-minute budget, 244,022,222 inputs, without a crash; the corpus is not checked in. The strengthened postcondition requires a None to be matched by a selected entry above f64::MAX / n, "so a solver returning None to make the target pass would fail it".

Note: docs/agent-ic-workflow.md records D-43 as "the first signature under this clause" and summarizes it as "what the owner decided was the contract, not the fix". That is the trust-tier method in ten words.

UX and roles

Id	Decided	Rejected	Why	Mission
D-05 Roles (09-04)	Commander, planner and intelligence analyst adopted; nine roles with D-30. Built and gated (gungnir-security/src/auth.rs).	Leaving them proposed	The authority matrix "cannot be enforced for the roles that hold area-layer authority and product release". Planner got view-only permission because the matrix has no Planner row and the owner would not infer one.	MT-06 step 3; MT-08 step 1; MT-09
D-12 Vocabulary (09-04)	NATO and joint terms by default; a per-deployment override table; an empty vocabulary refused. Built and gated.	One nation's terms hard-coded	"No single nation's doctrine is assumed."	Every role's panels

Id	Decided	Rejected	Why	Mission
D-17 Docking (09-04)	Dockable panels; viewport, approval queue and replay detachable; decision dialogs stay with the queue. Built and gated (gungnir-app/src/dock.rs).	One window, fixed grid per role	"A decision separated from the queue it came from is a decision taken without its context."	MT-01 step 6: map on one screen, queue on the other
D-19 Docking crate (09-05)	egui_tiles 0.10, the line on egui ^0.29; cargo tree -d shows no second egui. Built and gated .	egui_dock; egui_tiles 0.11	A tree of tabs and splits maps onto WorkspaceLayout::for_role.	As D-17
D-28 MOP-37 targets (09-06, re-planned 09-08)	Skip wireframes; one participant per role on rendered panels; targets from measured values. Package prepared; sessions not run .	A wireframe round; targets from a heuristic walkthrough	Tracing the seed found the allocator re-proposing an unchanged assignment every tick (GAP-097), fixed the same day.	MOP-37 across all roles
D-35 Theme tokens (09-06; built 09-08)	Flat constants at the time; then theme::Palette with day() and night(), chosen once at start-up, 404 references across 29 files converted (item 116). Built and gated .	A Palette as a corollary of the theme review	The record's requirement is that a shift in the picture's colors must never be a mid-session surprise. The row has no amendment recording the build.	MT-04 and MT-05, night threads by their triggers
D-36 to D-38 Halo, numerals, viewport (09-06)	Halo stays white; the assignment line moved to teal; compared numerals monospace via theme::numeral; viewport colors unchanged. D-36 built and gated (viewport_line_colors_are_pairwise_distinct); D-37 and D-38 built, ungated .	A cyan halo; a six-size type scale	White is reserved for selection; five map colors stay pairwise apart.	MT-01 step 6 (compare time-to-impact down the queue)

Governance, hosting, and licensing

Id	Decided	Rejected	Why	Mission
D-10 Hosting (09-04; amended 09-07, 09-08)	Self-hosted GitLab, then reversed to GitHub Actions and the GitHub Container Registry, eight workflows kept. The gpu runner on the owner's RTX 5060 Ti workstation; gpu-fusion.yml stays workflow_dispatch permanently. Hosting built; GAP-061 in progress; the GPU dispatch has never completed .	A pull_request trigger on a workstation runner; running the GPU tests by hand	Dispatch "can only be fired by a caller with write access, so no fork's pull request can reach the machine". Running by hand "produces no CI log", and the evidence package is "the logs of every gate that ran, assembled by nobody".	MT-09 step 6 "apply and audit"; the accreditator's evidence package
D-34 License (09-07)	AGPL-3.0-or-later with section 7 terms, a commercial license and a CLA; SPDX header on all 399 .rs files; PN-21 About panel. Built and gated (cargo-deny in ci.yml). Not reviewed by counsel.	Apache-2.0 or MIT	Under a permissive license "a competitor could run a modified gungnir-node as a hosted service for a customer, never convey a binary, and owe nothing"; section 8 makes that the normal deployment shape; AGPL section 13 closes it.	PN-21 reaches every role; no thread step

D-34 carries the most judgment per sentence. The license was chosen from the deployment architecture, not from preference: the hosted node is the normal profile, so the network clause is load-bearing, and attribution became an enforceable section 7 term rather than a request. Two consequences are not license text: AGPL 5(d) makes the notice obligation conditional on the program's own interface carrying notices, so the About panel was built the same day with a test asserting it says what NOTICE says; and deny.toml became "a distributability control rather than hygiene", with OpenSSL removed as its only GPL-incompatible entry. The review that followed made cargo deny check pass all four checks for the first time, after the owner chose to ignore two advisories (cgmath, ttf-parser) rather than fork, because "owning a font parser with no upstream is a larger accreditation question than the advisory it would close".

ML and GPU

Id	Decided	Rejected	Why	Mission
D-40 Inference runtime (09-08)	ort 2.0.0-rc.13 with load-dynamic, behind gungnir-ml's default-off onnx-runtime feature; GAP-077 un-deferred. Built, ungated: never executed in any test or CI job.	tract (smaller pure-Rust op subset); ort's default download-binaries	Downloaded binaries "may carry telemetry, not acceptable for this system". Without a matching runtime ort panics twice over -- the first panic poisons an internal global mutex that ort locks again from an atexit handler, which panics a second time in a context that cannot unwind -- and hard-aborts the process at exit; observed directly, not theorized, and confirmed by reproduction. Section 10 item 120 says "built from Microsoft's source"; the manifest says load-dynamic and the manifest is the truth.	MT-05 step 2 and MT-01 step 3, later (GAP-080 Open)

The GPU path has no numbered decision of its own; its governance is D-10's 2026-09-08 amendment. Its gate is the verification row "GPU path against CPU reference" (transform within 1e-3, inlier ratio within 0.01). The kernels are naga-validated in CI; the hardware tests are `#[ignore]`d behind `gpu-tests`; one pass on the RTX 5060 Ti is the implementing agent's self-report, and the registered runner has not completed a dispatch.

Decisions taken against the recommended default

The ledger marks a recommended default only from D-23 onward. D-23 to D-32 record the owner taking it, D-23 verbatim ("recommended by the agent, accepted by the owner, implemented the same day"), and D-19's `egui_tiles` likewise. For D-01 to D-22 the public repository records no default at all. Wayne's own notes name D-01, D-07, D-10, D-11 and D-14 as taken against the drafting agent's default; that is a recollection this document cannot verify from GitHub alone, so present it as one. What the repository does record is the owner going against, or beyond, what was on the table:

1. **D-43.** The register framed three answers and "the owner chose a fourth while walking it", a type that "says the one thing that is true: there is an optimal assignment, and its cost is not a number".
2. **D-40.** Un-deferred against the deferral's own two standing conditions, neither of which had changed, "on his own authority as the reviewer that deferral named, because Plan 09's schedule needs the runtime question settled ahead of the first trained model rather than behind it"; the telemetry-bearing binary download refused.
3. **D-41.** Full projection support "chosen over a narrower WGS84-only first step deliberately".
4. **D-42.** Two clouds, and two named exclusions (GCP KMS, Vault) recorded as future decisions rather than gaps.
5. **D-33.** The type tree pinned "at the owner's direction" where the first answer had left it open; the framing pinned a day later with the earlier refusal explicitly withdrawn.
6. **D-32 as executed.** The owner did not record the ADS-B capture the decision assigned to him; buying the specification later "changes only which oracle the same tests cite".
7. **D-10.** His own GitLab choice reversed to GitHub three days later; the GPU workflow kept on manual dispatch "not as a wait, as a decision".
8. **D-34.** AGPL over the permissive licenses on the deployment-shape argument; two advisories ignored rather than forked or held.
9. **D-28 sequencing.** The panels started before the usability round, "accepting the rework risk explicitly" (`docs/ux/usability-test-plan.md`).
10. **D-09 / GAP-064.** The paywalled STANAG 4676 specification not purchased, leaving that codec `NotImplemented` rather than written from memory.

The honest answer, if asked for the pattern: the variances all run one direction. Each refuses a plausible shortcut (a capture from memory, a binary from a CDN, a doc comment as a guard, a fork with no upstream) and records the cost of the refusal in the register.

Ledger rows a reviewer will find stale

Name these before a reviewer does; each is a documentation defect, not a hidden failure. D-41's row says "Not yet recorded in `agentic-coding-standards.md` section 2.9 or built" while `proj 0.31`, the `crs` feature, the `proj-crs` CI job, and the section 2.9 rows for `proj` and `proj-sys` all exist. D-39's row and the GAP-057 and GAP-060 closing actions say "not signed" while section 10 items 103, 104 and 111 record the 2026-09-08 signature, and four module headers in `gungnir-security` and `gungnir-remote` still say the same. D-29's row cites `gungnir-node/src/identity.rs`, a file that does not exist. D-20's row calls DN-23 an unsigned draft; DN-23 says signed 2026-09-05. `gungnir-resilience/src/lib.rs` still says the arbitration rule "is not yet locked"; D-03 locked it. D-35's row describes flat constants the Palette replaced. And D-16's

contradiction, docs/architecture.md against the verification table's section-2 preamble and the register's GAP-067 row, is the one most likely to be raised, because all three sit in the same repository at the same commit.

Chapter 12 -- The design notes, DN-01 to DN-30

What a design note is

A decision (chapter 11) says what was chosen. A design note says how one gap closes: the rule the component must keep, the dependency edge it takes or refuses, the model types it adds, the verification row agreed before any code, and, once built, what the code disagreed with. The thirty notes live in docs/design/. Twenty-two of them are the plan-11 set written from 2026-09-05; docs/plans/11-design-gap-closure.md counts those twenty-two alongside four consolidations -- the index README.md, dependency-edges.md, model-and-schema-deltas.md and verification-rows.md -- as its twenty-six documents. verification-rows.md records 23 rows across 22 capabilities written before their code existed (AP-17). Five of those twenty-two are human-owned (DN-08, DN-09, DN-10, DN-17, DN-22) and were signed 2026-09-05. The eight later notes, DN-23 to DN-30, sit in the same folder under the same format and carry their own signature dates, in the table below.

One documentation defect governs the status words below. Nothing here has run against a real sensor, a real peer or a real deployment: every result in this chapter comes from fixtures, fakes and replay. Most notes are covered by a section-2 row of docs/verification-capability-table.md: the tests run under cargo nextest run --workspace on every CI push, but that table's section-2 preamble (line 115) still says "None of these rows is a pass/fail gate yet", and its plan-11 subsection (line 247) repeats that the rows are "not gates yet" and that a row becomes a gate only when its test lands and its status moves in architecture.md, under GAP-067 -- while docs/architecture.md line 21 records that walk as done and confirmed by the owner on 2026-09-07, marking the rows it checked **Specified** in its own status table, and docs/mission/gap-analysis/gap-register.md line 101 keeps GAP-067 Open. All three texts are in the repository, and this is a known documentation defect rather than a question with a settled answer. Below, "built, tested to a pre-agreed criterion (§2)" means a test written to the agreed criterion runs in CI but the table still lists the row as Draft; "\$1" means the row sits in the tracking-core gate block with a measured disagreement.

The thirty notes

Note	Title	The rule it establishes	Thread step	Built (crate; status)
DN-01	Defended assets	An empty list yields exposure: None, never a hidden fallback point; an unknown priority "is an error, not a default"	MT-01 step 4	gungnir-model, gungnir-assessment; built, tested to a pre-agreed criterion (§2)
DN-02	Prediction and approach	"A prediction beyond the configured horizon is not produced. It is not clamped"; the predictor is named on every prediction	MT-01 step 4, MT-02 step 3	gungnir-assessment/src/prediction.rs; built, tested to a pre-agreed criterion (§2)
DN-03	Warning	"Failure is loud": an unreachable endpoint leaves the warning Failed; a late warning "is never closed by the passage of time"	MT-02, MT-04	gungnir-workflow/src/warning.rs; built, tested to a pre-agreed criterion (§2) against a stub; no transport, row not closed
DN-04	Effector model	layer has no default; "Never propose a resource at or below its reserve"; cost is unitless	MT-01 step 6	gungnir-model/src/effectors.rs, gungnir-intercept-service; built, tested to a pre-agreed criterion (§2)
DN-05	Fires	"Every check runs and every result is reported"; a check that cannot be evaluated fails, never passes	MT-06 steps 5-7	gungnir-model/src/plans.rs, gungnir-policy/src/fires.rs; built, tested to a pre-agreed criterion (§2), signed 2026-09-06
DN-06	Engagement and effect	"Indeterminate is the point of this design"; track-lifecycle evidence is labeled weak	MT-01 step 8	gungnir-intercept-service/src/engagement.rs; built, tested to a pre-agreed criterion (§2)
DN-07	Handoff	"A handoff is only ever produced from a recorded decision"; an undeliverable one is "never dropped and never presented as delivered"	MT-01 step 7, MT-06	gungnir-model/src/handoff.rs, gungnir-app/src/handoffs.rs; built, tested to a pre-agreed criterion (§2)

Not e	Title	The rule it establishes	Thread step	Built (crate; status)
DN-08	Policy configuration	"Silence about authority must deny; silence about expiry must not discard"	governs DN-09, DN-10	gungnir-model/src/policy_settings.rs, gungnir-config; built, tested to a pre-agreed criterion (\$2), signed 2026-09-05
DN-09	Authority and control status	Hold is the default; "If no rule matches, the answer is denied"; "A denial explains itself"	MT-01, MT-02, MT-07	gungnir-policy/src/authority.rs; built, tested to a pre-agreed criterion (\$2), signed 2026-09-05
DN-10	Queue expiry and escalation	No automatic accept on expiry "under any configuration"; an expiry is neither dropped nor auto-rejected	MT-01 saturation	gungnir-command/src/queue.rs; built, tested to a pre-agreed criterion (\$2), signed 2026-09-05
DN-11	Sensor control and tasking	"Local state does not change until the sensor acknowledges"; no adapter means Not Implemented, not apparent success	MT-07, MT-03, MT-08 steps 2-3	gungnir-sensor-management, gungnir-workflow, SAPIENT sink in gungnir-ingest; built, tested to a pre-agreed criterion (\$2), sink signed 2026-09-08
DN-12	Coverage and gaps	Single-sensor coverage "is reported as a gap of its own severity, not as coverage"	MT-09, MT-07	gungnir-analytics/src/coverage.rs; built, tested to a pre-agreed criterion (\$2)
DN-13	Sensor re-tasking	"An empty result is a real answer"; "It recommends; it does not task"	MT-07 step 3	gungnir-decision/src/sensor_plan.rs; built, tested to a pre-agreed criterion (\$2)
DN-14	Hazard layer	"The hazard layer is descriptive"; it never denies an engagement	MT-04	gungnir-geo/src/hazard.rs; built, tested to a pre-agreed criterion (\$2)
DN-15	Anomaly detectors	"Never a verdict"; no detector quarantines, drops or downgrades anything	MT-05, MT-07	gungnir-analytics/src/anomaly.rs; built, tested to a pre-agreed criterion (\$2)
DN-16	Peer sources	"Quality is assigned by us, not claimed by them"; a launch warning never creates a track	MT-01, MT-02	gungnir-model/src/exchange.rs, gungnir-ingest/src/adapters/peer.rs; built, tested to a pre-agreed criterion (\$2)
DN-17	Releasability	combine takes the most restrictive input; filtering is "by removal, with a count"	MT-05, MT-08	gungnir-model/src/releasability.rs, gungnir-api; built, tested to a pre-agreed criterion (\$2), signed 2026-09-05
DN-18	Coalition exchange	Agreement and marking are "two independent gates, and the restrictive one always decides"	MT-01 warning, MT-08	gungnir-model, gungnir-api, gungnir-remote/src/link.rs; built, tested to two of the five pre-agreed criteria (\$2); Warnings and Reports have no producer; amendment 2 signed 2026-09-08
DN-19	Order of battle	"Every figure traces to a journal"; the analyst concludes, not the product	MT-08 step 5	gungnir-reporting/src/order_of_battle.rs; built, tested to a pre-agreed criterion (\$2)
DN-20	After-action review	"A finding points at the record, not at a memory"; a review cannot close with an action open	MOE-12	gungnir-workflow/src/review.rs; built, tested to a pre-agreed criterion (\$2); API routes not built
DN-21	Battle rhythm	"A maintenance window never suppresses a coverage gap"; the scheduler runs on mission time	MOE-13	gungnir-model/src/rhythm.rs, gungnir-reporting/src/rhythm.rs; built, tested to a pre-agreed criterion (\$2)

N ote	Title	The rule it establishes	Thread step	Built (crate; status)
D N -2 2	Key management	seal and unseal, never get_key; "Rotation never rewrites existing data"; the desktop "starts with journal encryption off and says so"	ARCHITECTURE.md §8.5	gungnir-security; the key-export surface built and gated (§1 row, table line 42), the custody, rotation and escrow row built and tested to a pre-agreed criterion (§2 line 299, CAP-6.4, still Draft), signed through amendment 4 (2026-09-08); amendment 5 managed_service.rs built against a CloudKeyService fake, unsigned; no call has been made to a real AWS KMS or Azure Key Vault
D N -2 3	Operator authentication	"Authentication never invents attribution"; failure is rate-limited, "never a hard lockout by default"	CAP-6.1	gungnir-security/src/session.rs, token.rs, account_store.rs; built, tested to a pre-agreed criterion (§2), signed through amendment 2 (2026-09-08)
D N -2 4	Mission profiles and algorithm baselines	"Exactly one candidate per declared profile carries promoted: true"; a baseline id is stamped only once applied	MT-09	gungnir-model/src/profiles.rs, gungnir-modelops, gungnir-app/src/governance.rs; built, tested to a pre-agreed criterion (§2), signed 2026-09-05
D N -2 5	Cursor-on-Target	"A multicast mesh sink has no party"; "A self-report is not a track and never enters fusion"	MT-08 step 6, MT-01 step 7	friendly-set half in gungnir-policy/src/fires.rs: built, tested to a pre-agreed criterion (§2), unsigned; codec, feed and sink: not built
D N -2 6	Laydown options	"an options table with one row is theatre"; a laydown that could not be evaluated did not score zero	MT-05	gungnir-model/src/laydown.rs, gungnir-ui, gungnir-app; built, tested to a pre-agreed criterion (§2); gap-acceptance control not built
D N -2 7	Bearing-only detections	"A bearing must never be turned into a position by assuming a range"; a bearing may update a track and may not initiate one	MT-02, CAP-1.1	gungnir-model, gungnir-coord, gungnir-fusion-async, gungnir-viewport3d; built and gated (§1 rows), signed 2026-09-07 and 2026-09-09
D N -2 8	IMM in the pipeline	Scoped to the CV/CT pair alone; "One selection per pipeline instance, not per track"	scenario 1 turn (ARCHITECTURE.md §10 item 94)	TrackFilter in gungnir-fusion-async, gungnir-filters/src/imm.rs, gungnir-config; built and gated (§1 rows), signed 2026-09-07
D N -2 9	A third mode for the IMM	"This is not a tuning gap": a six-dimensional state has no acceleration component	DN-28 §7 follow-up	two MotionModel<9> types proposed for gungnir-core; not built, unsigned
D N -3 0	Measurement noise from the baseline	Validated "unconditionally" finite and positive, because the same array seeds a new track's prior covariance	DN-28 §7 follow-up	gungnir-config, PipelineSetting s::from_baseline; built, unsigned

Invariants the type system holds

This is the strongest Rust evidence in the note set; it is worth being exact about which claims are compile-time facts and which are conventions with a test behind them.

A violation does not build

- **DN-27, a bearing is not a position.** Measurement (gungnir-model/src/lib.rs) is a three-variant enum, Position, RangeAzimuthElevation, Bearing, and Measurement::position_enu() returns None for both angular variants because neither carries a sensor origin. In the pipeline a bearing is a separate struct, BearingDetection; FusionPipeline::initiate takes an SVector<f64, 3> position and offer_bearing never calls it. The gated test (gungnir-fusion-async/tests/bearing.rs) submits 100 consistent bearings from one fixed sensor and asserts 0 tracks initiate. elevation_rad: Option<f64> keeps a missing elevation apart from the horizon; SCHEMA_VERSION went from 2 to 3 for it.
- **DN-16, a launch warning cannot become a track.** ProtocolAdapter::poll returns detections only; warnings leave through a separate LaunchWarningSink queue (gungnir-ingest/src/adapters/peer.rs), and LaunchWarningReport carries no kinematic field.

- **DN-28 and DN-29, const generics as a wall.** `Imm<const N, const M>` holds `Vec<Box<dyn ModeFilter<N, M> + Send>>` (`gungnir-filters/src/imm.rs`); a `KalmanFilter<ConstantAcceleration, 9, 3>` can never be boxed as `dyn ModeFilter<6, 3>`, which is why DN-29 exists. The `+ Send` is load-bearing: `gungnir-fusion-async` holds a `TrackFilter` across an await point in a spawned task. `from_baseline` refuses any selection outside `IMPLEMENTED_FILTERS` (four names, `pipeline.rs:341`).
- **DN-08 and DN-11, required arguments instead of hidden defaults.** `ConfigStore::apply(&mut self, baseline, now)` takes the clock; `FileConfigStore::new(path, known: KnownVocabulary)` makes the action vocabulary a required argument rather than a permissive default, and `validate_authority_names` refuses an empty one at apply rather than at construction; `SensorControl::issue(..., requirement: Option<RequirementId>)` makes every caller say `None` deliberately.
- **DN-09, DN-17 and DN-23, safe defaults and separated facts.** `WeaponsControlStatus` is `Free, Tight, #[default] Hold` with `Ord`, so the least permissive is the type's own default; `Releasability` defaults to `Internal`. `SessionState` is `SignedIn | NobodySignedIn | Expired | StoreUnavailable`, not `Option<OperatorSession>`, because those are three different facts; `AuthFailure::Rejected` is one variant on purpose, so a failure never says which half was wrong.
- **DN-21 and DN-05, shape over checks.** Every `SensorTaskEvent` variant carries a `SensorTaskId`, so `task()` is infallible; the maintenance overrun became `RhythmEvent` rather than making that accessor fallible for one variant. DN-05's `PlanView.kind: PlanKind` replaced solutions, `SCHEMA_VERSION` 1 to 2 with no deprecated mirror, moving ten call sites across six crates (`model-and-schema-deltas.md §3`).

Held more weakly than the notes say

- DN-22's "no consumer can obtain key bytes" is held by two source-scan tests, not by the type system. `KeyProvider` (`gungnir-security/src/keys.rs`) exposes `active`, `state`, `seal`, `unseal`, `rotate` and `sign`, and no getter; TLS got sign through `rustls`'s `SigningKey` trait rather than an exporter. The two tests in `gungnir-app/tests/architecture_compliance.rs` fail the day one appears -- both were checked by adding an exporter and watching them fail -- but they are a nine-name signature heuristic plus a pin on the six method names, not a type-level proof.
- DN-07 §8 says a `handoff` "cannot be constructed without a decision record ... checked by the compiler". `Handoff::from_decision` is the only named constructor, but every field of `Handoff` is `pub` (`gungnir-model/src/handoff.rs`), so a struct literal compiles anywhere. What holds the rule is the required-argument signature plus a source scan, `gungnir-app/tests/no_execution_without_decision.rs`.
- DN-23 §3 says there is "no constructor that takes a bare `OperatorId`", and `gungnir-security/src/session.rs` adds that `OperatorSession` is "Constructed inside this module and nowhere else". Its fields are `pub` and it is built by literal in `gungnir-security/src/token.rs` and `gungnir-api/src/transport.rs`, both after a verification. The note's narrower claim holds; the module's wider one is convention, and no test scans for it.

Design versus reality: the amendments and the §3a corrections

Half the notes record where the code refused the design. Fourteen of the thirty carry a numbered amendment section or a design-versus-reality section -- DN-01, 03, 04, 06, 08, 10, 11, 14, 15, 16, 18, 21, 22, 23, counted by heading across `docs/design/DN-*.md` -- and only two of those, DN-01 and DN-15, carry a numbered §3a; DN-01 carries both. The remaining corrections below sit in a note's later sections, in a consolidation, or in the follow-up note the finding created. Read together they show the process reacting to the code rather than editing the record.

- DN-01: the assessor could not hold an `AssetListView`; assets are geodetic, tracks are local ENU, and the conversion lives in `gungnir-coord`, which `gungnir-assessment` does not depend on. The caller anchors each asset. The note records this as a frame mismatch found in implementation, not as a refused edge.
- DN-15: detectors typed as `fn(&[TrackView], ...)` "cannot be built" over the model edge D-13 refused to open for them; `TrackSnapshot` carries primitives, and it still does even though DN-12 later brought the unanticipated `gungnir-analytics` to `gungnir-model` edge for its coverage types (`dependency-edges.md §4a`).
- DN-20: `SessionId` was assumed to be a model type; it lived in `gungnir-store` and, shared by six crates, moved down to `gungnir-model` with the store re-exporting it, which avoided a sixth edge rather than adding one. The correction is recorded in `dependency-edges.md §4a` and in `docs/design/README.md`, not in either note.
- DN-10: every function was "implemented, fully tested, and called by nothing"; two defects were found by reading the note against the code while wiring it, not by the tests, which passed throughout.
- DN-21: `gungnir-reporting/src/rhythm.rs` held the types and nothing constructed them, so the "design only" line was already wrong.
- DN-23 amendment 1: `hash_passphrase` was called from tests in six crates and "from nowhere else -- no binary, no example", so no binary could create an account; `gungnir-node account add` now does.
- DN-03 amendment 3: `Warning::acknowledged` had no caller, so a delivered warning went `Sent` then `Late` for ever.
- DN-18 amendment 1: `Warnings`, `Reports` and `Handoffs` "appeared nowhere outside `gungnir-model`'s own unit tests"; three routes §6 said would not exist were added.

- DN-08 amendment 1: three of the four §8 criteria were unmet and the gap "had been closed on §6's schema alone".
- DN-05: the README row said the chain evaluated a fires task; FiresDeconflictionPolicy was constructed by nothing until GAP-036 placed it fourth in the desktop chain.
- DN-12 §6: the route returned a bare Vec<CoverageGap>, discarding what rule 2 preserves; it now returns the whole CoverageReport, or NotComputed.
- DN-24: §6 and §9 "could not both be true" about model.promote; nothing promotes at runtime, so no unreachable authority check was added.
- DN-28 §7: the whole-scenario criterion could not be scored; the comparison instant sits in a nine-dimensional constant-acceleration phase, and the default measurement noise understated the radar's height term twenty-five times. Both became DN-29 and DN-30.
- DN-27: the tracking service refused every bearing as NotAPosition until GAP-001 built the sensor-position resolver on 2026-09-07; a retained bearing's lifetime was not honored on screen until the 2026-09-09 review.
- DN-22 amendment 5 corrects §5 (the data key is in process memory) rather than pretending to satisfy it. One mismatch stays open: §14 says "once per rotation", while managed_service.rs::rotate() makes no service call.

Two further defects: docs/design/README.md's status table is stale where code moved after the row was written (the node resolver for DN-19, DN-26's rehearsal, DN-25's policy half, DN-22's OS keystore, DN-23's session route, DN-27's display, DN-04's speed field) and omits DN-29; and dependency-edges.md labels two different edges (s) under two sections both numbered 13. In none of these is the code behind the document.

Part V, Chapter 13 -- The algorithms

How to read this chapter

Every algorithm below is described the same way: the problem, the idea, the math, the variant chosen and why, where the mission set exercises it, and how it is verified. The status word in each heading is the one the style guide fixes. **Built and gated** means a test against a named oracle or criterion runs under cargo nextest run --workspace in CI and, where the crate is human-owned, the owner's signature is on record. **Built, unsigned** means the same without the signature. **Built, ungated** means the test exists and runs in CI but its row sits in section 2 of docs/verification-capability-table.md, whose preamble still says "None of these rows is a pass/fail gate yet" pending the GAP-067 walk, while docs/architecture.md records that walk as done on 2026-09-07 and the register keeps GAP-067 Open. That is a known documentation defect inside the repository; this chapter reports the table's wording. **Built, unit-tested, no table row** means the tests run in CI and the verification table does not name the algorithm in either section. **Not built** means an explicit NotImplemented.

Two facts govern the mission paragraphs. First, the pipeline that actually runs per track is a Joseph-form constant-velocity Kalman filter (or the CV/CT IMM when a baseline selects imm-cv-ct), a chi-square gate, Jonker-Volgenant global assignment, and the Stone-Soup-matched lifecycle, over a source-time reorder buffer (gungnir-fusion-async/src/pipeline.rs, doc lines 9-24). EKF, UKF, particle, square-root, RTS, JPDA, MHT, track-to-track fusion and the LMB are each gated in isolation and not selectable from a baseline (IMPLEMENTED_FILTERS is kf-cv, linear-kf, constant-velocity, imm-cv-ct). Second, the five engineering scenarios in gungnir-scenario are: 1 maneuvering aircraft, 2 maritime clutter, 3 urban convoy with injected bias and out-of-order video, 4 dense swarm, 5 adversarial geometry at 89.9 N on the antimeridian. No real sensor has fed any of them.

Oracle versions throughout: filterpy 1.4.5, Stone Soup 1.9.1, scipy 1.18.1, pymap3d 3.2.0, py-motmetrics 1.4.0. Fixtures are checked in under testdata/oracles/ with the oracle name and version stamped in each; MATLAB columns were never run (testdata/oracles/README.md).

Estimation

Coordinate frames (built and gated, signed 2026-09-05)

Problem. A radar reports relative to itself, a map wants latitude and longitude, a filter wants a flat local frame in meters. The hard direction is ECEF to geodetic, which has no closed elementary form on an ellipsoid and loses precision at the poles under the usual iterations.

Idea. Convert geodetic to Earth-centered Cartesian exactly, build one East-North-Up basis per origin from its sines and cosines, and use the transposed basis for the inverse so the pair is an exact inverse up to rounding.

Math. WGS-84 with $a = 6378137$, $f = 1/298.257223563$, $e^2 = f(2 - f)$. Forward: $N = a / \sqrt{1 - e^2 \sin^2 \phi}$, $x = (N + h) \cos \phi \cos \lambda$, $z = (N(1 - e^2) + h) \sin \phi$. Inverse: Heikkinen's closed-form solution of the quartic, with the radicand clamped at zero so a rounding-induced $-1e-30$ never becomes a NaN latitude (gungnir-coord/src/lib.rs:125-176).

This variant and why. Heikkinen rather than a Bowring iteration: the module doc says it is exact at every altitude and, unlike $h = p / \cos(\text{lat})$, does not lose precision at the poles, "which the Scenario 5 adversarial case drives through."

Mission. Scenario 5; every thread's tracking step, since sensors and assets are geodetic and tracks are ENU meters (MT-01 step 2).

Verification. pymap3d 3.2.0, criterion position error under $1e-6$ m, worst measured $2.1e-9$ m (gungnir-coord/tests/pymap3d_diff.rs). The inverse is compared in meters ($|\Delta lat| a$, $|\Delta lon| a \cos \phi$) so the criterion stays meaningful where longitude is degenerate. Proptest invariants in tests/invariants.rs cover mutual inverses, isometry, and no NaN to the poles and past the antimeridian.

Motion models CV, CA, CT (built and gated, signed 2026-09-05)

Problem. A filter needs, for a step dt , the transition F and the process-noise covariance Q . Three models: constant velocity (6 states), constant acceleration (9), coordinated turn (6, horizontal turn at rate ω).

Idea. State order is fixed as $[e, n, u, ve, vn, vu]$ because two other crates already read those blocks and it is what filterpy produces with `order_by_dim=False`. Q is continuous white noise integrated over the step, not the discrete model.

Math. Per axis for CV, $Q = q \begin{bmatrix} [dt^3/3, dt^2/2], [dt^2/2, dt] \end{bmatrix}$; the general element is $dt^{2 \cdot \text{ORDER} - i - j - 1} / ((\text{ORDER} - i - 1)! (\text{ORDER} - j - 1)! (2 \cdot \text{ORDER} - i - j - 1))$ (gungnir-core/src/lib.rs:96-129). CT uses $s = dt \operatorname{sinc}(\omega dt)$ and $c = dt (1 - \cos \omega dt) / (\omega dt)$, evaluated by Taylor series below $1e-3$ rad so the dropped term is $1.4e-22$; $(1 - \cos x)/x$ is written as $(x/2) \operatorname{sinc}^2(x/2)$ to avoid cancellation.

This variant and why. The doc states that picking discrete instead of continuous noise "rescales every covariance in the workspace by a factor of dt ," which is why the differential test pins it. Unit tests pin CT at $\omega = 0$ equal to CV to $1e-15$ and $F(2dt) = F(dt)^2$ to $1e-12$.

Mission. Scenario 1 runs all three phases (CV to 100 s, CT at 0.035 rad/s to 200 s, CA to 300 s); MT-01 step 2 uses CV, and CT through the IMM.

Verification. filterpy, Stone Soup and scipy expm; criterion exact match (about $1e-10$); worst $1.4e-14$ against filterpy, $4.4e-11$ against Stone Soup (gungnir-core/tests/motion_models_diff.rs). Finding about the oracle: Stone Soup's `KnownTurnRate` divides by the turn rate and returns NaN at $\omega = 0$, so those cases are adjudicated by scipy and listed as `degenerate_oracles` in the fixture.

Linear Kalman filter, Joseph form (built and gated, signed 2026-09-05)

Problem. Estimate position and velocity from noisy position measurements under a known linear motion model.

Idea. Predict with the model, update by weighting the innovation with a gain that balances model and measurement uncertainty.

Math. Predict $x = F x$, $P = F P F^T + Q$. Update $S = H P H^T + R$, $K = P H^T S^{-1}$, $x += K (z - H x)$, $P = (I - K H) P (I - K H)^T + K R K^T$, then $P = (P + P^T)/2$ (gungnir-filters/src/kalman.rs:165-196).

This variant and why. Joseph form, because "the short form is correct only for the optimal gain and in exact arithmetic; in floating point it loses symmetry and drifts indefinite." $F(dt)$ and $Q(dt)$ are evaluated at the actual dt every step, never cached, because Scenario 3 deliberately reports off nominal rate. A singular S is counted in `rejected_updates` rather than pseudo-inverted.

Mission. The default per-track estimator (`TrackFilter::ConstantVelocity`) in MT-01 step 2 and every whole-pipeline replay of Scenarios 1 to 5.

Verification. filterpy `KalmanFilter`; criterion $1e-6$ on state and on covariance Frobenius norm; worst $9.1e-13$ and $6.1e-14$ over 155 predict/update cycles in four synthetic cases (tests/linear_kalman_diff.rs), checked after every half-step because "two compensating errors in the gain and the covariance update" can reach the right answer by a wrong path. Honest caveat: filterpy's update is also Joseph form, so this is parity between two implementations of one formula, not an independent-method oracle.

Extended Kalman filter (built and gated, signed 2026-09-06)

Problem. A radar measures range, azimuth and elevation; converting to Cartesian first "throws away the shape of its uncertainty: a range-accurate, bearing-vague measurement becomes a fat circle instead of the thin arc it really is."

Idea. Linearize the measurement function at the predicted state and run the linear update with that Jacobian.

Math. $h(x) = [\operatorname{sqrte}(e^2 + n^2 + u^2), \operatorname{atan2}(e, n), \operatorname{atan2}(u, \operatorname{hypot}(e, n))]$ with the analytic Jacobian written out (nonlinear.rs:68-113); Joseph-form update, re-symmetrized.

This variant and why. Measurement models are a trait with an overridable `residual`, because "wrapping is a property of the measurement model, not of the estimator." Neither EKF nor UKF wraps an angle residual; the fixtures stay in one quadrant, and the doc says so.

Mission. Scenario 1 is its named data source; not selectable from a baseline (DN-28 section 6).

Verification. filterpy `ExtendedKalmanFilter`; criterion norm-wise relative error under $1e-4$ ($\|a - b\| / \max(\|b\|, 1)$ on the whole vector and matrix); two cases of 25 and 15 steps, every predict and update (tests/nonlinear_diff.rs). Finding about the oracle: the first EKF fixture was silently wrong because filterpy broadcast a column state against a 1-D

measurement into (3,3), so the recorded state had eighteen entries and "read exactly like a diverging filter"; every recorded state is now shape-asserted before it is written.

Unscented Kalman filter, Merwe scaled sigma points (built and gated, signed 2026-09-06)

Problem. Same as the EKF, without a Jacobian, for measurement functions where linearization is poor.

Idea. Push a deterministic set of $2n + 1$ points through the nonlinear functions and reconstruct mean and covariance from weighted moments.

Math. $\lambda = \alpha^2(n + \kappa) - n$; $w_{m,\theta} = \lambda / (n + \lambda)$, $w_{c,\theta} = w_{m,\theta} + (1 - \alpha^2 + \beta)$, the rest $1 / (2(n + \lambda))$; points $x \pm \text{column}_k(L)$ with $L L^T = (n + \lambda) P$; defaults $\alpha = 0.1$, $\beta = 2$, $\kappa = 0$ (nonlinear.rs:396-409).

This variant and why. Matches filterpy's MerweScaledSigmaPoints exactly, including the fact that filterpy factors with an upper U and uses its rows while nalgebra gives lower L, "and the kth row of U is the kth column of L." A covariance without a Cholesky factor is NotPositiveDefinite, reported rather than worked around.

Mission. Scenario 1 named; not selectable from a baseline.

Verification. filterpy UnscentedKalmanFilter; criterion as the EKF; mean weights w_m compared entry by entry to 1e-12 and summed to one. The covariance weights w_c , though present in the fixture, are not compared -- a gap worth naming, since $w_{c,\theta}$ carries the $(1 - \alpha^2 + \beta)$ term.

Square-root filter, QR array recursion (built and gated, signed 2026-09-06)

Problem. Over many cycles of a badly scaled problem (meters beside milliradians in one state) the standard form's P can go indefinite.

Idea. Carry a factor S with $P = S S^T$; a product of a matrix with its transpose cannot be indefinite. The price is a QR per step.

Math. Predict re-triangularizes the pre-array $[F S, G]$ with $G G^T = Q$. Update forms $[[R_c, H S], [0, S]]$, whose lower-triangular factor is $[[S_y, 0], [K_bar, S^+]]$, giving $K = K_bar S_y^{-1}$ (sqrt.rs:115-205). The factor is a QR of A^T , signs left alone because both results are invariant to a column sign flip.

This variant and why. One array_update serves both the linear and the Jacobian halves: "writing the recursion twice would have given two chances to get the array layout wrong."

Mission. Scenario 5's three nearly collinear polar radars are "the ill-conditioning the square-root filters exist to survive"; not wired into the pipeline, the standard filter stays the default.

Verification. filterpy's standard KalmanFilter (its SquareRootKalmanFilter exposes no process-noise factorization); criterion 1e-6 over 200 steps of two cases, one with measurement variances spanning 25 to 2.5e-5; a 100,500-cycle soak sampled every 500 cycles finds the reconstructed covariance finite, symmetric and PSD at all 201 checks (tests/sqrt_diff.rs). Two honest notes: the table row is titled "Square-root / UDU-factorized" and no UDU factorization exists, and the extended half is checked only against the workspace's own EKF over 120 steps on state alone.

Interacting multiple model, Blom/Bar-Shalom (built and gated, signed 2026-09-06; pipeline selection DN-28 signed 2026-09-07)

Problem. A target that sometimes flies straight and sometimes turns is tracked badly by either model alone.

Idea. Run one filter per model, mix their estimates before each predict by how likely each model transition is, and weight each model afterward by how well it explained the measurement.

Math. Mixing weights $\omega_{ij} = \pi_{ij} \mu_i / c_bar_j$; mixed prior $x_hat_{\theta j} = \sum_i \omega_{ij} x_i$, $P_{\theta j} = \sum_i \omega_{ij} [P_i + (x_i - x_hat_{\theta j})(x_i - x_hat_{\theta j})^T]$; mode update μ_j proportional to $c_bar_j \Lambda_j$ with $\Lambda_j = N(y_j; 0, S_j)$, evaluated before the mode's own update; combination adds the spread term so the output is "a moment-matched summary of a mixture and not the mixture itself" (imm.rs:360-440).

This variant and why. The Blom/Bar-Shalom recursion in IMMEstimator's order; likelihood floored at `f64::MIN_POSITIVE` as filterpy does; a transition row not summing to one within 1e-9 is refused, not renormalized. Modes are a trait because CV and CT are different Rust types.

Mission. Scenario 1's turn phase; MT-01 step 2 when a baseline selects imm-cv-ct, one selection per pipeline instance. Truth-scored: kf-cv confirms no track through the turn, imm-cv-ct confirms one track within the scenario's own 500 m truth bound, under two test-local overrides -- the scenario radar's modeled measurement noise instead of PipelineSettings::default()'s generic figure (the mismatch DN-28 section 7 named, which DN-30, built and gated but unsigned, later closed on the baseline) and the truth's own 0.035 rad/s turn rate (gungnir-tracking-service/tests/scenario_truth_replay.rs:346-387). Scenario 1's own whole-replay row measures a worst position error of 169 m against that bound with kf-cv, down from 238 m before DN-30; the IMM turn-phase row measures 138 m. The CA third phase is excluded because a six-state IMM cannot represent it (DN-29, not built).

Verification. Stone Soup 1.9.1 has no Gaussian IMM, so the oracle is filterpy IMMEstimator, recorded in the module, fixture and table; criterion state relative 1e-4, mode probabilities 1e-3; the table prints 0.0 and the measured residual is round-off, about 5e-16 on state and 3e-15 on mode (tests/imm_diff.rs, three 40-step cases, two linear modes). The "no IMM"

inspection is not reproducible from the repository because the pinned Python environment is not checked in.

Particle filter, SIR (built and gated, signed 2026-09-06)

Problem. Carry a weighted cloud instead of one Gaussian, so "the target went left or right" stays two lumps.

Idea. Sample states, weight each by measurement likelihood, and resample when the weights collapse onto few particles.

Math. Resample when $N_{\text{eff}} < 0.5 N$. Three resamplers, all with the same expected copy count and different variance: systematic (one uniform draw, then equally spaced positions along the cumulative weights), stratified (one draw per stratum), multinomial (one independent draw per particle). Process noise is drawn through `psd_factor`, an eigendecomposition $A = V \text{sqrt}(\Lambda)$ rather than Cholesky, because the same function is applied to caller-supplied P_0 and R that may be singular (`particle.rs:24-89`).

This variant and why. Systematic is the default "because it has the lowest resampling variance of the three and is what the reference filter uses, so the comparison is between two filters rather than between two resamplers." Every stochastic method takes `&mut impl Rng`; nothing calls `thread_rng`. The doc records that an earlier draft asserted the continuous CV Q was singular; the test caught it.

Mission. Scenario 4 named; not selectable from a baseline.

Verification. A hand-written numpy SIR over `filterpy`'s `systematic_resample` (`filterpy` has no particle filter), 40 seeded trials of 2,000 particles on one linear-Gaussian case, with the exact Kalman posterior carried alongside as the known answer (`tests/particle_diff.rs`). Criterion: KS statistic on each final-step position component below the $\alpha = 0.05$ critical value, and the across-trial mean within 0.5 measurement standard deviations of the exact posterior. Finding about the fixture: the first version required all 180 per-step, per-component means inside 2σ , which the null hypothesis itself violates about eight times in 180; it failed at 2.22σ , and the row was rewritten as a KS test plus an excursion count against a binomial bound, with neither number looser than the row's.

Rauch-Tung-Striebel smoother (built and gated, signed 2026-09-06)

Problem. After a run, revise every estimate using measurements that arrived later.

Idea. A backward pass that pulls each filtered estimate toward the smoothed one that follows it.

Math. $P_{k+1}^- = F P_k F^T + Q$, $C_k = P_k F^T [P_{k+1}^-]^{-1}$, $x_{k|N} = x_k + C_k [x_{k+1|N} - F x_k]$, $P_{k|N} = P_k + C_k [P_{k+1|N} - P_{k+1}^-] C_k^T$ (`lib.rs:49-104`).

This variant and why. The signature takes covariances because the scaffold's did not and "a smoother given only states cannot compute $C(k)$, so the old signature could never have been implemented."

Mission. Scenario 5 names it; no thread step runs it today.

Verification. `filterpy rts_smoother` over an already-gated forward pass, relative error under $1e-6$ at every step of two cases; the fixture's forward states are fed straight back in so a disagreement cannot come from re-running the filter (`tests/rts_diff.rs`).

Association

Chi-square gating (built and gated)

Problem. Decide which detections are close enough to a predicted track to be candidates at all.

Idea. Measure distance in units of the innovation's own uncertainty and cut at a tabulated quantile.

Math. $d^2 = y^T S^{-1} y$, $y = z - H x$, $S = H P H^T + R$; admit when $d^2 \leq \text{threshold}$, tabulated for 1 to 6 degrees of freedom at 95 and 99 percent (`gating.rs:57-74`).

This variant and why. d^2 by a Cholesky solve, not an explicit inverse, because the inverse "loses roughly a squared condition number of accuracy, and S is at its worst conditioned exactly when a track is well-observed in one axis and barely observed in another." The pipeline gates positions at 99 percent with 3 degrees of freedom and bearings at 99 percent with 1 or 2 (`pipeline.rs:278, 903, 913`).

Mission. Scenario 2's clutter; MT-01 step 2, MT-04 step 1.

Verification. Closed-form chi-square with thresholds from `scipy.stats.chi2.ppf`; membership exact on all 34 cases; $1.2e-15$ relative on the squared distance (`tests/association_diff.rs`).

Jonker-Volgenant assignment and GNN (built and gated, signed 2026-09-05; contract D-43 signed 2026-09-09)

Problem. Given a cost for every track-detection pair, choose the one-to-one pairing of least total cost.

Idea. Shortest augmenting paths with dual variables, $O(n^2 m)$; rectangular matrices solved directly, rows exceeding columns on the transpose, because padding "changes the optimum when the pad value is smaller than some real cost."

Math. Minimize $\sum c_{i, \sigma(i)}$ over injective σ ; the total is read back from the input matrix, not accumulated inside the solver; `Assignment::total_cost` is `Option<f64>`, `None` meaning the optimum's accumulated sum is not a representable `f64` (`assignment.rs:101, 263`).

This variant and why. D-43: the fuzz gate's first working run (2026-09-08) found ok with `total_cost = -inf` on an all-finite 5x2 matrix. The owner rejected a magnitude precondition (cannot be both safe and tight), an error (would drop a whole scan over a number no in-tree caller reads), and a doc comment (silent propagation), and chose the type. All three non-test callers use only the pairing; the pipeline's matrix is bounded near $1.1e4$. The 200,000-matrix characterization (31,679 overflowing optima, each brute-force confirmed unrepresentable) is recorded in `ARCHITECTURE.md` section 10 item 122 and the module doc, not reproducible from the repository.

Mission. MT-01 step 2, every scan; Scenario 2.

Verification. `scipy linear_sum_assignment`, gate $1e-12$ relative on cost, worst observed $1.9e-16$ on a 25x25; pairing compared only where uniqueness was proven by brute force over permutations, feasible up to dimension 7, which covers 11 of 15 cases (one proven tie, three too large to decide). After D-43 the manually dispatched fuzz run completed its fixed 20-minute budget, 244,022,222 inputs, clean (run 34355222718).

Joint probabilistic data association (built and gated, signed 2026-09-06)

Problem. A hard assignment "invents" an answer when two targets cross in clutter and may swap identities permanently.

Idea. Instead of choosing, compute for each track the probability that each gated detection, or none, belongs to it, summed over every consistent joint event.

Math. $L_j(t) = N(z_j; \hat{z}_t, S_t) P_D / \lambda$, $L_0(t) = 1 - P_D$; a joint event assigns each track one detection or nothing with no detection used twice; event weights are products, normalized; β_{tj} sums the events where t took j (`jpda.rs:27-46`).

This variant and why. Exact enumeration, bounded at 8 tracks, 12 detections and 200,000 events, and refused past those bounds, never truncated: "a truncated enumeration returns probabilities that look ordinary and are wrong." `TrackPrediction` carries only the predicted measurement and S , so association cannot depend on identity.

Mission. Scenario 2; MT-04 step 1 would use it, but the pipeline associates with GNN only, and `MAX_DETECTIONS = 12` is the dense-group trigger.

Verification. Stone Soup JPDA over `PDAHypothesiser`, six synthetic single-scan cases; the coded gate is an absolute difference under $1e-3$ on $[0,1]$ probabilities including the miss (the table's wording says relative); observed agreement about $1e-15$; per-track probabilities sum to one within $1e-9$ (`tests/jpda_diff.rs`).

Multiple hypothesis tracking, N-scan (built, unit-tested against hand-derived scenes; the table's criterion is measured by no code; signed 2026-09-06)

Problem. Defer the association decision across scans: "the scan at the crossing genuinely does not say which is which and the scan three later obviously does."

Idea. Keep a bounded tree of association histories, each with a log weight; prune to the K best after each scan; commit decisions older than N scans to the leading hypothesis and never revisit them.

Math. Log-domain weights, because a product of likelihood ratios "underflows to zero in double precision and every hypothesis looks equally good"; `log_gaussian_density` sums logs from the Cholesky diagonal (`mht.rs:115-129, 403-419`).

This variant and why. Zero scan depth is refused as "nearest-neighbour association wearing MHT's name." Branching reuses JPDA's bounds.

Mission. Scenario 2; a library type with no pipeline caller and no initiation or deletion.

Verification. Stone Soup 1.9.1 has no MHT hypothesiser and its multi-frame assignment will not import without `ortools`, so the tests are six in-module unit tests on hand-built two-track scenes. The honest reading: the crossing test asserts only the post-separation history, and with ground-truth predictions supplied the swap and no-swap hypotheses tie at the crossing scan, so it does not demonstrate resolution; the table's criterion "tree match at each pruning step; relative error $< 1e-3$ " is measured by no code; and `docs/gungnir-capabilities.md` still names the Stone Soup and `trackerTOMHT` oracle -- a documentation defect.

Random finite sets

GM-PHD (built and gated, signed 2026-09-06 as it then stood)

Problem. Under a raid of tens of targets, how many are there and where, without committing to identities.

Idea. Propagate an intensity function, a Gaussian mixture whose weights sum to the expected target count, not a probability.

Math. Predict scales weights by p_S and appends births; update keeps a $(1 - p_D)$ copy per component and, per detection z , a Kalman-updated copy weighted $p_D w N(z; H m, S) / (\kappa + \Sigma \dots)$; then prune below $1e-5$, merge within Mahalanobis 4.0, truncate at 100 components (`PhdConfig`'s default; the dense-group wiring raises it to 400, sized from

docs/performance-budgets.md's 200-track scenario) (gungnir-rfs/src/lib.rs:124-138, 1616-1760).

This variant and why. Textbook Vo-Ma. `extract_tracks` mints fresh identifiers every call because the intensity carries no identity.

Mission. Scenario 4; MT-01 step 2 under saturation, as the dense-group count only.

Verification. Stone Soup was driven and disagrees, so the oracle is the Vo-Ma recursion in `numpy`; criterion intensity weights within $1e-3$ and exact cardinality where unambiguous (the test applies the same $1e-3$ to both); observed 0.0 over three noiseless cases (`tests/phd_diff.rs`). Findings about the library: its `merge_components` clamps a merged weight to 1.0 (" $0.7 + 0.6$ merges to 1.0, checked directly"), wrong for weights that are expected counts; it applies survival inside the update. The first-scan gap turned out to be the generator's own transcription error (births stamped one scan in the past); fixing it and adopting both conventions collapses the gap to about $1e-16$, and the recorded worst disagreement "fell from 0.842 to 0.139 because a transcription error left the comparison, not because the library became an oracle." The 2026-09-08 explanation and regenerated fixture are gated but unsigned.

GM-CPHD (built and gated, signed 2026-09-09)

Problem. The PHD's count is only a mean: "a mean of 2.4 does not say whether the truth is 'almost always 2, sometimes 3' or 'often 0, occasionally 5'."

Idea. Propagate the whole cardinality distribution alongside the intensity.

Math. Vo, Vo and Cantoni 2007. Component shapes are the PHD's; the weights change by factors computed from the elementary symmetric functions of the detections' likelihood ratios against the prior cardinality. Predict thins the distribution binomially by p_s and convolves with a Poisson birth count (`lib.rs:208-216, 370-716`). Extraction commits to the mode.

This variant and why, and the finding. The $O(m^2)$ synthetic-division shortcut for the leave-one-out functions, $q_k = e_k - v_j q_{k-1}$, "is stable from only one end, and the forward pass is the wrong end whenever v_j is the largest value" -- exactly the well-matched target among clutter, with ratios of order 60 beside $1e-3$. On that vector with twelve clutter values the recurrence was 12x off at $k = 4$ and returned $+1.0e6$ where the truth is $1.0e-36$ at $k = 12$; in the filter it gave one component a weight of 1726 against a cardinality mean of 1.02 and, under a fat prior, committed to a birth 35 km away. Both the crate and its oracle shared the bug. Replaced by direct $O(m^3)$ recomputation, well under a millisecond; the abandoned recurrence is kept in both test modules and asserted to fail. Found in the owner's review before signing.

Mission. Scenario 4; dense-group mode when selected.

Verification. Stone Soup has no CPHD updater (confirmed by import), so the oracle is `gen_cphd_fixtures.py`'s own derivation, self-checked against brute-force association enumeration, the intensity-integral-equals-cardinality-mean identity, exact reduction to PHD under a Poisson prior, and a Monte Carlo variance comparison; criterion mean and intensity within $1e-3$, mode exact, four cases, the fourth one target in random annulus clutter under a fat prior, added only after two wrong drafts (a fixed ring of returns is indistinguishable from twelve stationary targets).

Labeled multi-Bernoulli (built, unsigned); delta-GLMB (not built)

Problem. The PHD family answers how many and where; the LMB answers which one is which, with a label issued at birth and never reissued.

Idea. Run the exact delta-GLMB update of an LMB prior over every association hypothesis, then project back onto an LMB by matching moments.

Math. Reuter, Vo, Vo and Dietmayer 2014. Association marginals come from a subset dynamic program over detection bitmasks, rows scaled by their peak (`lib.rs:1442-1540`), bounded at 12 detections per scan and refused past it.

This variant and why. "Every per-label marginal is exact after a single update"; the one approximation is discarding inter-label dependence, and "the error is what the next scan inherits." `GlmbFilter::labelled_tracks` returns `RfsError::NotImplemented`, naming the missing truncation scheme, and is kept "because naming it is how a reader can tell which of the two this build has" (`lib.rs:732-756`).

Mission. Scenario 4; not wired to the pipeline. The human-owned numerical-stability clause reaches `gungnir-rfs`, and `lib.rs:25` says "written and gated, not signed."

Verification. Stone Soup has no GLMB or LMB, established three ways by a generator that refuses to run if it finds one; the oracle enumerates marginals literally. Gate $1e-3$ on existence, association, spatial density and state; worst $3.7e-15$, $3.8e-15$, $7.5e-15$, $1.7e-14$ over five cases; an untruncated delta-GLMB alongside agrees exactly after the first update then diverges to $5.2e-7$ and $1.1e-4$, "the LMB projection's cost measured rather than asserted." Label continuity: over 40 scans with dropouts the identifier map changed on 0 of 36 LMB boundaries against 11 of 23 for the PHD on the same detections (`tests/lmb_label_continuity.rs`).

Track management and fusion

Track lifecycle (built and gated, signed 2026-09-05)

Problem. When does a run of associated detections become a track the operator sees, and when does a run of misses end it.

Idea. Confirm on cumulative hits, delete on consecutive misses, in a fixed order per step: prune last step's deletions, apply hits, then misses, coasting before deletion.

Math. Confirmation when `count(updates) >= min_points`; deletion on the `n`-th consecutive miss with `>= (lifecycle.rs:12-41)`. Pipeline defaults: confirm at 3 hits, delete after 3 misses.

This variant and why. Read off Stone Soup's `MultiMeasurementInitiator` and `UpdateTimeStepsDeleter`; "this corrects the scaffold," whose comment said consecutive hits.

Mission. Scenario 2's staggered land-mask occlusion windows and its below-one detection probability -- `plan_maritime_clutter` sets `radar.dropout = 0.0` and an occlusion window per vessel, and the replay tests run it at `pd: 0.8`, so 20 percent of detections are missed (`gungnir-scenario/src/lib.rs:462-467`, `gungnir-tracking-service/tests/scenario_truth_replay.rs:445-448`); the 0.15 dropout is `SensorModel::radar_coastal`'s unmodified default, which the scenario overrides. MT-04 step 1 coasting; MT-01 step 2.

Verification. 24 step indices exact across 12 sequences (`tests/lifecycle_diff.rs`); only the deleter is driven end to end, the confirmation condition is lifted from source. Finding about the oracle: an update built with `hypothesis=None` is not counted as an update by the deleter, which tests `isinstance(state, Update)` and `state.hypothesis`; a first fixture did that and reported a track deleted while being hit every step. The cases `dense_hits_never_deleted` and `hit_resets_the_miss_run` exist because they caught it. A known consequence: with `kf-cv` the pipeline fragments on the maneuvering scenario, which is what DN-28's IMM addresses.

Covariance intersection and information-matrix fusion (built and gated, signed 2026-09-06; not run in the pipeline)

Problem. The information sum $P^{-1} = P_1^{-1} + P_2^{-1}$ is correct only for independent errors, and "a track fused from A and B, fused again with B, counts B's evidence twice."

Idea. Blend the two information matrices with a weight ω chosen to minimize the fused determinant; the result is consistent for any unknown correlation.

Math. $P^{-1} = \omega P_1^{-1} + (1 - \omega) P_2^{-1}$, $x = P(\omega P_1^{-1} x_1 + (1 - \omega) P_2^{-1} x_2)$, ω by 100 iterations of golden-section search (`lib.rs:183-263`).

This variant and why. CI is the default; the information-matrix fuser "assumes the estimates' errors are independent, and says so here because nothing downstream can check it." Finding: with equal input covariances the determinant is flat in ω while the state sweeps the whole segment; the two sensors with identical covariances "landed 1.2e-6 apart" between Brent and golden section, so if the search does not improve the determinant by 1e-9 over $\omega = 0.5$, $\omega = 0.5$ is used, and the oracle implements the same rule from the description.

Mission. Scenario 3; MT-06 step 2. GAP-013 is Closed in the register with the note that two sensors at one instant still make two tracks.

Verification. Hand-derived CI with `scipy` bounded Brent, against Rust's golden section; criterion 1e-6 on state and covariance Frobenius; worst 2.2e-7 on covariance and 7.6e-8 on state across four cases (`tests/fusion_diff.rs`). Stone Soup's `ChernoffUpdater` at fixed $\omega = 0.5$ agrees with the hand derivation exactly; the table's "0.0" cell conflates that cross-check with the Rust result and should read about 2.2e-7 -- a documentation defect.

Sensor registration by weighted least squares (built and gated, signed 2026-09-06; not wired)

Problem. Two sensors see one object at two places; part of the difference is bias.

Idea. Weight each shared pair's difference by the inverse of the two position covariances and solve for one offset.

Math. $d = \text{pos}_A - \text{pos}_B$, $W = (P_{A,\text{pos}} + P_{B,\text{pos}})^{-1}$, $\text{bias} = (\sum W)^{-1} \sum W d$, with `residual_spread` the RMS of $d - \text{bias}$ (`lib.rs:354-410`).

This variant and why. Position blocks only: "folding the velocity covariance in would weight a pair by how well its speed is known." Mismatched lengths are `UnmatchedTracks`. Translation only; no rotation or time bias.

Mission. Scenario 3's `isr_video` sensor carries `bias_m`; MT-06 step 2.

Verification. Hand-derived WLS with the bias injected by construction; 1e-3 against truth and 1e-6 against the oracle's estimator; the noiseless case recovers the injected bias exactly, and the noisy case is checked against the oracle only.

The bearing path

Bearing crossing and the single-bearing update (both built and gated; the single-bearing update signed 2026-09-06, the retained bearing's aging 2026-09-09, the crossing outside both signatures)

Problem. A direction finder gives an azimuth and no range. One degree at 1 km is 17 m across; at 30 km it is 520 m. DN-27's rule: "a bearing must never be turned into a position by assuming a range."

Idea. Two bearings may cross into a position with an honest covariance; one bearing may refine an existing track and can never initiate one.

Math. Crossing: the angle $\theta = \text{atan2}(|\text{cross}|, |\text{dot}|)$ is tested before any solve; Cramer's rule for the ray parameters; per-bearing cross-range $\sigma_1 = t_1 \sqrt{\text{var}_{\text{az},1}}$; Fisher information $J = \sum n_i n_i^T / \sigma_1^2$, $P = J^{-1}$, with $\det J = \sin^2 \theta / (r_1 \sigma_1 r_2 \sigma_2)^2$ (gungnir-coord/src/bearing.rs:183-315). Update: an extended update with a one-row BearingOnly model, $R = [\sigma_{\text{az}}^2]$, or two rows with elevation, gated at 99 percent, nearest gated neighbor (pipeline.rs:895-950).

This variant and why. Refuse below 15 degrees rather than widen the covariance: at 15 degrees "the long axis is already near four times the short one, and it doubles again by 7 degrees." Initiation is impossible by type: BearingDetection has no path to FusionPipeline::initiate, "not a flag to set, not a branch to take wrongly." cross_bearings has no non-test caller at this commit, and the gap register records that the 2026-09-06 signature "does not reach what DN-27 built and did not wire," naming cross_bearings among those things (docs/mission/gap-analysis/gap-register.md:254); it also lives in gungnir-coord, whose 2026-09-05 signature was given on the crate's gates rather than required by a tier, and that signature does not reach cross_bearings either.

Mission. MT-03 step 1 and the acoustic half of MT-01 step 1; no engineering scenario feeds bearings, the tests are synthetic.

Verification. Crossing: a closed form from the isosceles geometry with $\cot^2$ of the half-angle for the axis ratio; within 1e-6 m on six symmetric and one asymmetric geometry; refusal at the boundary to 1 part in 1e9 (tests/crossing.rs). Update: 100 consistent bearings from one fixed sensor initiate 0 tracks; the recovered cross-range variance matches $(r \sigma)^2$ to better than 1 percent at 2 km and 20 km through the scalar identity $1/P_{\text{post}} = 1/P_{\text{prior}} + 1/R$ (tests/bearing.rs), self-referential.

The pipeline

Out-of-sequence reorder buffer (built and gated, signed 2026-09-06)

Problem. Sensors with different latencies deliver detections out of measurement order.

Idea. Hold each detection until the newest source time seen is a horizon ahead of it, then process in source-time order; group detections within an epoch into one scan with one global assignment.

Math. reorder_horizon_s default 1.0, epoch_s default 0.05; each epoch predicts every track to scan time, gates, assigns, updates, initiates, ages. A detection older than the cursor is counted in PipelineStats::too_late and refused.

This variant and why. Retrodiction with stored filter histories "is the better algorithm for long latencies and it is not what is built." Cross-task state is "two channels and nothing else."

Mission. Scenario 3 (35 percent out-of-order video at 2.5 s latency); MT-01 step 2, MT-06 step 1.

Verification. Stone Soup's OOS updater was not run; the comparison is the same pipeline offline in source order against the async path in receipt order, criterion 1e-4, measured 0.0, non-vacuous because too_late == 0 is asserted (tests/oos_convergence.rs). Ten committed test-track sets replay the same way within 1e-6 with source-time back-jumps of 1.50 to 4.67 s observed (sample_set_replay.rs).

Dense-group mode (built, unsigned; off by default)

Problem. Under a raid the per-track path fragments and the operator needs a count and a shape.

Idea. When a scan exceeds MAX_DETECTIONS (12), run a PHD or CPHD beside the per-track filters and report a DenseGroupEstimate.

Math. The estimate carries a count, a cardinality distribution under CPHD, and unlabeled components; "no TrackId anywhere in it," asserted by exhaustive destructuring so adding one stops the test compiling.

This variant and why. The RFS output is never presented as tracks; that is the design's safety content. It is off by default for a measured reason: about 6 ms added per epoch for a 20-target raid and about 196 ms for 200, inline on the tokio executor, against a 4 ms frame budget; spawn_blocking is named as debt. Nothing draws it yet.

Mission. Scenario 4; MT-01 step 2 saturation.

Verification. Self-referential: track output identical field for field with the mode on and off over a 16-target raid; engages at 13 and not at 12; a 20-target raid counted as 21.1 (tests/dense_group.rs). The register says "written and gated, not signed."

PipelineSnapshot bundling and the retained bearing's lifetime (built and gated, signed 2026-09-09)

Problem. Tracks, retained bearings, stats and the dense-group estimate must be read from one instant, or the display shows an epoch skew.

Idea. Bundle every field onto one channel with no .await between reads.

Math. Not numerical; the proof is loom. Three positive models drive the real ingest_with loop, and a permanent negative model publishes over two channels the old way and must find the skew: it does not fire at preemption bound 0 (1 execution) and fires at 1, 2 and 3 (9, 36, 99 executions, hand-measured figures in the module doc). CI runs at bound 3 and fails any model that explored only one interleaving (gungnir-fusion-async/src/loom_model.rs:339-345 asserts explored > 1, on the grounds that "a model that explores one interleaving is not a model check").

This variant and why. loom's own mpsc does not model disconnection, so `sync.rs` models crossbeam's contract from loom primitives; a bug inside crossbeam itself is out of reach and the doc says so.

Mission. Scenario 4's snapshot row; PN-02 for MT-01 and MT-03.

Verification and the finding. The owner's review before signing found that a retained bearing's lifetime was never honored on screen: `expire_bearings` ran only when another bearing arrived and `poll` never read `valid_until`, so the last unmatched bearing from a quiet feed stayed drawn for the rest of the session while PN-08 said it was retained for `bearing_retention_s`. Fixed on both clocks: expiry on every submission and in `LiveTrackingService::poll(now)` (ARCHITECTURE.md section 10 item 115; tests named at table line 52).

Decision support

Bellman dynamic-programming allocation (built and gated, signed 2026-09-06)

Problem. Pair resources with tracks over several steps when a serviced track leaves the pool and a resource does not. "That asymmetry is the whole reason this is a dynamic program rather than one assignment problem repeated."

Idea. Exact value iteration over subsets of unserved tracks.

Math. $V(S, k) = \max \text{ over matchings } M \text{ within } S \text{ of } (\text{reward}(M) + V(S \setminus M, k + 1))$, $V(\cdot, \text{horizon}) = 0$; bitmask states, `MAX_TRACKS = 16` (2^{16} states), `MAX_RESOURCES = 8`, larger inputs refused "rather than switching quietly to a heuristic" (`bellman.rs:11-60`).

This variant and why, and the finding. The model has no time preference, so acting now and deferring tie, and enumeration order kept the fewest-pair matching: `solve_exact` "returned an empty first step for every input at every horizon above one, while reporting the full optimal value," so any node configured with the example default in `deploy/node/config.example.json`, `allocation_horizon: 10`, would have been handed a permanently empty plan that read as solved. Found by inspection, not by a test. Fix: on a tie within tolerance prefer more pairs, which "changes which optimal policy is reported, never what the optimum is." A second fix returns indices, not identifiers, after a version that wrapped positions in `ResourceId/TrackId` was type-correct and wrong wherever ids differed from positions.

Mission. MT-01 step 5, MT-02 step 5, through `DpInterceptService`.

Verification. A textbook Python DP with a different construction (`gen_allocation_fixtures.py`); exact value function to $1e-9$; 0.0 across six cases of at most 3 resources by 4 tracks at horizon 5 (`tests/bellman_diff.rs`); `tests/horizon_first_step.rs` pins the diagonal at horizons 1, 2, 3, 5, 10 and 16.

Threat scoring and closest point of approach (built, ungated)

Problem. Rank tracks against a prioritized asset list and say when each would arrive.

Idea. Proximity times priority times lethality, with time to impact only for an approaching track; predict at constant velocity or with the tracker's own filter.

Math. `closing = -v · rel/|rel|`; `proximity = clamp(1 - range/max_range, 0, 1)`; score halves when not approaching; $t^* = -\text{rel}_0 \cdot v / |v|^2$ clamped into `[0, horizon]` (`gungnir-assessment/src/lib.rs:58-97`, `prediction.rs:173-192`).

This variant and why. A stale track is not predicted at all and scores nothing; an empty asset list yields "no exposure and an unconfigured health state, never a zero score presented as real"; the predictor kind is named on every prediction because a CV prediction of a maneuvering drone "is wrong in a way the operator can compensate for only if they know."

Mission. MT-01 step 4, MT-04 step 4.

Verification. Section 2 rows with tests (`closest_approach_finds_the_analytic_minimum` to $1e-6$; MOP-28 monotonicity in `assets.rs`); ungated pending the walk.

CLEAR MOT metrics (built and gated, signed 2026-09-05)

Problem. Score a tracker against truth: misses, false positives, identity switches, precision.

Idea. Reproduce `motmetrics` stage by stage: carry forward matches inside the gate, assign the rest by minimum cost, count what is left.

Math. $\text{MOTA} = 1 - (\text{misses} + \text{switches} + \text{FP})/\text{objects}$, $\text{MOTP} = \sum \text{distance} / \text{detections}$; 0/0 is NaN as in `quiet_divide`, because a perfect score for an empty sequence "would be worse." Beyond-gate pairs are made unassignable by `motmetrics'` large-constant substitution, reproduced in `add_expensive_edges` with its derivation (`clear.rs:5-40`).

This variant and why. The scaffold's flat-slice signature "could not have been implemented as written": switches are undefinable on a snapshot.

Mission. The scoring tool for test-track runs; Scenario 4's counts; no operator thread step calls it.

Verification. py-motmetrics 1.4.0 within 1e-3; measured 2.2e-16 with every count exact (tests/motmetrics_diff.rs). Finding: purity is not a motmetrics metric; it is computed in the fixture from motmetrics' event frame, so that check is of the aggregation given an agreed matching.

Point clouds and geometry

Kabsch and point-to-plane ICP (Kabsch built, ungated; point-to-plane built, unit-tested, no table row)

Problem. Register one point cloud onto another.

Idea. Kabsch: center both clouds, take the SVD of the cross-covariance, read the rotation off it. Point-to-plane: minimize the distance along each target normal instead of to each point, which converges faster on smooth surfaces.

Math. $H = \sum s_i t_i^T = U \Sigma V^T$, $R = V U^T$, last column of V flipped when $\det(V U^T) < 0$ "so a reflection is never reported as a rotation" (transform_solve.rs). Point-to-plane: with $R = I + [\omega]_x$, the identity $n \cdot (\omega \times p) = \omega \cdot (p \times n)$ makes the residual linear in $x = [\omega; t]$, $a = [p \times n; n]$, normal equations $A x = b$ with $A = \sum a_i a_i^T$, ω mapped back by Rodrigues; refused below an eigenvalue ratio of 1e-6 (point_to_plane.rs:5-38).

This variant and why, and the finding. The surface two sibling test files share, $z = 0.4 \sin(1.3x + 0.7y)$, is degenerate for this solve specifically: the condition number came out above 1e17, recorded from an uncommitted numpy check, and a second sinusoid brings it to about 280. The GPU path is point-to-point only.

Mission. The point cloud under the picture for MT-09 step 2; no engineering scenario.

Verification. CPU ICP section 2 row at 0.05 m and 0.5 mrad, tested in-module; point-to-plane has 7 module tests recovering a known small transform to about 2 mm, CI-tested, no verification-table row. The GPU-vs-CPU tests are #[ignore]d behind gpu-tests; one hardware pass is the implementing agent's self-report.

PCA normals (built, unit-tested, no table row)

Problem. Estimate a surface normal at each point for the point-to-plane solve.

Idea. The unit eigenvector of the smallest eigenvalue of the k-nearest-neighbor covariance.

Math. Brute-force kNN, symmetric eigendecomposition (normals.rs:65-110).

This variant and why. Orientation is not resolved: "nothing here knows which way is 'outward'," and guessing by viewpoint is a step the function deliberately does not take. Coincident or collinear neighborhoods are Degenerate, not an arbitrary axis.

Mission. As ICP.

Verification. Unit tests in-module; no table row.

Line of sight, viewshed and coverage gaps (built, ungated)

Problem. What can each sensor see over terrain, where do the sensors overlap, and where is the sector uncovered.

Idea. Sample the segment to each point, compare against the nearest terrain vertex, count covering sensors per sample, report contiguous runs below threshold.

Math. Visible if every sample clears the nearest vertex height; covers when range is within $\max_range_m$ and elevation $\text{atan2}(d_u, \text{horizontal}) \geq \text{min_elevation_rad}$ (gungnir-analytics/src/lib.rs:32-97, coverage.rs).

This variant and why. Single-sensor coverage is GapSeverity::SingleSensor, not coverage: "one sensor gives a bearing and a range; it does not give a fusible track" (DN-12). Flat-terrain LOS is labeled optimistic.

Mission. MT-07 step 2, MT-09 step 2; TT-07's radar loss.

Verification. Section 2 tests including removing_a_sensor_never_shrinks_the_reported_gap_set, the monotonicity property verification-rows.md ranks second.

Anomaly detectors (built, ungated; DN-15)

Problem. Notice loitering, implausible kinematics, a transponder gone quiet, a feed gone silent, without deciding what any of it means.

Idea. Pure functions over primitive snapshots that return findings, never verdicts.

Math. Speed and climb rate against class envelopes; feed rate departure $|rate - baseline| / baseline > tolerance$ (anomaly.rs).

This variant and why. "No detector quarantines, drops, or downgrades anything"; one that cannot evaluate returns nothing. They take a primitive TrackSnapshot because typing them over TrackView would need the analytics -> model edge D-13 forbids. The node runs no detector; only the desktop does.

Mission. MT-05 step 2 (TT-05's AIS-off loiterer and spoofed coaster), MT-07 step 1.

Verification. Ten in-module tests plus gungnir-app/tests/anomalies.rs; section 2 row.

Geofence containment across the antimeridian (built, ungated)

Problem. Is a track inside a circular fence, including one straddling 180 degrees longitude.

Idea. Great-circle distance on angular differences rather than raw longitudes.

Math. Haversine, $\text{asin}(\sqrt{h})$ on the mean sphere, altitude ignored (gungnir-geo/src/lib.rs:40-64).

This variant and why. Only no_go fences deny; the hazard layer next to it is "descriptive, not a rule," and a source scan fails if gungnir-policy or gungnir-command ever names a hazard.

Mission. MT-03 step 4, MT-04 step 5; Scenario 5's geometry.

Verification. fence_straddling_the_antimeridian_contains_points_on_both_sides and tests/no_hazard_in_the_policy_chain.rs; section 2 row.

Time and data

Clock-skew least-offset estimator (built, ungated)

Problem. A sensor whose clock is wrong reports source times that make its detections land in the wrong epoch.

Idea. Per source, the smallest receipt-minus-source offset ever seen is the sample with the least transit delay in it, so it is closest to the clock error alone; "negative is the tell, because a message cannot arrive before it was sent."

Math. $\text{offset} = \text{receipt} - \text{source}$; out of sync when ahead beyond 0.25 s, or behind beyond the late-data policy's allowance (gungnir-time/src/lib.rs:89-190).

This variant and why. sources_observed exists so that "no skew across no sources" is never read as "every clock agrees."

Mission. MT-07 step 1 (TT-07's three skewed sensors); Scenario 3's timing disagreement.

Verification. Four unit tests; section 2 row.

ADS-B: CPR and CRC-24 gated by algebra (built and gated; consensus, not conformance)

Problem. Decode 1090ES squitters without a normative specification that is both free and permissively licensed.

Idea. Gate the decoder by agreement with two open-source Rust decoders, and gate the two parts that arithmetic can settle, parity and compact position reporting, by arithmetic.

Math. Parity: generator $0x01FF_F409$ over the frame; for address-carrying formats the residual is the ICAO address. CPR: 17-bit fields, $NZ = 15$, $NL(\text{lat}) = \text{floor}(2\pi / \text{acos}(1 - (1 - \cos(\pi/(2NZ))) / \cos^2(\text{lat})))$ clamped to $[1, 59]$; global decode from an even/odd pair with ZoneDisagreement meaning "wait for the next pair"; local decode refused past half a cell (adsb/cpr.rs).

This variant and why. The two oracles share pyModeS lineage, so a shared misreading would pass; that is why the CRC and CPR are not left to consensus.

Mission. MT-08 (TT-08's intermittent transponder), MT-02 step 5 friendly aircraft, MT-03 step 2.

Verification. rs1090 0.6.0 and adsb_deku 0.7.1 over two vendored public captures -- an AVR prefix of adsb_deku's Los Angeles corpus and dump1090's modes1.bin IQ recording, demodulated in this build: address agreement on all 13,324 AVR squitters and 141 IQ-demodulated frames; field agreement on the 10,310 + 141 frames whose type codes this build interprets; 3,014 frames carried raw. Three oracle defects named (seven-character callsign, u16 altitude, no velocity without vertical rate). CRC: linearity, every single-bit error, every burst to 24 bits, only the generator undetectable at 25. CPR: dump1090's cprtests.c vectors at the 1e-6 degree tolerance that file states; the 1e-9 the table records is an observation, not the gate.

ASTERIX Categories 048, 034, 205 and 129 (built, ungated; 205 adapter signed 2026-09-09, 129 adapter unsigned)

Problem. Read the radar, service, direction-finder and UAS-identification messages real sensors emit.

Idea. One bounds-checked framing layer naming the byte offset of every failure; per-category decoders that carry uninterpreted items raw and write every conversion loss to provenance.

Math. Cat 048 polar $I048/040$ in 1/256 NM and azimuth clockwise from north, placed relative to the site's ENU origin; Cat 205 bearings become Measurement::Bearing with the site's configured accuracy.

This variant and why, and the finding. Cat 205 edition 1.0's own Table 1 lists $I205/070$ and $I205/080$ at 0.1 degrees while its item definitions state 0.01; "the item definitions govern; this decoder follows them," and the 2026-09-09 review added range refusals ($0 \leq \text{THETA} < 360$, $-90 \leq \text{ELEVATION} \leq 90$) that nothing downstream would otherwise enforce. Cat 129 was built from the primary PDF converted to text twice by independent means, since no machine-readable cross-check exists. Encode for 048 and 205, and all of STANAG 4676, return NotImplemented. Of the four, only the 205 adapter arm carries an owner signature; ARCHITECTURE.md section 10 records the 048/034 landing with the editions "pinned by the owner" but no signature statement, and the register says the 129 adapter is neither reviewed nor signed.

Mission. MT-01 step 1 (048/034), MT-03 step 1 (205/129).

Verification. 048/034 against one public Croatia Control capture; 205 and 129 against hand-built records from the specifications' byte tables, since no capture exists anywhere; the asterix_feed fuzz target ran 25,923,346 executions clean. No thread step has seen a live feed.

MISB ST 0601 KLV with bounded BER lengths (built, ungated; ingest adapter signed 2026-09-09)

Problem. Read ISR video metadata so a placeable fix becomes a detection.

Idea. A 16-byte universal label, a BER short- or long-form length, one-byte local-set tags; interpret 17 tags and carry the rest raw.

Math. A long-form length of zero bytes, or one larger than the frame, is an error; the checksum-discard rule is enforced and counted.

This variant and why, and the findings. Tag semantics come from paretech/klvdata, a secondary source, because the primary text sits behind a bot gateway. The vendored worked example's stated checksum 0xAA43 does not close (computed 0x3E1E, every plausible byte range tried), recorded on checksum_valid. In review before signing, "a single corrupt length stalled the feed for good -- twenty valid frames queued behind a header claiming four gigabytes produced nothing across two hundred polls"; now bounded at MAX_FRAME_BYTES = 65,535, counted and resynchronized past.

Mission. MT-06 step 1.

Verification. tests/misb0601_fixtures.rs and adapter tests against that one fixture and hand-built frames; no live or recorded UAS stream has been through it.

Identity similarity correlation (built, ungated)

Problem. Is this session's new track the same entity an earlier session saw.

Idea. Propagate each known lineage's last state at constant velocity across the gap, measure the normalized distance, discount for class disagreement, merge above a threshold, and record every merge with its basis.

Math. $\text{distance_sq} = \sum_i (p_i - \text{predicted}_i)^2 / (\text{var}_{\text{last},i} + \text{var}_{\text{cand},i} + q \text{ gap_s})$ over the three position axes; $\text{kinematic} = \exp(-0.5 \text{ distance_sq})$; $\text{confidence} = \text{kinematic}$ if classes agree (Unknown agrees with anything) else 0.1 kinematic; merge at 0.5 (similarity.rs:90-133).

This variant and why. Never merge on class alone, "because two hostiles are not one hostile"; only within [min_gap_s, max_gap_s].

Mission. MT-06 step 3 (TT-06's convoy through a 20-minute cover gap), MT-08 step 5.

Verification. gungnir-app/tests/order_of_battle.rs; section 2 row.

Reconciliation by mission time (built, ungated)

Problem. After a link outage, the desktop and the node each hold a journal; merge them without losing or duplicating a decision.

Idea. Concatenate, sort by mission time, drop an envelope whose time and event both match one already merged, and report conflicts rather than resolve them.

Math. A DecisionConflict is the same PlanId decided differently (gungnir-resilience/src/lib.rs:97-133).

This variant and why. D-15 made resolution a person's act, so the desktop never switches back on its own. RoleRankArbiter (higher role wins, earlier wins a tie, expiry checked first) is the locked default: D-03 locked the arbitration rule on 2026-09-04 and the expiry-first amendment was signed 2026-09-05. gungnir-resilience/src/lib.rs:11 still calls it "not yet locked" -- a stale doc comment, named here rather than repeated.

Mission. MT-10 steps 4 and 5 (TT-10's 660-second outage); Scenario 4.

Verification. Unit tests and gungnir-app/tests/failover_e2e.rs over a real in-process node and real GET /v2/history; the cross-layer row's criterion still says "resolved by the arbitration rule" pending the walk, and the 100,000-entry outbox bound is never filled by a test.

The honest answer, if asked what runs: a Joseph-form CV Kalman filter or the CV/CT IMM, a chi-square gate, Jonker-Volgenant assignment and the Stone-Soup-matched lifecycle over a reorder buffer, with the PHD or CPHD count beside it if the dense-group mode is switched on -- it is off by default, and nothing draws its output yet. Of everything else in this chapter, the rows with a running gate in section 1 of docs/verification-capability-table.md -- estimation, association, the RFS filters that are built, track management and fusion, the bearing path, the pipeline, allocation, metrics, and the ADS-B parity and position decode -- are each waiting for their own increment; most are gated against a checked-in fixture, MHT and the bearing crossing against hand-derived scenes. The rest carry unit tests against a section 2 row, or no row at all, and say so in their headings.

Part VI, Chapter 14 -- Process, defects, what is not built, and likely questions

This is the honesty chapter: what happened when, what a signature means and who has one, what the process caught in its own work, what is not built, and the answers to the questions a reviewer who opens github.com/WayneRoessling/gungnir will ask. Every status word is the repository's own at main ee12f7e (2026-09-09).

14.1 Timeline

State the provenance before a reviewer finds it: the public git history runs 2026-09-07 to 2026-09-09, 250 commits all authored by Wayne Roessling, 162 carrying a Co-Author-By: Claude trailer, 84 merged pull requests (`git log, gh pr list`). The documented program runs from 2026-09-04; the three earlier days have no version control, and their dates are the documents' own (`docs/plans/README.md`, `ARCHITECTURE.md` section 10).

Date	What landed	Record
2026-09-04	Eleven plans confirmed; D-01 to D-17 resolved with the owner, D-16 walked measure by measure; ten plans reach first draft the same day (13 mission documents, a 56-leaf capability taxonomy, a 76-gap register, 62 UAF views then, the TOGAF set); budgets confirmed as provisional gates (D-04)	<code>docs/plans/README.md</code> ; <code>docs/mission/gap-analysis/decisions-needed.md</code>
2026-09-05	Plan 11: 22 design notes written and implemented the same day, 340 passing tests; D-18 to D-22 (transport, docking, cryptography crates); first implementation tranche, section 10 items 27 to 79 (transforms, motion models, scenario generator, first tracking gates, journal durability); mutual TLS and journal sealing (item 64, GAP-060, with DN-22 amendment 1 and D-22 signed the same day); 17 TOGAF principles and 17 contracts signed	<code>ARCHITECTURE.md</code> section 10; <code>docs/architecture/togaf/README.md</code>
2026-09-06	Items 80 to 101 in eight batches of ten to fifteen gaps; Area A, the pipeline and the allocator, <code>PIPELINE_IMPLEMENTED</code> true (item 92); its remaining mathematics, IMM, particle, square-root, JPDA, MHT, track fusion, registration, GM-PHD (item 94); D-23 to D-33; the theme review (item 89, D-35 to D-38)	<code>ARCHITECTURE.md</code> section 10
2026-09-07	Initial commit eb89a93; repository hosted (c4b8c49, GAP-061); AGPL-3.0-or-later licensing, D-34 (PRs #1 and #5, cargo deny passing for the first time); CI made honest (Gate 6 able to fail, GAP-092 and GAP-093); a development-status review rewriting section 10 and the README against the tree; 76 commits; PRs #1 to #15 merged (<code>gh merge times, UTC -- git log's local dates</code> put #16 and #17 on the 7th as well)	<code>git log; gh pr list</code> ; <code>docs/release-governance.md</code>
2026-09-08	134 commits; PRs #16 to #45, #47 and #48 merged (<code>gh merge times, UTC</code> ; the commit count is <code>git log's local date</code>); parallel gap batches (exchange write path, OS keystore, GAP-096 bearings, GM-CPHD and LMB, MISB, Cat 205, Cat 129, GPU ICP, the ort runtime, night theme); GAP-061 finds three CI gates that had checked nothing (f276088, merged 2026-09-09 as PR #61); the repaired fuzz gate finds the <code>-inf</code> total on its first working run; the OS-keystore trio signed (PR #63)	<code>git log; gh pr list</code> ; register GAP-061, GAP-103
2026-09-09	40 commits; PRs #46 and #49 to #84 merged (<code>gh merge times, UTC</code> ; the commit count is <code>git log's local date</code>); D-43 signed (PR #81); signature queue items 109 (PR #80), 113 and 114 (PR #83), 115 (PR #84); CI critical-path work (PRs #78, #82); <code>gen_gaps.py</code> duplicate-id guard (PR #76); the tip ee12f7e is the merge of PR #84	<code>git log; gh pr list</code>

The citable test figure is 1,863 passed and 3 skipped in CI run 34361396023 on ce609437 (2026-09-09, cargo-nextest, 112.7 s). The run on the tip ee12f7e is red on one gungnir-node provisioning-harness race (BrokenPipe at `gungnir-node/tests/account_provisioning.rs:50`), a test-harness defect rather than a product one.

14.2 Trust tiers and what "signed by the owner" means

`docs/agent-ic-workflow.md` defines three tiers. The low-trust tier, human-owned, where agents may draft but not merge unsupervised, is: any unsafe block anywhere; concurrency correctness in `gungnir-fusion-async`; numerical-stability guarantees, applied in practice to `gungnir-filters`, `gungnir-association`, `gungnir-track-fusion`, `gungnir-rfs` and `gungnir-allocation`; the recommend-versus-act boundary in `gungnir-policy` and `gungnir-command`; `gungnir-security`; the `gungnir-ingest` gateway; `gungnir-remote/src/identity.rs` (added by the owner 2026-09-06, scoped to the path); `gungnir-api` write endpoints; and any change to a pass criterion in `docs/verification-capability-table.md`. Every change passes an implementer agent, a reviewer agent with a fixed

adversarial checklist (numerical stability, no unexplained `unwrap()`, no undrawn dependency edge, "wiring not logic" in the binaries, no health flag claiming what does not work), and a verifier gate that "cannot be waived by either agent"; the low-trust tier then waits for the owner.

Mechanically a signature is a dated sentence in the ARCHITECTURE.md section 10 item and the register entry, often in a commit whose subject ends "sign item N" (c10d522, 3932003, 0b4f4e3) and otherwise in one whose subject says "signed by the owner" with the date (ac95e22, bec37c3). Three precedents define the edge: the PolicyChain lifetime parameter (2026-09-05) touched no verdict logic and was brought to the owner anyway; D-43, where "what the owner decided was the contract, not the fix"; and GAP-057, where "what was signed is the blocker, not the answer." The three records drift, and a reviewer will see it: `gungnir-security/src/os_keystore.rs`, `account_store.rs`, `keystore.rs` and `gungnir-remote/src/identity.rs` still read "not signed" although section 10 items 103, 104 and 111 carry the 2026-09-08 signature, and the register's GAP-015, GAP-057, GAP-060, GAP-096 and GAP-100 entries lag the same way -- GAP-015 still calls the GM-CPHD "written and gated, not signed" and GAP-096 says the same of PipelineSnapshot, although section 10 items 109 and 115 carry the 2026-09-09 signature and `gungnir-fusion-async/src/lib.rs:127` now reads "Signed by the owner 2026-09-09." Section 10 is the later record.

Item	Subject	Status as of 2026-09-09
102	Exchange write path (PR #20) and the Supervisor RELEASE_PRODUCT/PUBLISH_EXCHANGE grant it flagged (PR #22)	signed 2026-09-08
103, 104, 111	OS keystore, node account store, keystore-issued TLS identity (PR #63)	signed 2026-09-08; a first-run <code>ensure_secret</code> race found in review, which the code says it narrows and section 10 says it closed
106	SAPIENT TcpTaskSink (PR #80)	signed 2026-09-08
109	GM-CPHD (PR #80)	signed 2026-09-09
113, 114	MISB ST 0601 adapter; ASTERIX Cat 205 adapter (PR #83)	signed 2026-09-09
115	PipelineSnapshot bundling and the retained bearing's lifetime (PR #84)	signed 2026-09-09
122	D-43, Assignment::total_cost: Option<f64> (PR #81)	signed 2026-09-09
none	LMB filter (<code>gungnir-rfs/src/lib.rs:25</code>); dense-group wiring in <code>gungnir-fusion-async</code> ; DN-30 measurement noise from the baseline; ManagedService cloud KMS; Cat 129 adapter (GAP-101); the fires three-state friendly check (GAP-090, <code>gungnir-policy/src/fires.rs</code>); the loom model checks (GAP-061)	built, unsigned
none	DN-29 third IMM mode; DN-25's Cursor-on-Target codec half	design only, no code

14.3 Defects the process caught in its own work

#	Defect, and what was done	Record
1	<code>AppState::save_session</code> called <code>journal.sync()</code> without draining the event bus, an <code>fsync</code> of a file the envelope had never reached. Fixed 2026-09-05 as drain, then sync.	GAP-005; section 10 item 66
2	A mutual-TLS test passed while admitting anonymous clients: under TLS 1.3 the client finishes before the server validates the certificate. The test now makes a request; the implementation had been right.	item 64; <code>gungnir-api/tests/mutual_tls.rs</code>
3	<code>read_session</code> 's tolerance for a torn final line swallowed a sealing failure, so a journal with a missing key returned an empty session. A sealing error is never treated as a torn line now.	item 64; <code>gungnir-store</code>
4	The allocator returned an empty first step at every horizon above one while reporting the full optimum; with <code>deploy/node/config.example.json</code> 's <code>allocation_horizon: 10</code> the desktop's plan was permanently empty and read as solved. Found by inspection; pinned at horizons up to 16.	GAP-029; <code>gungnir-allocation/tests/horizon_first_step.rs</code>
5	Three failover failures were read as slowness and the deadline raised from 5 s to 10 s (PR #36); PR #79 showed "connected is not subscribed," a pure ordering race losing 1 envelope in 25. The fix waits on <code>subscriber_count()</code> ; the deadline returned to 5 s.	<code>gungnir-app/tests/failover_e2e.rs</code>
6	<code>TcpTaskSink::send</code> treated a transient <code>WouldBlock</code> as a dead connection, leaving a newline-less fragment that corrupted the next task's framing; exhaustion appeared only with many discrete writes. Closed with a bounded retry before signing.	item 106; <code>gungnir-ingest/src/adapters/sapient.rs</code>

#	Defect, and what was done	Record
7	The CPHD's leave-one-out elementary-symmetric-function recurrence was unstable when one likelihood dominates, returning +1e6 where the truth is 1e-36, in the crate and in its oracle generator alike. Replaced by direct $O(m^3)$ recomputation; the failing recurrence is kept as a negative test on both sides.	item 109; gungnir-rfs/src/lib.rs
8	An unchanged plan minted a fresh PlanId every tick and flooded its own approval queue, found while reviewing the usability script rather than the crate. A fifty-tick test now pins one plan.	GAP-097; item 110
9	Three CI gates had never checked anything: oracle-diff ran an empty crate 12 times, loom model-checked zero interleavings 22 times, fuzz-nightly failed before fuzzing a byte. The first two gained anti-vacuous assertions; the third was fixed by --fuzz-dir and the nightly toolchain.	GAP-061; PR #61
10	Gate 6 had never saved a baseline because github.ref is refs/pull/<n>/merge on a pull request. Once able to fail it failed on runner variance, so it is advisory with the 10 percent threshold unchanged.	GAP-093; bench-regression.yml
11	The repaired fuzz gate found solve_assignment returning Ok with total_cost = -inf on an all-finite 5x2 matrix. The contract became Option<f64> (D-43); the 200,000-matrix characterization is recorded in the docs and not reproducible from the repository.	GAP-103; gungnir-association/src/assignment.rs
12	Both binaries filled the ENU-meter sensor map straight from geodetic radians, so a sensor at 55 N 12 E sat 0.98 m from the origin instead of 1.7 km, visible only once bearing wedges were drawn. Fixed by a typed SensorPositions::from_geodetic constructor; the untyped from_sensors stays public.	GAP-104; item 121
13	A dependency claim, "quick-xml enters only through vtkio," was checked with cargo tree on Windows and was false on Linux, where zbus_xml brings it in. The advisory acceptance was withdrawn.	GAP-094; deny.toml header
14	Found against a real GPU adapter the implementing agent's own sandbox had (an RTX 5060 Ti, self-reported, no log in the repository), the spatial hash's default cell size crowded a 36-point cloud into one or two cells and dropped points; separately, eager GpuContext::new cost three 0.01 s tests over 300 CPU-seconds. Both fixed; resolution is lazy (ac47b64).	item 117; gungnir-app/src/fusion.rs
15	A retained bearing's lifetime was never honored on screen: expire_bearings ran only when another bearing arrived and poll never read valid_until. Found in the owner's review before signing; fixed on both clocks.	item 115; c10d522
16	The Stone Soup GM-PHD first-scan difference was the generator's own transcription error, births stamped one scan in the past. The worst disagreement fell from 0.842 to 0.139 because an error left the comparison, not because the library became an oracle.	GAP-015; gen_phd_fixtures.py

14.4 What is not built

Designed-and-unbuilt means a design note or decision exists and the code refuses by name; unplanned means no design and no survey.

Capability	Status	Kind
STANAG 4676 codec	not built: Stanag4676Codec returns InteropError::NotImplemented both ways (gungnir-interop/src/lib.rs); GAP-064 Open; the owner declined to purchase the paywalled specification	designed, blocked on the specification
Cursor-on-Target codec (DN-25, GAP-091)	not built: schema and TAK protobuf framing pinned at atak-civ 5.5.1.8, the recorder testdata/cot/tools/record_cot.py present, nothing recorded, no Rust codec	designed and unbuilt
EO/IR	not built: no survey and no pinned specification; docs/design/external-standards.md names it nowhere	unplanned
ISR video decode and display	not built by decision; only the MISB ST 0601 KLV metadata half is built and signed, 17 of the standard's 90-plus tags	deliberately out of scope
Full delta-GLMB	not built: GlmbFilter::labelled_tracks returns a named NotImplemented citing the missing hypothesis-truncation scheme; the LMB is built, unsigned	designed refusal

Capability	Status	Kind
Trained models (GAP-080) and journal-row extraction (GAP-079)	not built: no model trained, exported, loaded or evaluated; the ort seam compiles behind a default-off feature and has never executed a graph; GAP-079 In progress with the journal rows remaining	designed and unbuilt
Node-side identity and evidence	cross-session correlation is built (edge (s), gungnir-node/src/entities.rs) although the section 10 Open bullet still says no edge exists, a documentation defect; node-side cooperative evidence fusion has no edge to gungnir-identification, a decision left open	half built, half undecided
Key escrow	built and gated (D-27, gungnir-security/src/asymmetric.rs, signed 2026-09-06), contrary to the DN-22 README row; what is not built is a real cloud KMS round trip: ManagedService is built and unsigned with its SDK tests #[ignore]d for lack of credentials	the cloud half is unbuilt
Also not built or never run	gRPC (tonic and prost pinned, used by no member, D-21); the PN-19 assistant (GAP-044 Planned); 3D Tiles streaming; Cat 048 and Cat 205 encode; the disconnected-delegation expiry half of D-15, not found in code; IFF (deferred, D-09); the GAP-067 row promotion in the verification table and the register (the owner's walk ran 2026-09-07; neither document was updated to match); the usability round (no session has run); release.yml (never run); the gpu-fusion.yml dispatch (queued, never completed); every MATLAB oracle column; any real sensor or deployment	mixed

14.5 Fifteen likely questions

Why not an existing tracking library? The libraries were used as oracles, not dependencies: filterpy 1.4.5, Stone Soup 1.9.1, scipy 1.18.1, pymap3d 3.2.0 and py-motmetrics 1.4.0, pinned and stamped into every fixture. They also ran out where the table went: Stone Soup 1.9.1 has no IMM at all, no MHT hypothesis generator, no CPHD updater and no GLMB or LMB, and its GM-PHD disagrees for reasons recorded per row: merge_components clamps a merged weight to 1.0, which is the divergence from scan 3; it applies prob_survival inside the update; and the first-scan gap was the fixture generator's own transcription error.

How do you know the filters are right? Each section 1 row names the method actually run -- a pinned library oracle, a hand derivation, or an explicitly self-referential or truth-scored check -- with a criterion and a recorded worst disagreement: linear KF 9.1e-13 against filterpy, transforms 2.1e-9 m against pymap3d, JV assignment 1.9e-16 against scipy. The limits are recorded too: filterpy uses the same Joseph form, so that row is parity, not an independent method; the UKF geometry stays in one quadrant; CPHD and LMB rest on in-house derivations; no MATLAB column ever ran; no real sensor.

What is broken right now? CI on the tip ee12f7e is red on a harness race in gungnir-node/tests/account_provisioning.rs:50, so the citable green run is 34361396023. The repository also contradicts itself: docs/architecture.md says the GAP-067 walk promoted section 2 rows while the verification table still says "not gates yet" and the register keeps GAP-067 Open; Cargo.toml says rcgen is never a normal dependency while gungnir-remote depends on it; DN-22 section 14 says "once per rotation" while rotate() makes no service call.

Why AGPL? D-34: under Apache-2.0 or MIT a competitor could run a modified gungnir-node as a hosted service, never convey a binary, and owe nothing, and ARCHITECTURE.md section 8 makes that shape the normal deployment; AGPL section 13 closes it. The section 7 terms govern credit and identity only; a commercial license and a CLA sit beside it; the PN-21 about panel is a licensing surface tested against NOTICE. CLA.md states it has not been reviewed by counsel.

Why does a solo project count as architecture leadership? It is not a record of leading people. It is the record of the standards a team would work under, written and enforced first: two coding standards, 17 principles and 17 contracts signed 2026-09-05 with 7 machine-checked on every push, a review pipeline with a verifier gate no agent can waive, 43 decisions with outcomes and reasoning recorded, and a governance document that calls the board of one "a governance weakness rather than a simplification."

How was AI-assisted development governed? By docs/agent-ic-workflow.md: human-authored specs and pass criteria, agent drafts, an adversarial reviewer pass, structural gates (the dependency-graph test, the unwrap scan, the citation resolver, the generator drift jobs), and the owner's signature on the human-owned tier. 162 of 250 commits carry a Co-Authored-By: Claude trailer. The owner's reviews found defects the agent's tests had missed: the ESF instability, the WouldBlock fragmentation, the bearing lifetime, the Option<f64> contract.

What would you do first with a team? Break the board of one: put the seven unsigned human-owned items in front of two reviewers, then reconcile the three records that drift, section 10, the register and the crate doc comments, and record the GAP-067 row promotion the owner already walked in the two documents that never took it. Second, real data: no Cat 205,

Cat 129, SAPIENT, MISB or CoT capture exists, and no sensor has ever fed the pipeline. Third, run the fourteen-task usability round that is scripted and has not run.

Has this ever run against a real sensor? No. Every gate is synthetic or a public capture: the Croatia Control ASTERIX pcap, adsb_deku's Los Angeles AVR corpus, dump1090's IQ recording, gpsd AIS logs. The Cat 205 and Cat 129 fixtures are hand-built from the specifications' byte tables; SAPIENT reads the protobuf-JSON mapping over TCP against vendored samples and a loopback fixture; the scenario crate generates everything else at a fictional estuary.

The running pipeline is a constant-velocity Kalman filter with GNN. Where is the rest? Gated in isolation and not selectable from a baseline: the only two `FilterSelection` variants are the constant-velocity Kalman filter and the CV/CT IMM, and `IMPLEMENTED_FILTERS` accepts only baseline names for those two (kf-cv, linear-kf and constant-velocity for the first, imm-cv-ct for the second), so EKF, UKF, particle and square-root forms are library types (DN-28 section 6); JPDA and MHT have no pipeline caller; track-to-track fusion is not run, so two sensors at one instant make two tracks; the RFS filters run only as the off-by-default dense-group count.

Your Stone Soup oracle disagrees with your PHD, and you have no library oracle for CPHD, LMB, IMM or MHT. True, and recorded per row. The PHD disagreement is explained: the weight clamp, survival applied in the update, and a fixture transcription error. CPHD and LMB rest on in-house derivations checked against brute-force enumeration and closed-form identities; the IMM uses filterpy's `IMMEstimator` as a recorded substitute; the MHT crossing test asserts only the post-separation history and does not demonstrate resolution.

Is the GPU path verified? Four WGSF kernels are naga-validated in plain cargo test; the GPU-versus-CPU tests are `#[ignore]`d behind `gpu-tests`; one pass on an RTX 5060 Ti is the implementing agent's self-report in section 10 item 117, with no log in the repository; the registered self-hosted runner has never completed a dispatch. The path is point-to-point only, and `fuse_voxels` has no oracle and no caller.

Has a node in a container ever talked to a desktop over a network? No. The container image is built and smoke-run in CI; failover and mutual TLS are exercised in-process over loopback with `rcgen` certificates; `release.yml` has never run; no cloud KMS round trip has been made; the PROJ conversion has run only in the Linux CI job, never on the author's machine.

Two channels and loom, but the outbound channel is unbounded and the dense-group update runs inline on the executor. Both are stated in the code rather than hidden. A consumer that stops polling accumulates full snapshots. The dense-group PHD adds about 6 ms per epoch at 20 targets and about 196 ms at 200 against a 4 ms frame budget, which is why the mode is off by default and `spawn_blocking` is named as `debt` in `docs/performance-budgets.md`.

The verification table says section 2 rows are not gates yet, but docs/architecture.md says the walk is done. Both are in the repository. The tests exist and run on every CI push; `docs/architecture.md` records the owner's 2026-09-07 walk promoting rows to Specified; the table's section 2 preamble and the register, GAP-067 Open, were not updated to match. It is a documentation defect found while preparing this guide, not one a reviewer has to discover.

Zero unsafe and zero unwrap(): enforced how? Measured, not forbidden. No `forbid(unsafe_code)` exists; a `grep` of 399 files finds the word once, in a doc comment. `miri.yml` triggers on any added unsafe token but runs miri over 12 tracking and service crates only. The `unwrap` scan is a line-based text test in `gungnir-app/tests/architecture_compliance.rs`, exempting tests, main, benches, examples, fuzz targets and the verifier crates; `todo!()` is scanned tree-wide.